Parte V

Administración de memoria

Capítulo 30

Funciones y operaciones

El único espacio de almacenamiento que el procesador puede utilizar directamente, más allá de los registros (que si bien le son internos y sumamente rápidos, son de capacidad demasiado limitada) es la memoria física. Todas las arquitecturas de procesador tienen instrucciones para interactuar con la memoria, pero ninguna lo tiene para hacerlo con medios *persistentes* de almacenamiento, como las unidades de disco. Cabe mencionar que cuando se encuentre en un texto referencia al *almacenamiento primario* siempre se referirá a la memoria, mientras que el *almacenamiento secundario* se refiere a los discos u otros medios de almacenamiento persistente.

Todos los programas a ejecutar deben cargarse a la memoria del sistema antes de ser utilizados. En este capítulo se mostrará cómo el sistema operativo administra la memoria para permitir que varios procesos la compartan — esta tarea debe preverse desde el proceso de compilación de los programas (en particular, la fase de *ligado*). Hoy en día, además, casi todos los sistemas operativos emplean implementaciones que requieren de hardware especializado: la *Unidad de Manejo de Memoria* (MMU, por sus siglas en inglés). En el transcurso de este capítulo se describirá cómo se manejaban los sistemas multitarea antes de la universalización de las MMU, y qué papel juegan hoy en día.

En esta primer sección se presentarán algunos conceptos base que se emplearán en las secciones subsecuentes.

30.1. Espacio de direccionamiento

La memoria está estructurada como un arreglo direccionable de bytes. Esto es, al solicitar el contenido de una dirección específica de memoria, el hardware entregará un byte (8 bits), y no menos. Si se requiere hacer una operación sobre *bits* específicos, se deberá solicitar y almacenar bytes enteros. En algunas arquitecturas, el *tamaño de palabra* es mayor —por ejemplo, los procesadores Alpha incurrían en *fallas de alineación* si se solicitaba una dirección de memoria no alineada a 64 bits, y toda llamada a direcciones *mal alineadas* tenía que ser *atrapada* por el sistema operativo, re-alineada, y entregada.

Un procesador que soporta un *espacio de direccionamiento* de 16 bits puede referirse *directamente* a hasta 2^{16} bytes, esto es, hasta 65 536 bytes (64 KB). Estos procesadores fueron comunes en las décadas de 1970 y 1980 — los más conocidos incluyen al Intel 8080 y 8085, Zilog Z80, MOS 6502 y 6510, y Motorola 6800. Hay que recalcar que estos procesadores son reconocidos como procesadores de *8 bits*, pero con *espacio de direccionamiento* de 16 bits. El procesador empleado en las primeras PC, el Intel 8086, manejaba un direccionamiento de 20 bits (hasta 1 024 KB), pero al ser una arquitectura *real* de 16 bits requería del empleo de *segmentación* para alcanzar toda su memoria.

Con la llegada de la era de las *computadoras personales*, diversos fabricantes introdujeron, a mediados de los años 1980, los procesadores con arquitectura de 32 bits. Por ejemplo, X86-32 de Intel tiene su inicio oficial con el procesador 80386 (o simplemente 386). Este procesador podía referenciar desde el punto de vista teórico hasta 2^{32} bytes (4 GB) de RAM. No obstante, debido a las limitaciones tecnológicas (y tal vez estratégicas) para producir memorias con esta capacidad, tomó más de 20 años para que las memorias ampliamente disponibles alcanzaran dicha capacidad.

Hoy en día, los procesadores dominantes son de 32 o 64 bits. En el caso de los procesadores de 32 bits, sus registros pueden referenciar hasta 4 294 967 296 bytes (4 GB) de RAM, que está ya dentro de los parámetros de lo esperable hoy en día. Una arquitectura de 32 bits sin extensiones adicionales no puede emplear una memoria de mayor capacidad. No obstante, a través de un mecanismo llamado PAE (extensión de direcciónes físicas, *Physical Address Extension*) permite extender esto a rangos de hasta 2^{52} bytes a cambio de un nivel más de indirección.

Un procesador de 64 bits podría direccionar hasta 18 446 744 073 709 551 616 bytes (16 Exabytes). Los procesadores comercialmente hoy en día no ofrecen esta capacidad de direccionamiento principalmente por un criterio económico: al resultar tan poco probable que exista un sistema con estas capacidades, los chips actuales están limitados a entre 2^{40} y 2^{48} bits — 1 y 256 terabytes. Esta restricción debe seguir teniendo sentido económico por muchos años aún.

30.2. Hardware: la unidad de manejo de memoria (MMU)

Con la introducción de sistemas multitarea, es decir, donde se tienen dos o más programas ejecutandose, apareció la necesidad de tener más de un programa cargado en memoria. Esto conlleva que el sistema operativo, contando con información de los programas a ejecutar, debe resolver cómo ubicar los programas en la memoria física disponible.

A medida que los sistemas operativos y los programas crecieron en tamaño, fue necesario *abstraer* el espacio de almacenamiento para dar la ilusión de contar con más memoria de la que está directamente disponible. Con asistencia del hardware, es posible configurar un espacio lineal contiguo para cada proceso -y para el mismo sistema operativo que se *proyecta* a memoria física y a un almacenamiento secundario.

Se explicará cómo la MMU cubre estas necesidades y qué mecanismos emplea para lograrlo —y qué cuidados se deben observar, incluso como programadores de aplicaciones en lenguajes de alto nivel, para aprovechar de la mejor manera estas funciones (y evitar, por el contrario, que los programas se vuelvan lentos por no manejar la memoria correctamente).

La MMU es también la encargada de verificar que un proceso no tenga acceso a leer o modificar los datos de otro —si el sistema operativo tuviera que verificar cada una de las instrucciones ejecutadas por un programa para evitar errores en el acceso a la memoria, la penalización en velocidad sería demasiado severa.[1]

Una primera aproximación a la protección de acceso se implementa usando un *registro base* y un *registro límite*: si la arquitectura ofrece dos registros del procesador que sólo pueden ser modificados por el sistema operativo (esto es, el hardware define la modificación de dichos registros como una operación privilegiada

[1] Y de hecho está demostrado que no puede garantizarse que una verificación estática sea suficientemente exhaustiva

que requiere estar ejecutando en *modo supervisor*), la MMU puede comparar sin penalidad *cada acceso a memoria* para verificar que esté en el rango permitido.

Por ejemplo, si a un proceso le fue asignado un espacio de memoria de 64 KB (65 536 bytes) a partir de la dirección 503 808 (492 KB), el *registro base* contendría 503 808, y el *registro límite* 65 536. Si hubiera una instrucción por parte de dicho proceso que solicitara una dirección menor a 503 808 o mayor a 569 344 (556 KB), la MMU lanzaría una excepción o *trampa* interrumpiendo el procesamiento, e indicando al sistema operativo que ocurrió una *violación de segmento* (*segmentation fault*).[2] El sistema operativo entonces procedería a terminar la ejecución del proceso, reclamando todos los recursos que tuviera asignados y notificando a su usuario.

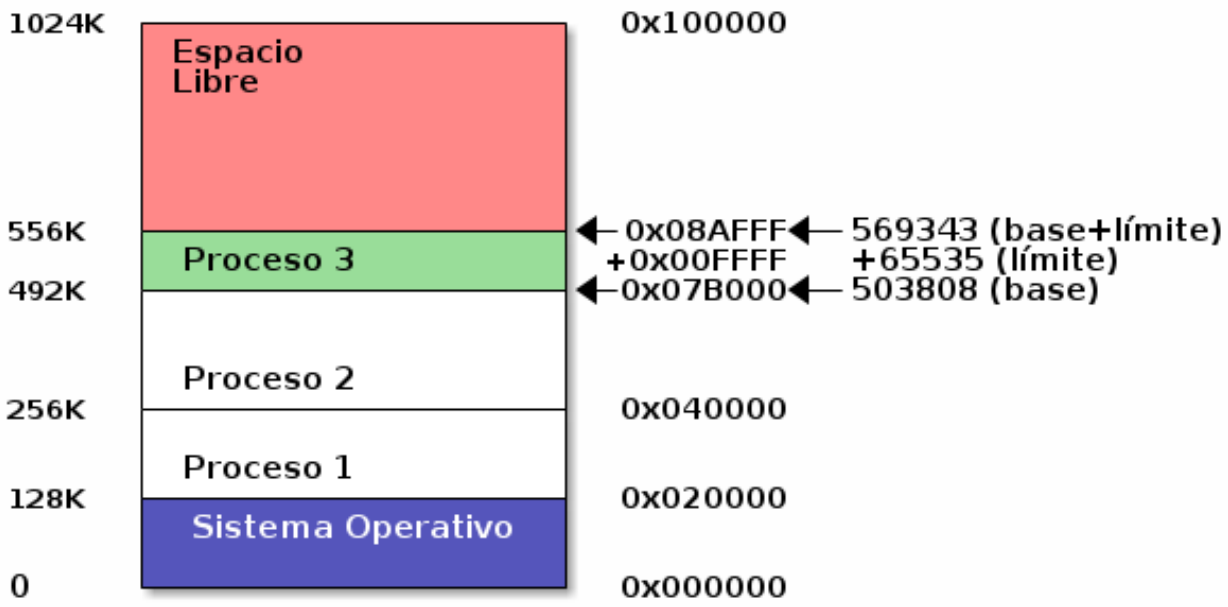


Figura 30.1: Espacio de direcciones válidas para el proceso 3 definido por un registro base y uno límite.

30.3. La memoria *caché*

Hay otro elemento en la actualidad que se asume como un hecho: la memoria *caché*. Si bien su manejo es (casi) transparente para el sistema operativo, es muy importante mantenerlo en mente.

Conforme el procesador avanza en la ejecución de las instrucciones (aumentando el valor almacenado en el registro de conteo de instrucción), se producen accesos a memoria. Por un lado, tiene que buscar en memoria la siguiente instrucción a ejecutar. Por otro, estas instrucciones pueden requerirle uno o más operadores adicionales que deban ser leídos de la memoria. Por último, la instrucción puede requerir guardar su resultado también en memoria.

Hace años esto no era un problema —la velocidad del procesador estaba básicamente sincronizada con la del manejador de memoria, y el flujo podía mantenerse básicamente estable. Pero conforme los procesadores se fueron haciendo más rápidos, y conforme se ha popularizado el procesamiento en paralelo, la tecnología de la memoria no ha progresado a la misma velocidad. La memoria de alta velocidad es demasiado cara, e incluso las distancias de unos pocos centímetros se convierten en obstáculos insalvables por la velocidad máxima de los electrones viajando por pistas conductoras.

[2] ¿Por qué *de segmento*? Véase la sección 32.

Cuando el procesador solicita el contenido de una dirección de memoria y esta no está aún disponible, *tiene que detener su ejecución* (*stall*) hasta que los datos estén disponibles. El CPU no puede, a diferencia del sistema operativo, "congelar" todo y guardar el estado para atender otro proceso: para el procesador, la lista de instrucciones a ejecutar es estrictamente secuencial, y todo tiempo que requiere esperar una transferencia de datos es tiempo perdido.

La respuesta para reducir esa espera es la *memoria caché*. Esta es una memoria de alta velocidad, situada *entre* la memoria principal y el procesador propiamente, que guarda copias de las *páginas* que van siendo accesadas, partiendo del principio de la *localidad de referencia*:

Localidad temporal Es probable que un recurso que fue empleado recientemente vuelva a emplearse en un futuro cercano.

Localidad espacial La probabilidad de que un recurso *aún no requerido* sea accesado es mucho mayor si fue requerido algún recurso cercano.

Localidad secuencial Un recurso, y muy particularmente la memoria, tiende a ser requerido de forma secuencial.

Aplicando el concepto de localidad de referencia, cuando el procesador solicita al hardware determinada dirección de memoria, el hardware no sólo transfiere a la memoria caché el byte o palabra solicitado, sino que transfiere un bloque o *página* completo.

Cabe mencionar que hoy en día (particularmente desde que se detuvo la *guerra de los Megahertz*), parte importante del diferencial en precios de los procesadores líderes en el mercado es la cantidad de memoria caché de primero y segundo nivel con que cuentan.

30.4. El espacio en memoria de un proceso

Cuando un sistema operativo inicia un proceso, no se limita a volcar el archivo ejecutable a memoria, sino que tiene que proporcionar la estructura para que éste vaya guardando la información de estado relativa a su ejecución.

Sección (o segmento) de texto Es el nombre que recibe la imagen en memoria de las instrucciones a ser ejecutadas. Usualmente, la sección de texto ocupa las direcciones *más bajas* del espacio en memoria.

Sección de datos Espacio fijo preasignado para las variables globales y datos inicializados (como las cadena de caracteres por ejemplo). Este espacio es fijado en tiempo de compilación, y no puede cambiar (aunque los datos que cargados allí sí cambian en el tiempo de vida del proceso).

Espacio de *libres* Espacio de memoria que se emplea para la asignación dinámica de memoria *durante la ejecución* del proceso. Este espacio se ubica por encima de la sección de datos, y *crece hacia arriba*. Este espacio es conocido en inglés como el *Heap*.

Cuando el programa es escrito en lenguajes que requieren *manejo dinámico manual de la memoria* (como C), esta área es la que se maneja mediante las llamadas de la familia de `malloc` y `free`. En lenguajes con gestión automática, esta área es monitoreada por los *recolectores de basura*.

Pila de llamadas Consiste en un espacio de memoria que se usa para almacenar la secuencia de funciones que han sido llamadas dentro del proceso, con sus parámetros, direcciones de *retorno*, variables locales, etc. La pila ocupa la parte *más alta* del espacio en memoria, y *crece hacia abajo*. En inglés, la pila de llamadas es denominada *Stack*.

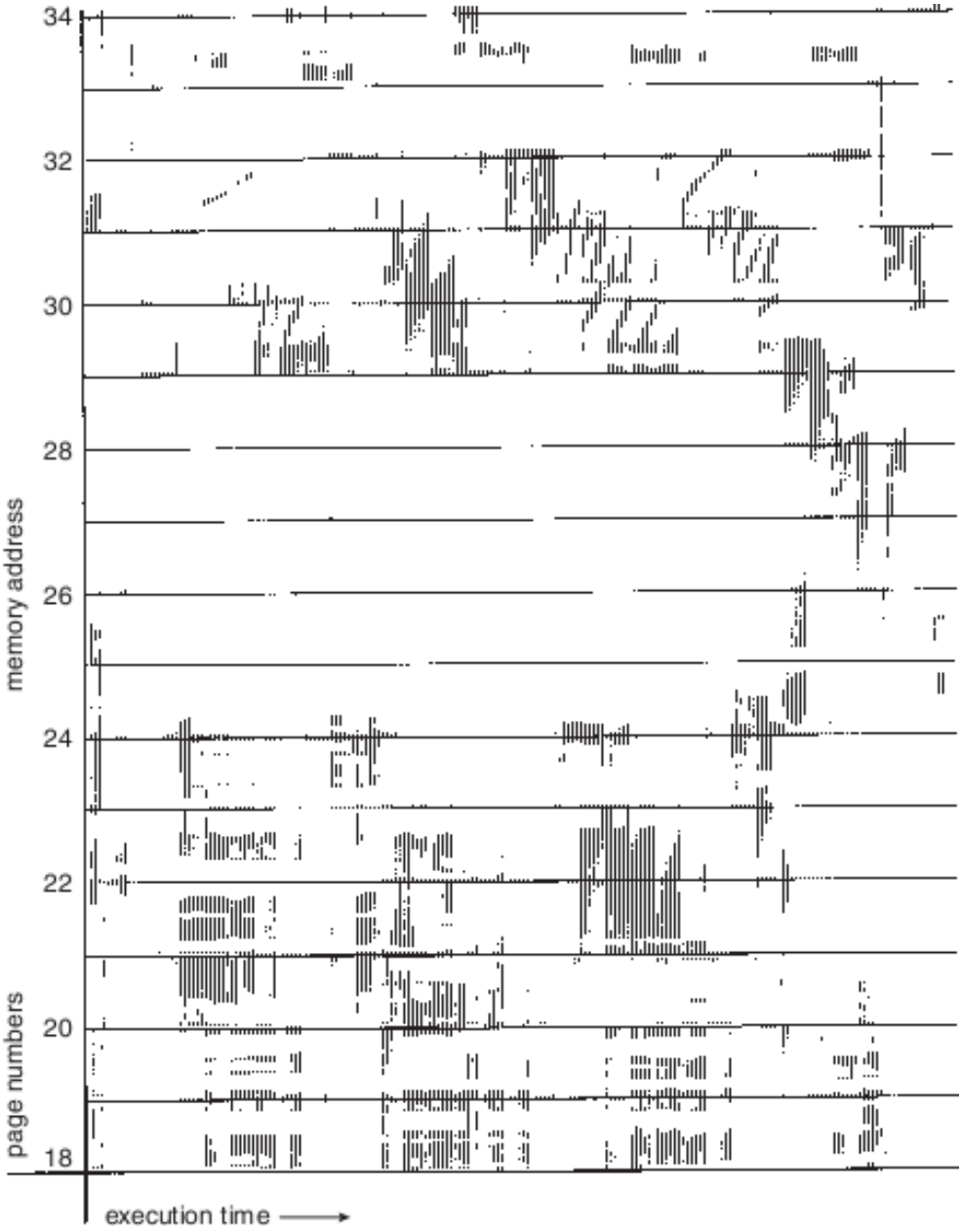


Figura 30.2: Patrones de acceso a memoria, demostrando la localidad espacial/temporal (Silberschatz, Galvin y Gagné 2010: 350).

30.5. Resolución de direcciones

Un programa compilado no emplea nombres simbólicos para las variables o funciones que llama;[3] el compilador, al convertir el programa a lenguaje máquina, las sustituye por la dirección en memoria donde se encuentra la variable o la función.[4]

[3]Cuando se hace *ligado dinámico* a bibliotecas externas sí se mantiene la referencia por nombre, pero para los propósitos de esta sección, se habla de las referencias internas únicamente

[4]De hecho, una vez que el programa se pasa a lenguaje de máquina, no hay diferencia real entre la dirección que ocupa una variable o código ejecutable. La diferencia se establece por el uso que se dé a la referencia de memoria. En la sección 35.1 se abordará un ejemplo de cómo esto puede tener importantes consecuencias.

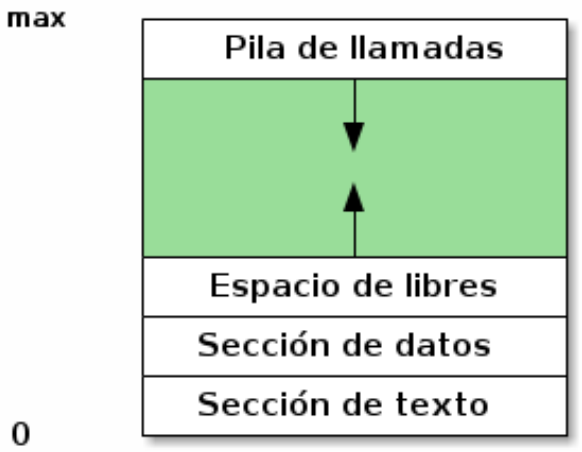


Figura 30.3: Regiones de la memoria para un proceso.

Ahora bien, en los sistemas actuales, los procesos requieren coexistir con otros, para lo cual las direcciones indicadas en el *texto* del programa pueden requerir ser traducidas al lugar *relativo al sitio de inicio del proceso en memoria* —esto es, las direcciones son *resueltas* o traducidas. Hay diferentes estrategias de resolución, que se pueden clasificar a grandes rasgos[5] en:

En tiempo de compilación El texto del programa tiene la dirección *absoluta* de las variables y funciones. Esto era muy común en las computadoras previas al multiprocesamiento. En la arquitectura compatible con PC, el formato ejecutable `.COM` es un volcado de memoria directo de un archivo objeto con las direcciones indicadas de forma absoluta. Esto puede verse hoy principalmente en sistemas embebidos o de función específica.

En tiempo de carga Al cargarse a memoria el programa y antes de iniciar su ejecución, el *cargador* (componente del sistema operativo) actualiza las referencias a memoria dentro del texto para que apunten al lugar correcto —claro está, esto depende de que el compilador indique dónde están todas las referencias a las variables y las funciones.

En tiempo de ejecución El programa nunca hace referencia a una ubicación absoluta de memoria, sino que lo hace siempre relativo a una *base* y un *desplazamiento* (offset). Esto permite que el proceso sea incluso reubicado en la memoria mientras está siendo ejecutado sin tener que sufrir cambios, pero requiere de hardware específico (como una MMU).

Esto es, los nombres simbólicos (por ejemplo, una variable llamada `contador`) para ser traducidos ya sea a ubicaciones en la memoria, pueden resolverse en tiempo de compilación (y quedar plasmada en el programa en disco con una ubicación explícita y definitiva: 510 200), en tiempo de carga (sería guardada en el programa en disco como *inicio* + 5 986 bytes, y el proceso de carga incluiría sustituirla por la dirección resuelta a la suma del registro base, 504 214, y el *desplazamiento*, 5 986, esto es, 510 200).

Por último, para emplear la resolución en tiempo de ejecución, se mantiene en las instrucciones a ser ejecutadas por el proceso la etiqueta relativa al módulo actual, *inicio* + 5 986 bytes, y es resuelta cada vez que sea requerido.

[5] Esta explicación simplifica muchos detalles; para el lector interesado en profundizar en este tema, se recomienda el libro *Linkers and Loaders* (*Ligadores y cargadores*) (Levine 1999), disponible en línea.

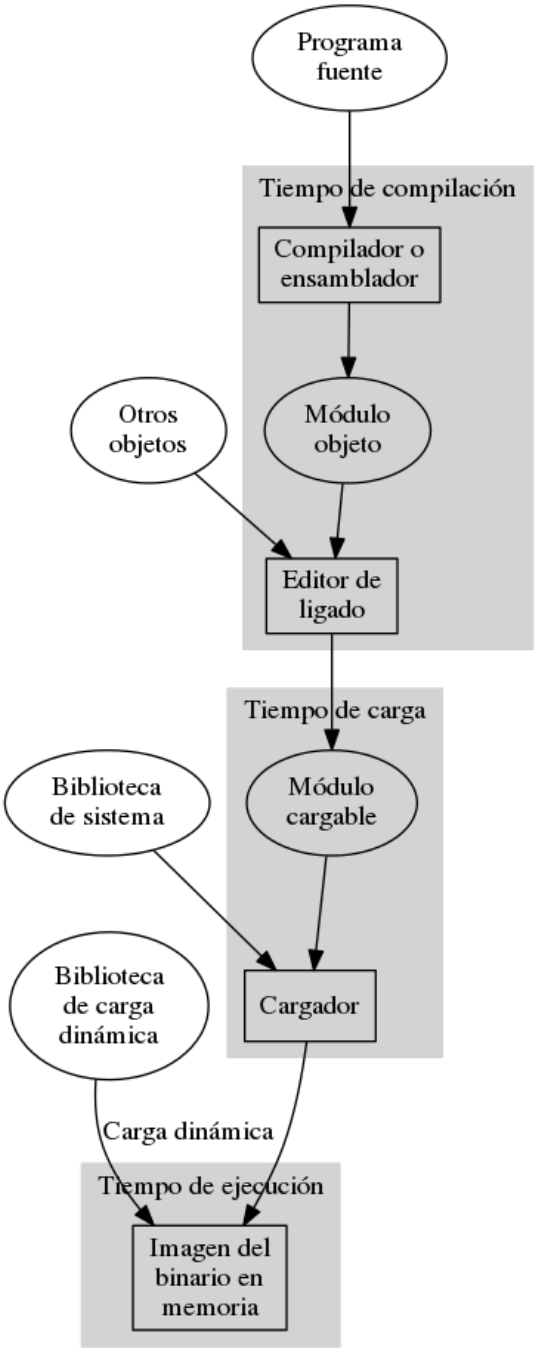


Figura 30.4: Proceso de compilación y carga de un programa, indicando el tipo de resolución de direcciones (Silberschatz, Galvin y Gagné 2010: 281).

Capítulo 31

Asignación de memoria contigua

En los sistemas de ejecución en lotes, así como en las primeras computadoras personales, sólo un programa se ejecutaba a la vez. Por lo que, más allá de la carga del programa y la satisfacción de alguna eventual llamada al sistema solicitando recursos, el sistema operativo no tenía que ocuparse de la asignación de memoria.

Al nacer los primeros sistemas operativos multitarea, se hizo necesario resolver cómo asignar el espacio en memoria a diferentes procesos.

31.1. Partición de la memoria

La primer respuesta, claro está, es la más sencilla: asignar a cada programa a ser ejecutado un bloque *contiguo* de memoria de un tamaño fijo. En tanto los programas permitieran la resolución de direcciones en tiempo de carga o de ejecución, podrían emplear este esquema.

El sistema operativo emplearía una región específica de la memoria del sistema (típicamente la *región baja* —desde la dirección en memoria 0x00000000 hacia arriba), y una vez terminado el espacio necesario para el núcleo y sus estructuras, el sistema asigna espacio a cada uno de los procesos. Si la arquitectura en cuestión permite limitar los segmentos disponibles a cada uno de los procesos (por ejemplo, con los registros *base* y *límite* discutidos anteriormente), esto sería suficiente para alojar en memoria varios procesos y evitar que interfieran entre sí.

Desde la perspectiva del sistema operativo, cada uno de los espacios asignados a un proceso es una *partición*. Cuando el sistema operativo inicia, toda la memoria disponible es vista como un sólo bloque, y conforme se van ejecutando procesos, este bloque va siendo subdividido para satisfacer sus requisitos. Al cargar un programa el sistema operativo calcula cuánta memoria va a requerir a lo largo de su vida prevista. Esto incluye el espacio requerido para la asignación dinámica de memoria con la familia de funciones `malloc` y `free`.

31.1.1. Fragmentación

Es un fenómeno que se manifiesta a medida que los procesos terminan su ejecución, y el sistema operativo libera la memoria asignada a cada uno de ellos. Si los procesos se encontraban en regiones de memoria, apartadas entre sí, comienzan a aparecer regiones de memoria disponible, *interrumpidas* por regiones de memoria usada por los procesos que aún se encuentran activos.

Si la computadora no tiene hardware específico que permita que los procesos resuelvan sus direcciones en tiempo de ejecución, el sistema operativo no puede reasignar los bloques existentes, y aunque pudiera hacerlo, mover un proceso entero en memoria puede resultar una operación costosa en tiempo de procesamiento.

Al crear un nuevo proceso, el sistema operativo tiene tres estrategias según las cuales podría asignarle uno de los bloques disponibles:

Primer ajuste El sistema toma el primer bloque con el tamaño suficiente para alojar el nuevo proceso. Este es el mecanismo más simple de implementar y el de más rápida ejecución. No obstante, esta estrategia puede causar el desperdicio de memoria, si el bloque no es exactamente del tamaño requerido.

Mejor ajuste El sistema busca entre todos los bloques disponibles cuál es el que mejor se ajusta al tamaño requerido por el nuevo proceso. Esto implica la revisión completa de la lista de bloques, pero permite que los bloques remanentes, una vez que se ubicó al nuevo proceso, sean tan pequeños como sea posible (esto es, que haya de hecho un *mejor ajuste*).

Peor ajuste El sistema busca cuál es el bloque más grande disponible, y se lo asigna al nuevo proceso. Empleando una estructura de datos como un *montículo*, esta operación puede ser incluso más rápida que la de primer ajuste. Con este mecanismo se busca que los bloques que queden después de otorgarlos a un proceso sean tan grandes como sea posible, de cierto modo balanceando su tamaño.

La *fragmentación externa* se produce cuando hay muchos bloques libres entre bloques asignados a procesos; la *fragmentación interna* se refiere a la cantidad de memoria dentro de un bloque que nunca se usará —por ejemplo, si el sistema operativo maneja *bloques* de 512 bytes y un proceso requiere sólo 768 bytes para su ejecución, el sistema le entregará dos bloques (1 024 bytes), con lo cual desperdicia 256 bytes. En el peor de los casos, con un bloque de n bytes, un proceso podría solicitar $kn + 1$ bytes de memoria, desperdiciando por fragmentación interna $n - 1$ bytes.

Según análisis estadísticos (Silberschatz, Galvin y Gagne 2010: 289), por cada N bloques asignados, se perderán del orden de $0.5N$ bloques por fragmentación interna y externa.

31.1.2. Compactación

Un problema importante que va surgiendo como resultado de esta fragmentación es que el espacio total libre de memoria puede ser mucho mayor que lo que requiere un nuevo proceso, pero al estar *fragmentada* en muchos bloques, éste no encontrará una partición contigua donde ser cargado.

Si los procesos emplean resolución de direcciones en tiempo de ejecución, cuando el sistema operativo comience a detectar un alto índice de fragmentación, puede lanzar una operación de *compresión* o *compactación*. Esta operación consiste en mover los contenidos en memoria de los bloques asignados para que ocupen espacios contiguos, permitiendo unificar varios bloques libres contiguos en uno solo.

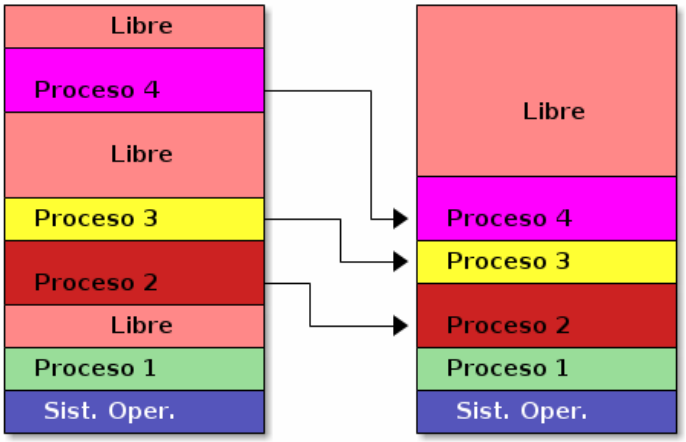


Figura 31.1: Compactación de la memoria de procesos en ejecución.

La compactación tiene un costo alto —involucra mover prácticamente la totalidad de la memoria (probablemente más de una vez por bloque).

31.1.3. Intercambio (*swap*) con el almacenamiento secundario

Siguiendo de cierto modo la lógica requerida por la compactación se encuentran los sistemas que utilizan *intercambio* (*swap*) entre la memoria primaria y secundaria. En éstos, el sistema operativo puede *comprometer* más espacio de memoria del que tiene físicamente disponible. Cuando la memoria se acaba, el sistema suspende un proceso (usualmente un proceso "bloqueado") y almacena una copia de su imagen en memoria en almacenamiento secundario para luego poder restaurarlo.

Hay algunas restricciones que observar previo a suspender un proceso. Por ejemplo, se debe considerar si el proceso tiene pendiente alguna operación de entrada/salida, en la cual el resultado se deberá copiar en su espacio de memoria. Si el proceso resultara suspendido (retirándolo de la memoria principal), el sistema operativo no tendría dónde continuar almacenando estos datos conforme llegaran. Una estrategia ante esta situación podría ser que todas las operaciones se realicen únicamente a *buffers* (regiones de memoria de almacenamiento temporal) en el espacio del sistema operativo, y éste las transfiera el contenido del buffer al espacio de memoria del proceso suspendido una vez que la operación finalice.

Esta técnica se popularizó en los sistemas de escritorio hacia finales de los 1980 y principios de los 1990, en que las computadoras personales tenían típicamente entre 1 y 8 MB de memoria.

Se debe considerar que las unidades de disco son del orden de decenas de miles de veces más lentas que la memoria, por lo que este proceso resulta muy caro. Por ejemplo, si la imagen en memoria de un proceso es de 100 MB, bastante conservador hoy en día, y la tasa de transferencia sostenida al disco de 50 MB/s, intercambiar un proceso al disco toma dos segundos. Cargarlo de vuelta a la memoria toma otros dos segundos — y a esto debe sumarse el tiempo de posicionamiento de la cabeza de lectura/escritura, especialmente si el espacio a emplear no es secuencial y contiguo. Resulta obvio por qué esta técnica ha caído en desuso conforme aumenta el tamaño de los procesos.

Capítulo 32

Segmentación

Al desarrollar un programa en un lenguaje de alto nivel, el programador usualmente no se preocupa por la ubicación en la memoria física de los diferentes elementos que lo componen. Esto se debe a que en estos lenguajes las variables y funciones son referenciadas por sus *nombres*, no por su ubicación[1]. No obstante, cuando se compila el programa para una arquitectura que soporte segmentación, el compilador ubicará a cada una de las secciones presentadas en la sección 30.4 en un segmento diferente.

Esto permite activar los mecanismos que evitan la escritura accidental de las secciones de memoria del proceso que no se deberían modificar (aquellas que contienen código o de sólo lectura), y permitir la escritura de aquellas que sí (en las cuales se encuentran las variables globales, la pila o *stack* y el espacio de asignación dinámica o *heap*).

Así, los elementos que conforman un programa se organizan en *secciones*: una sección contiene el espacio

[1] Al programar en lenguaje C por ejemplo, un programador puede trabajar a este mismo nivel de abstracción, puede referirse directamente a las ubicaciones en memoria de sus datos empleando *aritmética de apuntadores*.

para las variables globales, otra sección contiene el código compilado, otra sección contiene la *tabla de símbolos*, etc.

Luego, cuando el sistema operativo crea un proceso a partir del programa, debe organizar el contenido del archivo ejecutable en memoria. Para ello carga en memoria algunas secciones del archivo ejecutable (como mínimo la sección para las variables globales y la sección de código) y puede configurar otras secciones como la pila o la sección de libres. Para garantizar la protección de cada una de estas secciones en la memoria del proceso, el sistema puede definir que cada *sección* del programa se encuentra en un *segmento* diferente, con diferentes tipos de acceso.

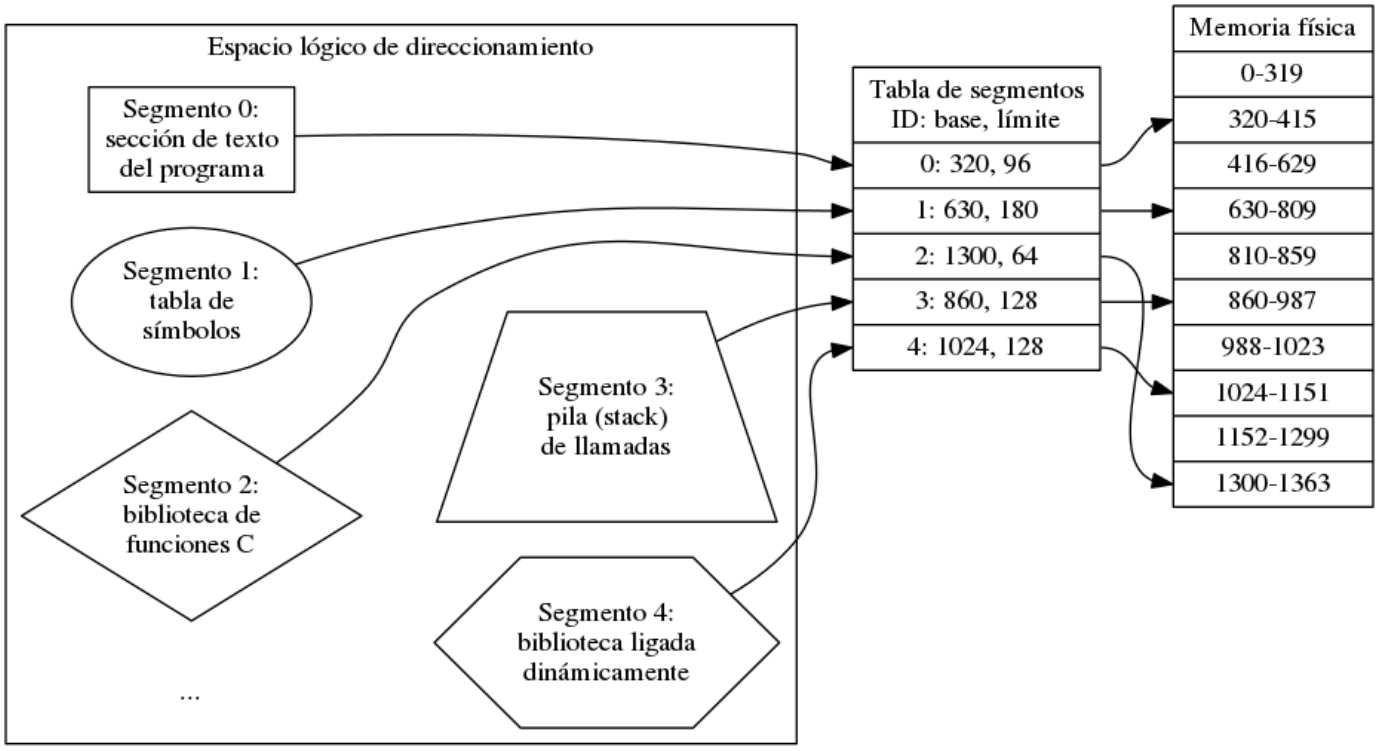


Figura 32.1: Ejemplo de segmentación.

La segmentación es un concepto que se aplica directamente a la arquitectura del procesador. Permite separar las regiones de la memoria *lineal* en *segmentos*, cada uno de los cuales puede tener diferentes permisos de acceso, como se explicará en la siguiente sección. La segmentación también ayuda a incrementar la *modularidad* de un programa: es muy común que las bibliotecas *ligadas dinámicamente* estén representadas en segmentos independientes.

Un código compilado para procesadores que implementen segmentación siempre generará referencias a la memoria en un espacio *segmentado*. Este tipo de referencias se denominan *direcciones lógicas* y están formadas por un *selector* de segmento y un *desplazamiento* dentro del segmento. Para interpretar esta dirección, la MMU debe tomar el selector, y usando alguna estructura de datos, obtiene la dirección base, el tamaño del segmento y sus atributos de protección. Luego, aplicando el mecanismo explicado en secciones anteriores, toma la dirección base del segmento y le suma el desplazamiento para obtener una *dirección lineal física*.

La traducción de una dirección lógica a una dirección lineal puede fallar por diferentes razones: si el segmento no se encuentra en memoria, ocurrirá una *excepción* del tipo *segmento no presente*. Por otro lado, si el desplazamiento especificado es mayor al tamaño definido para el segmento, ocurrirá una excepción del tipo *violación de segmento*.

32.1. Permisos

Una de las principales ventajas del uso de segmentación consiste en permitir que cada uno de los segmentos tenga un distinto *juego de permisos* para el proceso en cuestión: el sistema operativo puede indicar, por ejemplo, que el *segmento de texto* (el código del programa) sea de lectura y ejecución, mientras que las secciones de datos, libres y pila (donde se almacena y trabaja la información misma del programa) serán de lectura y escritura, pero la ejecución estará prohibida.[2]

De este modo, se puede evitar que un error en la programación resulte en que datos proporcionados por el usuario o por el entorno modifiquen el código que está siendo ejecutado.[3] Es más, dado que el acceso de *ejecución* está limitado a sólo los segmentos cargados del disco por el sistema operativo, un atacante no podrá introducir código ejecutable tan fácilmente —tendría que cargarlo como un segmento adicional con los permisos correspondientes.

La segmentación también permite distinguir *niveles* de acceso a la memoria: para que un proceso pueda efectuar *llamadas al sistema*, debe tener acceso a determinadas estructuras compartidas del núcleo. Claro está, al ser memoria privilegiada, su acceso requiere que el procesador esté ejecutando en *modo supervisor*.

32.2. Intercambio parcial

Un uso muy común de la segmentación, particularmente en los sistemas de los 1980, era el de permitir que sólo *ciertas regiones* de un programa sean intercambiadas al disco: si un programa está compuesto por porciones de código que nunca se ejecutarán aproximadamente al mismo tiempo en sucesión, puede separar su texto (e incluso los datos correspondientes) en diferentes segmentos.

A lo largo de la ejecución del programa, algunos de sus segmentos pueden no emplearse por largos periodos. Éstos pueden ser enviados al *espacio de intercambio* (*swap*) ya sea a solicitud del proceso o por iniciativa del sistema operativo.

32.2.1. Rendimiento

En la sección 31.1.3, donde se presenta el concepto de intercambio, se explicó que intercambiar un proceso completo resultaba demasiado caro. Cuando se tiene de un espacio de memoria segmentado y, muy particularmente, cuando se usan bibliotecas de carga dinámica, la sobrecarga es mucho menor.

En primer término, se puede hablar de la cantidad de información a intercambiar: en un sistema que sólo maneja regiones contiguas de memoria, intercambiar un proceso significa mover toda su información al disco. En un sistema con segmentación, puede enviarse a disco cada uno de los segmentos por separado, según el sistema operativo lo juzgue necesario. Podría *sacar* de memoria a alguno de los segmentos, eligiendo no necesariamente al que más *estorbe* (esto es, el más grande), sino el que más probablemente no se utilice: emplear el principio de localidad de referencia para intercambiar al segmento *menos recientemente utilizado* (LRU, *least recently used*).

Además de esto, si se tiene un segmento de texto (sea el código programa base o alguna de las bibliotecas) y su acceso es de sólo lectura, una vez que éste fue copiado ya al disco, no hace falta volver a hacerlo: se tiene la certeza de que no será modificado por el proceso en ejecución, por lo que basta marcarlo como *no presente* en las tablas de segmentos en memoria para que cualquier acceso ocasione que el sistema operativo lo traiga de disco.

[2] Si bien este es el manejo clásico, no es una regla inquebrantable: el código *automodificable* conlleva importantes riesgos de seguridad, pero bajo ciertos supuestos, el sistema debe permitir su ejecución. Además, muchos lenguajes de programación permiten la *metaprogramación*, que requiere la ejecución de código construido en tiempo de ejecución.

[3] Sin embargo, incluso bajo este esquema, dado que la *pila de llamadas* (*stack*) debe mantenerse como escribible, es común encontrar ataques que permiten modificar la *dirección de retorno* de una subrutina, como será descrito en la sección 35.1.

Por otro lado, si la biblioteca en cuestión reside en disco (antes de ser cargada) como una imagen directa de su representación en memoria, al sistema operativo le bastará identificar el archivo en cuestión al cargar el proceso; no hace falta siquiera cargarlo en la memoria principal y guardarlo al área de intercambio, puede quedar referido directamente al espacio en disco en que reside el archivo.

Claro está, el acceso a disco sigue siendo una fuerte penalización cada vez que un segmento tiene que ser cargado del disco (sea del sistema de archivos o del espacio de intercambio), pero este mecanismo reduce dicha penalización, haciendo más atractiva la flexibilidad del intercambio por segmentos.

32.3. Ejemplificando

A modo de ejemplo, y conjuntando los conceptos presentados en esta sección, si un proceso tuviera la siguiente tabla de segmentos:

Segmento	Inicio	Tamaño	Permisos	Presente
0	15 208	160	RWX	sí
1	1 400	100	R	sí
2	964	96	RX	sí
3	-	184	W	no
4	10 000	320	RWX	sí

En la columna de permisos, se indica con *R* el permiso de lectura (*Read*), *W* el de escritura (*Write*), y *X* el de ejecución (*eXecute*).

Un segmento que ha sido enviado al espacio de intercambio (en este caso, el 3), deja de estar presente en memoria y, por tanto, no tiene ya dirección de inicio registrada.

El resultado de hacer referencia a las siguientes direcciones y modos:

Dirección lógica	Tipo de acceso	Dirección física
0-100	R	15 308
2-84	X	1 048
2-84	W	Atrapada: violación de seguridad
2-132	R	Atrapada: desplazamiento fuera de rango
3-16	W	Atrapada: segmento faltante
3-132	R	Atrapada: segmento faltante; violación de seguridad
4-128	X	10 128
5-16	X	Atrapada: segmento inválido

Cuando se atrapa una situación de excepción, el sistema operativo debe intervenir. Por ejemplo, la solicitud de un segmento inválido, de un desplazamiento mayor al tamaño del segmento, o de un tipo de acceso que no esté autorizado, típicamente llevan a la terminación del proceso, en tanto que una de segmento faltante (indicando un segmento que está en el espacio de intercambio) llevaría a la suspensión del proceso, lectura del segmento de disco a memoria, y una vez que éste estuviera listo, se permitiría la continuación de la ejecución.

En caso de haber más de una excepción, como se observa en la solicitud de lectura de la dirección 3-132, el sistema debe reaccionar primero a la *más severa*: si como resultado de esa solicitud iniciara el proceso de carga del segmento, sólo para abortar la ejecución del proceso al detectarse la violación de tipo de acceso, sería un desperdicio injustificado de recursos.

Capítulo 33

Paginación

La fragmentación externa y, por tanto, la necesidad de compactación pueden evitarse por completo empleando la *paginación*. Ésta consiste en que cada proceso está dividio en varios bloques de tamaño fijo (más pequeños que los segmentos) llamados *páginas*, dejando de requerir que la asignación sea de un área *contigua* de memoria. Claro está, esto requiere de mayor soporte por parte del hardware, y mayor información relacionada a cada uno de los procesos: no basta sólo con indicar dónde inicia y termina el área de memoria de cada proceso, sino que se debe establecer un *mapeo* entre la ubicación real (*física*) y la presentada a cada uno de los procesos (*lógica*). La memoria se presentará a cada proceso como si fuera de su uso exclusivo.

La memoria física se divide en una serie de *marcos* (*frames*), todos ellos del mismo tamaño, y el espacio para cada proceso se divide en una serie de páginas (*pages*), del mismo tamaño que los marcos. La MMU se encarga del mapeo entre páginas y marcos mediante *tablas de páginas*.

Cuando se trabaja bajo una arquitectura que maneja paginación, las direcciones que maneja el CPU ya no son presentadas de forma absoluta. Los bits de cada dirección se separan en un *identificador de página* y un *desplazamiento*, de forma similar a lo presentado al hablar de resolución de instrucciones en tiempo de ejecución. La principal diferencia con lo entonces abordado es que cada proceso tendrá ya no un único espacio en memoria, sino una multitud de páginas.

El tamaño de los marcos (y, por tanto, las páginas) debe ser una *potencia de dos*, de modo que la MMU pueda discernir fácilmente la porción de una dirección de memoria que se refiere a la *página* del *desplazamiento*. El rango varía, según el hardware, entre los 512 bytes (2^9) y 16 MB (2^{24}); al ser una potencia de dos, la MMU puede separar la dirección en memoria entre los primeros *m* bits (referentes a la página) y los últimos *n* bits (referentes al desplazamiento).

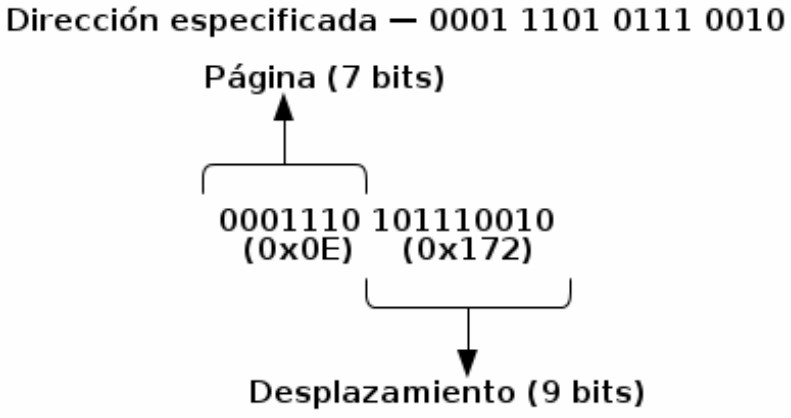


Figura 33.1: Página y desplazamiento, en un esquema de direccionamiento de 16 bits y páginas de 512 bytes.

Para poder realizar este mapeo, la MMU requiere de una estructura de datos denominada *tabla de páginas* (*page table*), que *resuelve* la relación entre páginas y marcos, convirtiendo una *dirección lógica* (en el espacio del

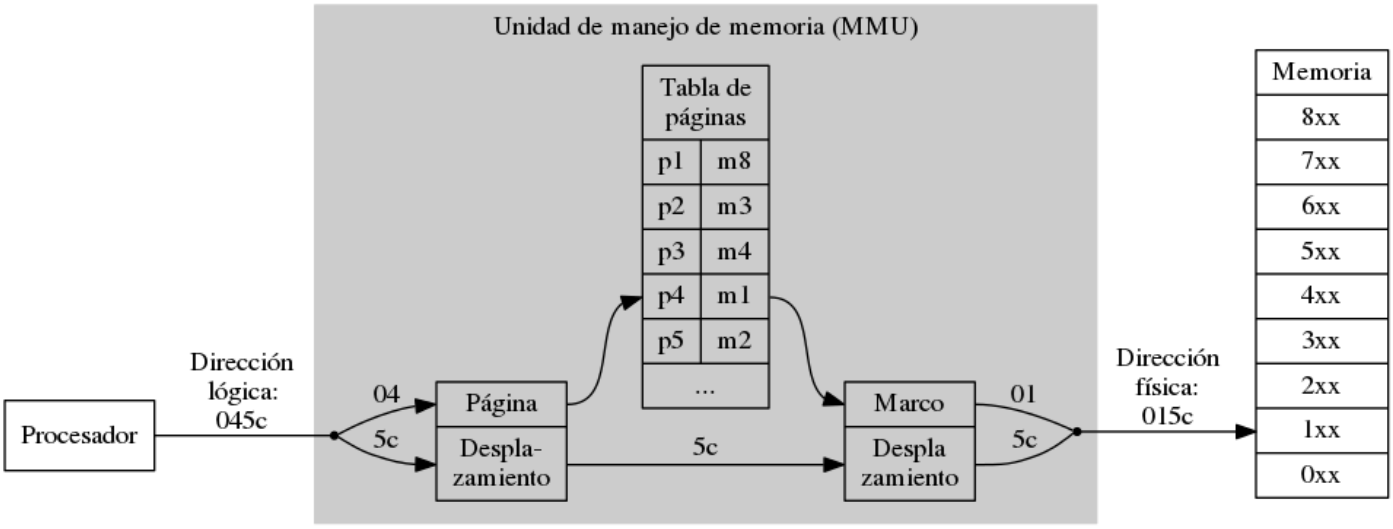


Figura 33.2: Esquema del proceso de paginación, ilustrando el papel de la MMU.

proceso) en la *dirección física* (la ubicación en que *realmente* se encuentra en la memoria del sistema).

Se puede tomar como ejemplo para explicar este mecanismo el esquema presentado en la figura 33.3 (Silberschatz, Galvin y Gagne 2010: 292). Éste muestra un esquema minúsculo de paginación: un *espacio de direccionamiento* de 32 bytes (cinco bits), organizado en ocho páginas de cuatro bytes cada una (esto es, la página es representada con los tres bits *más significativos* de la dirección, y el desplazamiento con los dos bits *menos significativos*).

El proceso que se presenta tiene una visión de la memoria como la columna del lado izquierdo: para el proceso hay cuatro páginas, y tiene sus datos distribuidos en orden desde la dirección 00 000 (0) hasta la 01 111 (15), aunque en realidad en el sistema éstas se encuentren desordenadas y ubicadas en posiciones no contiguas.

Cuando el proceso quiere referirse a la letra *f*, lo hace indicando la dirección 00 101 (5). De esta dirección, los tres bits más significativos (001, 1 —y para la computadora, lo *natural* es comenzar a contar por el 0) se refieren a la página uno, y los dos bits menos significativos (01, 1) indican al *desplazamiento* dentro de ésta.

La MMU verifica en la tabla de páginas, y encuentra que la página 1 corresponde al marco número 6 (110), por lo que traduce la dirección lógica 00 101 (5) a la física 11 001 (26).

Se puede tomar la paginación como una suerte de resolución o traducción de direcciones en tiempo de ejecución, pero con una *base* distinta para cada una de las páginas.

33.1. Tamaño de la página

Ahora, si bien la fragmentación externa se resuelve al emplear paginación, el problema de la fragmentación interna persiste: al dividir la memoria en bloques de longitud preestablecida de 2^n bytes, un proceso en promedio desperdiciará $\frac{2^n}{2}$ (y, en el peor de los casos, hasta $2^n - 1$). Multiplicando esto por el número de procesos que están en ejecución en todo momento en el sistema, para evitar que una proporción sensible de la memoria se pierda en fragmentación interna, se podría tomar como estrategia emplear un tamaño de página tan pequeño como fuera posible.

Sin embargo, la sobrecarga administrativa (el tamaño de la tabla de paginación) en que se incurre por gestionar demasiadas páginas pequeñas se vuelve una limitante en sentido opuesto:

- Las transferencias entre unidades de disco y memoria son mucho más eficientes si pueden mantenerse

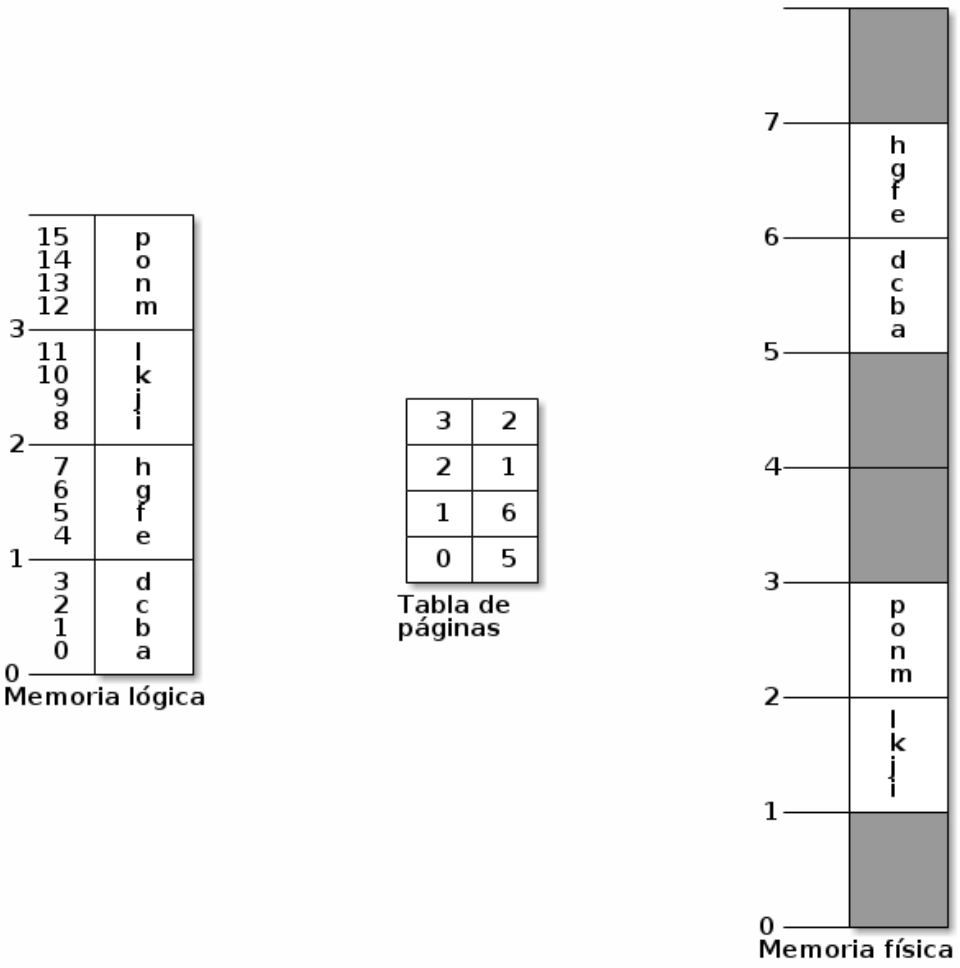


Figura 33.3: Ejemplo (minúsculo) de paginación, con un espacio de direccionamiento de 32 bytes y páginas de cuatro bytes.

como recorridos continuos. El controlador de disco puede responder a solicitudes de acceso directo a memoria (DMA) siempre que tanto los fragmentos en disco como en memoria sean continuos; fragmentar la memoria demasiado jugaría en contra de la eficiencia de estas solicitudes.

- El bloque de control de proceso (PCB) incluye la información de memoria. Entre más páginas tenga un proceso (aunque éstas fueran muy pequeñas), más grande es su PCB, y más información requerirá intercambiar en un cambio de contexto.

Estas consideraciones opuestas apuntan a que se debe mantener el tamaño de página más grande, y se regulan con las primeras expuestas en esta sección.

Hoy en día, el tamaño habitual de las páginas es de 4 u 8 KB (2^{12} o 2^{13} bytes). Hay algunos sistemas operativos que soportan múltiples tamaños de página — por ejemplo, Solaris puede emplear páginas de 8 KB y 4 MB (2^{13} o 2^{22} bytes), dependiendo del tipo de información que se declare que almacenarán.

33.2. Almacenamiento de la tabla de páginas

Algunos de los primeros equipos en manejar memoria paginada empleaban un conjunto especial de registros para representar la tabla de páginas. Esto era posible dado que eran sistemas de 16 bits, con páginas de 8 KB (2^{13}). Esto significa que contaban únicamente con ocho páginas posibles ($16 - 13 = 3; 2^3 = 8$), por lo que resultaba sensato dedicar un registro a cada una.

En los sistemas actuales, mantener la tabla de páginas en registros resultaría claramente imposible: teniendo un procesador de 32 bits, e incluso si se definiera un tamaño de página *muy* grande (por ejemplo, 4 MB), existirían 1 024 páginas posibles;[1] con un tamaño de páginas mucho más común (4 KB, 2^{12} bytes), la tabla de páginas llega a ocupar 5 MB.[2] Los registros son muy rápidos, sin embargo, son en correspondencia muy caros. El manejo de páginas más pequeñas (que es lo normal), y muy especialmente el uso de espacios de direccionamiento de 64 bits, harían prohibitivo este enfoque. Además, nuevamente, cada proceso tiene una tabla de páginas distinta —se haría necesario hacer una transferencia de información muy grande en cada cambio de contexto.

Otra estrategia para enfrentar esta situación es almacenar la propia tabla de páginas en memoria, y apuntar al inicio de la tabla con un juego de registros especiales: el *registro de base de la tabla de páginas* (PTBR, *page table base register*) y el *registro de longitud de la tabla de páginas* (PTLR, *page table length register*).[3] De esta manera, en el cambio de contexto sólo hay que cambiar estos dos registros, y además se cuenta con un espacio muy amplio para guardar las tablas de páginas que se necesiten. El problema con este mecanismo es la velocidad: Se estaría penalizando a *cada acceso a memoria* con uno adicional —si para resolver una dirección lógica a su correspondiente dirección física hace falta consultar la tabla de páginas en memoria, el tiempo efectivo de acceso a memoria se duplica.

33.2.1. El buffer de traducción adelantada (TLB)

La salida obvia a este dilema es el uso de un caché. Sin embargo, más que un caché genérico, la MMU utiliza un caché especializado en el tipo de información que maneja: el *buffer de traducción adelantada* o *anticipada*. (*Translation lookaside buffer*. El TLB es una tabla asociativa (un *hash*) en memoria de alta velocidad, una suerte de registros residentes dentro de la MMU, donde las *llaves* son las páginas y los *valores* son los marcos correspondientes. De este modo, las búsquedas se efectúan en tiempo constante.

El TLB típicamente tiene entre 64 y 1 024 entradas. Cuando el procesador efectúa un acceso a memoria, si la página solicitada está en el TLB, la MMU tiene la dirección física de inmediato.[4] En caso de no encontrarse la página en el TLB, la MMU lanza un *fallo de página* (*page fault*), con lo cual consulta de la memoria principal cuál es el marco correspondiente. Esta nueva entrada es agregada al TLB; por las propiedades de *localidad de referencia* que se presentaron anteriormente, la probabilidad de que las regiones más empleadas de la memoria durante un área específica de ejecución del programa sean cubiertas por relativamente pocas entradas del TLB son muy altas.

Como sea, dado que el TLB es limitado en tamaño, es necesario explicitar una política que indique dónde guardar las nuevas entradas (esto es, qué entrada reemplazar) una vez que el TLB está lleno y se produce un fallo de página. Uno de los esquemas más comunes es emplear la entrada *menos recientemente utilizada* (LRU,

[1] 4 MB es 2^{22} bytes; $\frac{2^{32}}{2^{22}} = 2^{10} = 1\,024$.

[2] $\frac{2^{32}}{2^{12}} = 2^{20} = 1\,048\,576$, cada entrada con un mínimo de 20 bits para la página y otros 20 para el marco. ¡La tabla de páginas misma ocuparía 1˜280 páginas!

[3] ¿Por qué es necesario el segundo? Porque es prácticamente imposible que un proceso emplee su espacio de direccionamiento completo; al indicar el límite máximo de su tabla de páginas por medio del PTLR se evita desperdiciar grandes cantidades de memoria indicando todo el espacio no utiilzado.

[4] El tiempo efectivo de acceso puede ser 10% superior al que tomaría sin emplear paginación (Silberschatz, Galvin y Gagne 2010: 295).

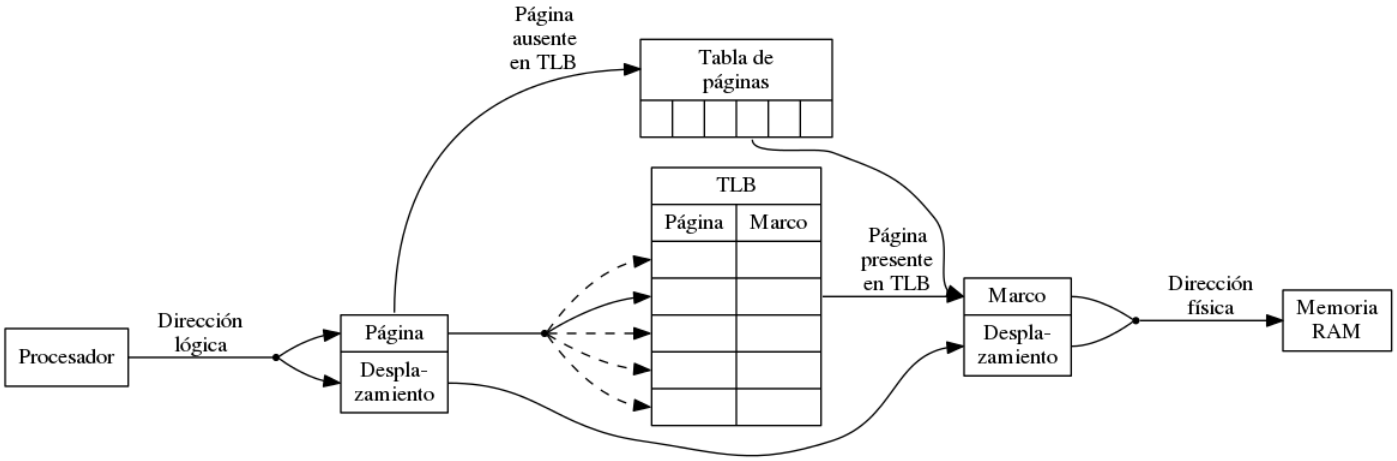


Figura 33.4: Esquema de paginación empleando un *buffer de traducción adelantada* (TLB).

Least Recently Used), nuevamente apelando a la localidad de referencia. Esto tiene como consecuencia necesaria que debe haber un mecanismo que contabilice los accesos dentro del TLB (lo cual agrega tanto latencia como costo). Otro mecanismo (con obvias desventajas) es el reemplazar una página al azar. Se explicarán con mayor detalle, más adelante, algunos de los mecanismos más empleados para este fin, comparando sus puntos a favor y en contra.

33.2.2. Subdividiendo la tabla de páginas

Incluso empleando un TLB, el espacio empleado por las páginas sigue siendo demasiado grande. Si se considera un escenario más frecuente que el propuesto anteriormente: empleando un procesador con espacio de direccionamiento de 32 bits, y un tamaño de página estándar (4 KB, 2^{12}), se tendría 1 048 576 (2^{20}) páginas. Si cada entrada de la página ocupa 40 bits[5] (esto es, cinco bytes), cada proceso requeriría de 5 MB (cinco bytes por cada una de las páginas) sólamente para representar su mapeo de memoria. Esto, especialmente en procesos pequeños, resultaría más gravoso para el sistema que los beneficios obtenidos de la paginación.

Aprovechando que la mayor parte del espacio de direccionamiento de un proceso está típicamente vacío (la pila de llamadas y el heap), se puede subdividir el identificador de página en dos (o más) niveles, por ejemplo, separando una dirección de 32 bits en una *tabla externa* de 10, una *tabla interna* de 10, y el *desplazamiento* de 12 bits.

Este esquema funciona adecuadamente para computadoras con direccionamiento de hasta 32 bits. Sin embargo, se debe considerar que cada nivel de páginas conlleva un acceso adicional a memoria en caso de fallo de página —emplear paginación jerárquica con un nivel externo y uno interno implica que un fallo de página *triplica* (y no duplica, como sería con un esquema de paginación directo) el tiempo de acceso a memoria. Para obtener una tabla de páginas manejable bajo los parámetros aquí descritos en un sistema de 64 bits, se puede *septuplicar* el tiempo de acceso (cinco accesos *en cascada* para fragmentos de 10 bits, y un tamaño de página de 14 bits, más el acceso a la página destino).

Otra alternativa es emplear *funciones digestoras* (*hash functions*)[6] para mapear cada una de las páginas a un *espacio muestral* mucho más pequeño. Cada página es mapeada a una lista de correspondencias simples.[7]

[5] 20 bits identificando a la página y otros 20 bits al marco; omitiendo aquí la necesidad de alinear los accesos a memoria a *bytes* individuales, que lo aumentarían a 24.

[6] Una función digestora puede definirse como $H : U \to M$, una función que *mapea* o *proyecta* al conjunto U en un conjunto M mucho menor; una característica muy deseable de toda función hash es que la *distribución resultante* en M sea homogénea y tan poco dependiente de la secuencialidad de la entrada como sea posible.

[7] A una lista y no a un valor único dado que una función digestora es necesariamente proclive a presentar *colisiones*; el sistema debe

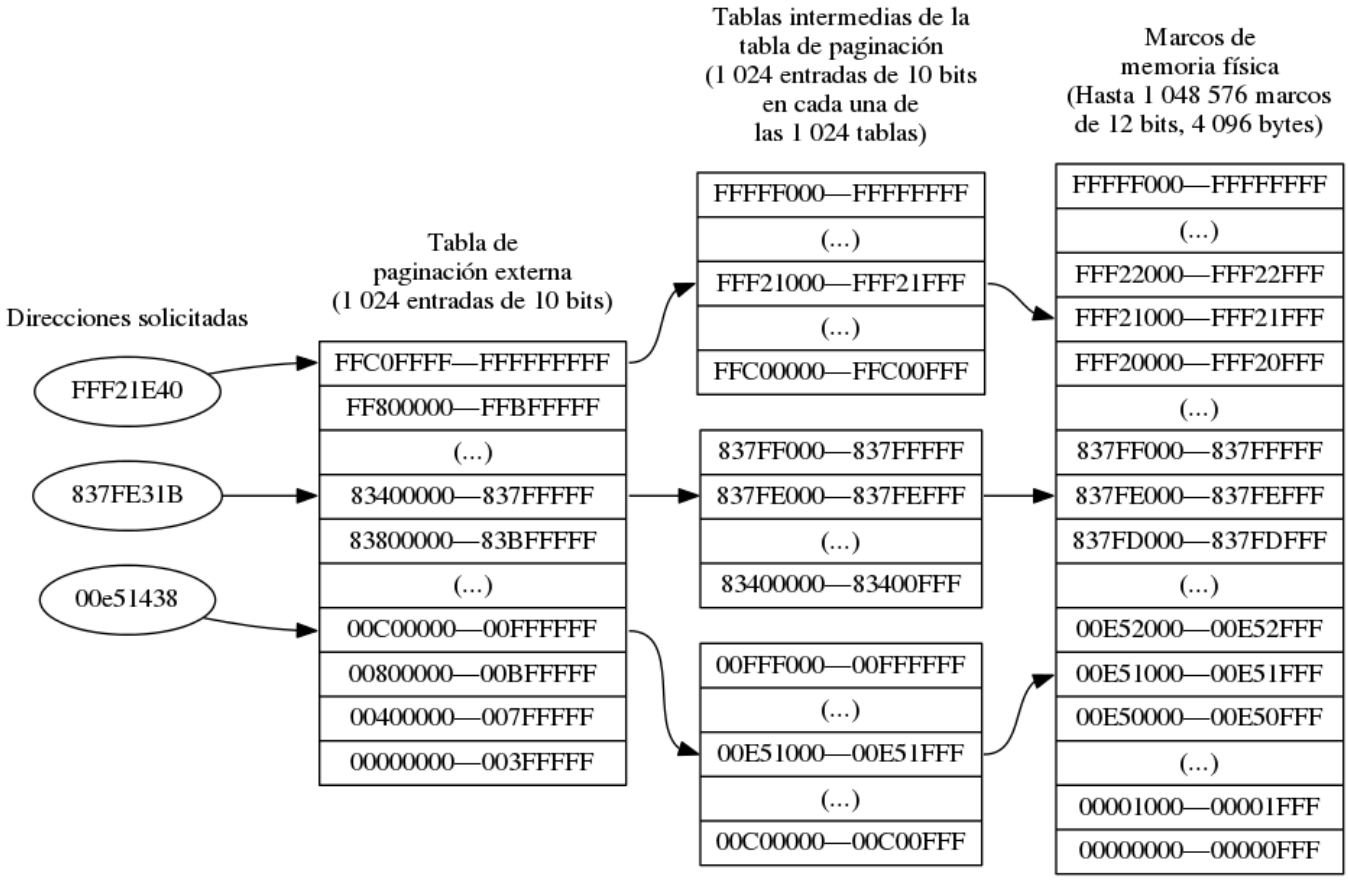


Figura 33.5: Paginación en dos niveles: una tabla externa de 10 bits, tablas intermedias de 10 bits, y marcos de 12 bits (esquema común para procesadores de 32 bits).

Un esquema basado en funciones digestoras ofrece características muy deseables: el tamaño de la tabla de páginas puede variar según crece el uso de memoria de un proceso (aunque esto requiera recalcular la tabla con diferentes parámetros) y el número de accesos a memoria en espacios tan grandes como el de un procesador de 64 bits se mantiene mucho más tratable. Sin embargo, por la alta frecuencia de accesos a esta tabla, debe elegirse un algoritmo digestor muy ágil para evitar que el tiempo que tome calcular la posición en la tabla resulte significativo frente a las alternativas.

33.3. Memoria compartida

Hay muchos escenarios en que diferentes procesos pueden beneficiarse de compartir áreas de su memoria. Uno de ellos es como mecanismo de comunicación entre procesos (IPC, *inter process communication*), en que dos o más procesos pueden intercambiar estructuras de datos complejas sin incurrir en el costo de copiado que implicaría copiarlas por medio del sistema operativo.

Otro caso, mucho más frecuente, es el de *compartir código*. Si un mismo programa es ejecutado varias veces, y dicho programa no emplea mecanismos de *código auto-modificable*, no tiene sentido que las páginas en que se representa cada una de dichas instancias ocupe un marco independiente —el sistema operativo puede asignar

poder resolver dichas colisiones sin pérdida de información.

a páginas de diversos procesos *el mismo conjunto de marcos*, con lo cual puede aumentar la capacidad percibida de memoria.

Y si bien es muy común compartir los *segmentos de texto* de los diversos programas que están en un momento dado en ejecución en la computadora, este mecanismo es todavía más útil cuando se usan *bibliotecas del sistema*: hay bibliotecas que son empleadas por una gran cantidad de programas[8].

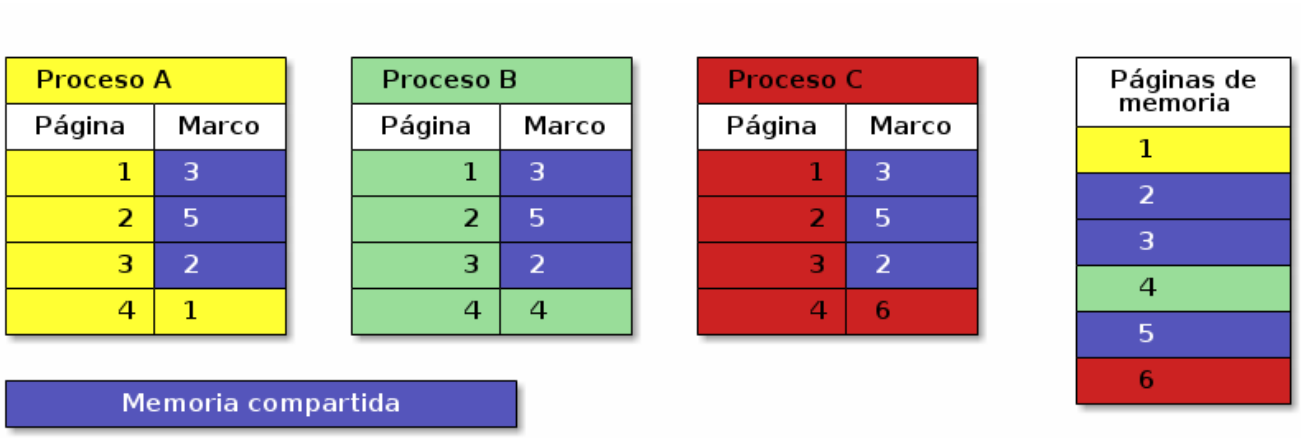


Figura 33.6: Uso de memoria compartida: tres procesos comparten la memoria ocupada por el texto del programa (azul), difieren sólo en los datos.

Claro está, para ofrecer este modelo, el sistema operativo debe garantizar que las páginas correspondientes a las *secciones de texto* (el código del programa) sean de sólo lectura.

Un programa que está desarrollado y compilado de forma que permita que todo su código sea de sólo lectura posibilita que diversos procesos entren a su espacio en memoria sin tener que sincronizarse con otros procesos que lo estén empleando.

33.3.1. Copiar al escribir (*copy on write*, *CoW*)

En los sistemas Unix, el mecanismo más frecuentemente utilizado para crear un nuevo proceso es el empleo de la llamada al sistema `fork()`. Cuando es invocado por un proceso, el sistema operativo crea un nuevo proceso *idéntico* al que lo llamó, diferenciándose únicamente en *el valor entregado* por la llamada a `fork()`. Si ocurre algún error, el sistema entrega un número negativo (indicando la causa del error). En caso de ser exitoso, el proceso nuevo (o proceso *hijo*) recibe el valor `0`, mientras que el preexistente (o proceso padre) recibe el PID (número identificador de proceso) del hijo. Es frecuente encontrar el siguiente código:

```
/* (...) */
int pid;
/* (...) */
pid = fork();
if (pid == 0) {
  /* Soy el proceso hijo */
  /* (...) */
 } else if (pid < 0) {
```

[8] Algunos ejemplos sobresalientes podrían ser la `libc` o `glibc`, que proporciona las funcinoes estándar del lenguaje C y es, por tanto, requerida por casi todos los programas del sistema; los diferentes entornos gráficos (en los Unixes modernos, los principales son `Qt` y `Gtk`); bibliotecas para el manejo de cifrado (`openssl`), compresión (`zlib`), imágenes (`libpng`, `libjpeg`), etcétera.

```
 /* Ocurrió un error, no se creó un proceso hijo */
} else {
 /* Soy el proceso padre */
 /* La variable 'pid' tiene el PID del proceso hijo */
 /* (...) */
}
```

Este método es incluso utilizado normalmente para crear nuevos procesos, transfiriendo el *ambiente* (variables, por ejemplo, que incluyen cuál es la *entrada* y *salida* estándar de un proceso, esto es, a qué terminal están conectados, indispensable en un sistema multiusuario). Frecuentemente, la siguiente instrucción que ejecuta un proceso hijo es `execve()`, que carga a un nuevo programa sobre el actual y transfiere la ejecución a su primera instrucción.

Cuesta trabajo comprender el por qué de esta lógica si no es por el empleo de la memoria compartida: el costo de `fork()` en un sistema Unix es muy bajo, se limita a crear las estructuras necesarias en la memoria del núcleo. Tanto el proceso padre como el proceso hijo comparten *todas* sus páginas de memoria, como lo ilustra la figura 33.7(a), sin embargo, siendo dos procesos independientes, no deben poder modificarse más que por los canales explícitos de comunicación entre procesos.

Esto ocurre así gracias al mecanismo llamado *copiar al escribir* (frecuentemente referido por sus siglas en inglés, *CoW*). Las páginas de memoria de ambos procesos son las mismas *mientras sean sólo leídas*. Sin embargo, si uno de los procesos modifica cualquier dato en una de estas páginas, ésta se copia a un nuevo marco, y deja de ser una página compartida, como se puede ver en la figura 33.7(b). El resto de las páginas seguirá siendo compartido. Esto se puede lograr marcando *todas* las páginas compartidas como *sólo lectura*, con lo cual cuando uno de los dos procesos intente modificar la información de alguna página se generará un fallo. El sistema operativo, al notar que esto ocurre sobre un espacio CoW, en vez de responder al fallo terminando al proceso, copiará sólo la página en la cual se encuentra la dirección de memoria que causó el fallo, y esta vez marcará la página como *lectura y escritura*.

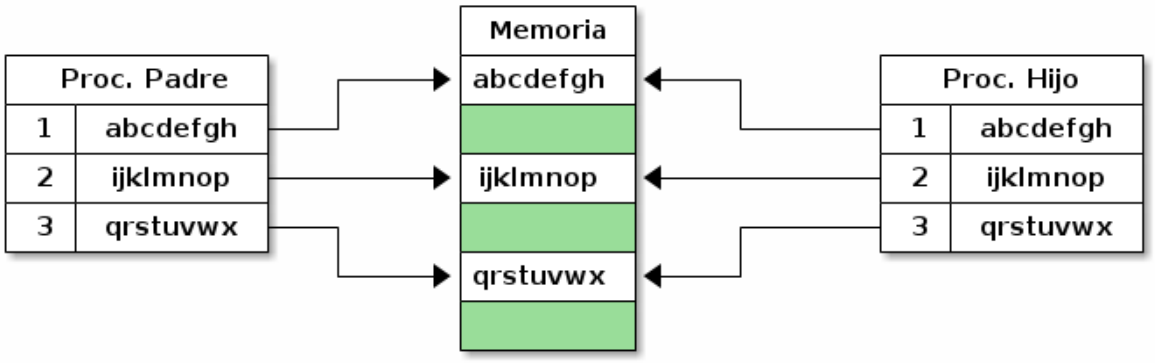
Incluso cuando se ejecutan nuevos programas mediante `execve()`, es posible que una buena parte de la memoria se mantenga compartida, por ejemplo, al referirse a copias de bibliotecas de sistema.

Capítulo 34

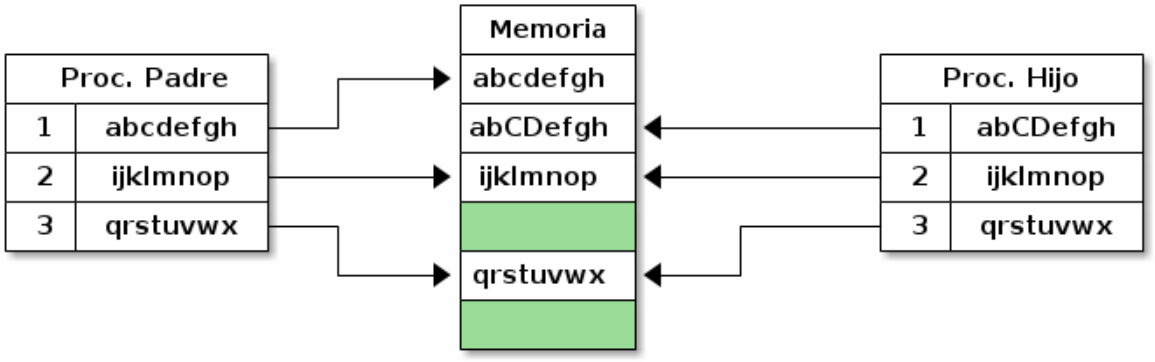
Memoria virtual

Varios de los aspectos mencionados en la sección 33 (*paginación*) van conformando a lo que se conoce como *memoria virtual*: en un sistema que emplea paginación, un proceso no conoce su dirección en memoria relativa a otros procesos, sino que trabajan con una *idealización* de la memoria, en la cual ocupan el espacio completo de direccionamiento, desde el cero hasta el límite lógico de la arquitectura, independientemente del tamaño físico de la memoria disponible.

Y si bien en el modelo mencionado de paginación los diferentes procesos pueden *compartir* regiones de memoria y *direccionar* más memoria de la físicamente disponible, no se ha presentado aún la estrategia que



(a) Inmediatamente después de la creación del proceso hijo por `fork()`



(b) Cuando el proceso hijo modifica información en la primer página de su memoria, se crea como una página nueva.

Figura 33.7: Memoria de dos procesos en un sistema que implementa *copiar al escribir*.

se emplearía cuando el total de páginas solicitadas por todos los procesos activos en el sistema superara el total de espacio físico. Es ahí donde entra en juego la *memoria virtual*: para ofrecer a los procesos mayor espacio en memoria de con el que se cuenta físicamente, el sistema emplea espacio en *almacenamiento secundario* (típicamente, disco duro), mediante un esquema de *intercambio* (*swap*) guardando y trayendo páginas enteras.

Es importante apuntar que la memoria virtual es gestionada *de forma automática y transparente* por el sistema operativo. No se hablaría de memoria virtual, por ejemplo, si un proceso pide explícitamente intercambiar determinadas páginas.

Puesto de otra manera: del mismo modo que la segmentación (sección 32) permitió hacer mucho más cómodo y útil al intercambio (31.1.3) por medio del intercambio parcial (32.2), permitiendo que continuara la ejecución del proceso, incluso con ciertos segmentos *intercambiados* (*swappeados*) a disco, la memoria virtual lo hace aún más conveniente al aumentar la *granularidad* del intercambio: ahora ya no se enviarán a disco secciones lógicas completas del proceso (segmentos), sino que se podrá reemplazar página por página, aumentando significativamente el rendimiento resultante. Al emplear la memoria virtual, de cierto modo la memoria física se vuelve sólo una *proyección parcial* de la memoria lógica, potencialmente mucho mayor a ésta.

Técnicamente, cuando se habla de memoria virtual, no se está haciendo referencia a un *intercambiador* (*swapper)*, sino al *paginador*.

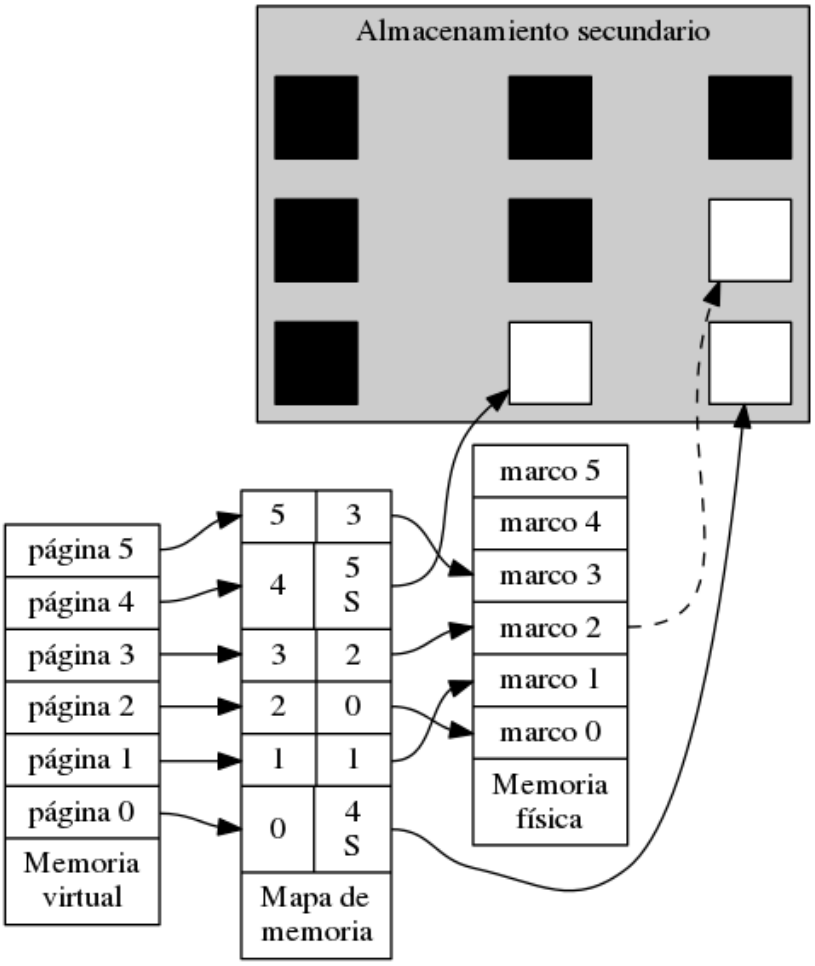


Figura 34.1: Esquema general de la memoria, incorporando espacio en almacenamiento secundario, representando la memoria virtual.

34.1. Paginación sobre demanda

La memoria virtual entra en juego desde la carga misma del proceso. Se debe considerar que hay una gran cantidad de *código durmiente* o *inalcanzable*: aquel que sólo se emplea eventualmente, como el que responde ante una situación de excepción o el que se emplea sólo ante circunstancias particulares (por ejemplo, la exportación de un documento a determinados formatos, o la verificación de que no haya tareas pendientes antes de cerrar un programa). Y si bien a una computadora le sería imposible ejecutar código que no esté cargado en memoria,[1] éste sí puede comenzar a ejecutarse sin estar *completamente* en memoria: basta con haber cargado la página donde están las instrucciones que permiten continuar con su ejecución actual.

La *paginación sobre demanda* significa que, para comenzar a ejecutar un proceso, el sistema operativo carga *solamente la porción necesaria* para comenzar la ejecución (posiblemente una página o ninguna), y que a lo largo de la ejecución, el paginador *es flojo*:[2] sólo carga a memoria las páginas cuando van a ser utilizadas. Al emplear un paginador *flojo*, las páginas que no sean requeridas nunca serán siquiera cargadas a memoria.

[1] Una computadora basada en la arquitectura von Neumann, como prácticamente todas las existen hoy en día, no puede *ver* directamente más que la memoria principal.

[2] En cómputo, muchos procesos pueden determinarse como *ansiosos* (*eager*), cuando buscan realizar todo el trabajo que puedan desde el inicio, o *flojos* (*lazy*), si buscan hacer el trabajo mínimo en un principio y diferir para más tarde tanto como sea posible.

La estructura empleada por la MMU para implementar un paginador flojo es muy parecida a la descrita al hablar del buffer de tradución adelantada (sección 33.2.1): la *tabla de páginas* incluirá un *bit de validez*, indicando para cada página del proceso si está presente o no en memoria. Si el proceso intenta emplear una página que esté marcada como no válida, esto causa un fallo de página, que lleva a que el sistema operativo lo suspenda y traiga a memoria la página solicitada para luego continuar con su ejecución:

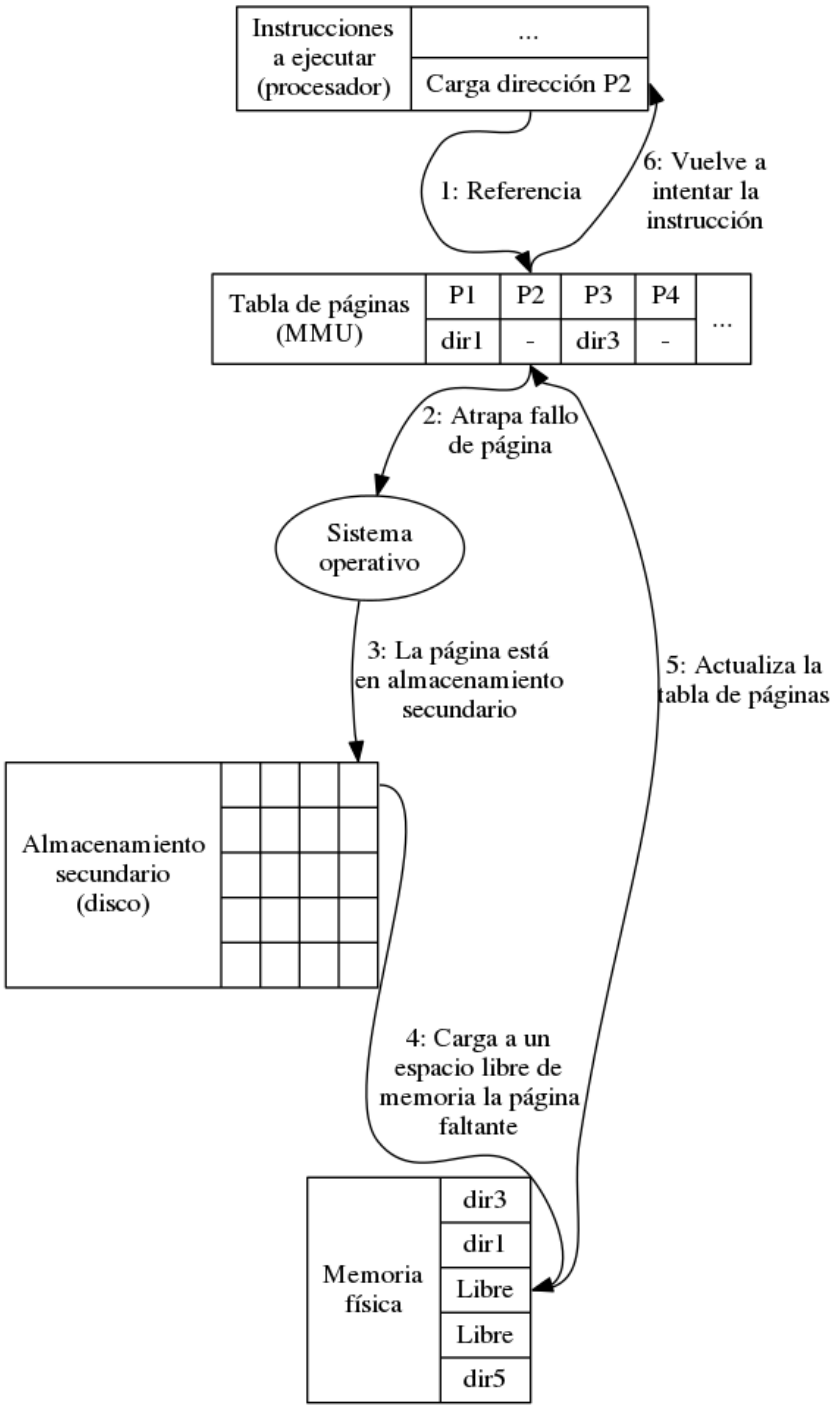


Figura 34.2: Pasos que atraviesa la respuesta a un fallo de página.

1. Verifica en el PCB si esta solicitud corresponde a una página que ya ha sido asignada a este proceso.

2. En caso de que la referencia sea inválida, se termina el proceso.
3. Procede a traer la página del disco a la memoria. El primer paso es buscar un marco disponible (por ejemplo, por medio de una tabla de asignación de marcos).
4. Solicita al disco la lectura de la página en cuestión hacia el marco especificado.
5. Una vez que finaliza la lectura de disco, modifica tanto al PCB como al TLB para indicar que la tabla está en memoria.
6. Termina la suspensión del proceso, continuando con la instrucción que desencadenó al fallo. El proceso puede continuar sin notar que la página había sido intercambiada.

Llevando este proceso al extremo, se puede pensar en un sistema de *paginación puramente sobre demanda* (*pure demand paging*): en un sistema así, *ninguna* página llegará al espacio de un proceso si no es mediante de un fallo de página. Un proceso, al iniciarse, comienza su ejecución *sin ninguna página en memoria*, y con el apuntador de siguiente instrucción del procesador apuntando a una dirección que no está en memoria (la dirección de la rutina de *inicio*). El sistema operativo responde cargando esta primer página, y conforme avanza el flujo del programa, el proceso irá ocupando el espacio real que empleará.

34.2. Rendimiento

La paginación sobre demanda puede impactar fuertemente el rendimiento de un proceso -se ha explicado ya que un acceso a disco es varios miles de veces más lento que el acceso a memoria. Es posible calcular el tiempo de acceso efectivo a memoria (t_e) a partir de la probabilidad que en un proceso se presente un fallo de página ($0 \leq p \leq 1$), conociendo el tiempo de acceso a memoria (t_a) y el tiempo que toma atender a un fallo de página (t_f):

$$t_e = (1 - p)t_a + pt_f$$

Ahora bien, dado que t_a ronda hoy en día entre los 10 y 200ns, mientras que t_f está más bien cerca de los 8 ms (la latencia típica de un disco duro es de 3 ms, el tiempo de posicionamiento de cabeza de 5ms, y el tiempo de transferencia es de 0.05 ms), para propósitos prácticos se puede ignorar a t_a. Con los valores presentados, seleccionando el mayor de los t_a presentados, si sólo un acceso a memoria de cada 1000 ocasiona un fallo de página (esto es, $p = \frac{1}{1\,000}$):

$$t_e = (1 - \frac{1}{1\,000}) \times 200ns + \frac{1}{1\,000} \times 8\,000\,000ns$$

$$t_e = 199{,}8ns + 8\,000ns = 8\,199{,}8ns$$

Esto es, en promedio, se tiene un tiempo efectivo de acceso a memoria *40 veces* mayor a que si no se empleara este mecanismo. Con estos mismos números, para mantener la degradación de rendimiento por acceso a memoria por debajo de 10%, se debería reducir la probabilidad de fallos de página a $\frac{1}{399\,990}$.

Cabe mencionar que esta repercusión al rendimiento no necesariamente significa que una proporción relativamente alta de fallos de página para un proceso afecte negativamente a todo el sistema —el mecanismo de paginación sobre demanda permite, al no requerir que se tengan en memoria todas las páginas de un proceso, que haya *más procesos activos* en el mismo espacio en memoria, aumentando el grado de multiprogramación del equipo. De este modo, si un proceso se ve obligado a esperar por 8 ms a que se resuelva un fallo de página, durante ese tiempo pueden seguirse ejecutando los demás procesos.

34.2.1. Acomodo de las páginas en disco

El cálculo recién presentado, además, asume que el acomodo de las páginas en disco es óptimo. Sin embargo, si para llegar a una página hay que resolver la dirección que ocupa en un sistema de archivos (posiblemente navegar una estructura de directorio), y si el espacio asignado a la memoria virtual es compartido con los archivos en disco, el rendimiento sufrirá adicionalmente.

Una de las principales deficiencias estructurales en este sentido de los sistemas de la familia Windows es que el espacio de almacenamiento se asigna en el espacio libre del sistema de archivos. Esto lleva a que, conforme crece la fragmentación del disco, la memoria virtual quede esparcida por todo el disco duro. La generalidad de sistemas tipo Unix, en contraposición, reservan una partición de disco *exclusivamente* para paginación.

34.3. Reemplazo de páginas

Si se aprovechan las características de la memoria virtual para aumentar el grado de multiprogramación, como se explicó en la sección anterior, se presenta un problema: al *sobre-comprometer* memoria, en determinado momento, los procesos que están en ejecución pueden caer en un patrón que requiera cargarse a memoria física páginas por un mayor uso de memoria que el que hay físicamente disponible.

Y si se tiene en cuenta que uno de los objetivos del sistema operativo es otorgar a los usuarios la *ilusión* de una computadora dedicada a sus procesos, no sería aceptable terminar la ejecución de un proceso ya aceptado y cuyos requisitos han sido aprobados, porque no hay suficiente memoria. Se vuelve necesario encontrar una forma justa y adecuada de llevar a cabo un *reemplazo de páginas* que permita continuar satisfaciendo sus necesidades.

El reemplazo de páginas es una parte fundamental de la paginación, ya que es la pieza que posibilita una verdadera separación entre memoria lógica y física. El mecanismo básico a ejecutar es simple: si todos los marcos están ocupados, el sistema deberá encontrar una página que pueda liberar (una *página víctima*) y llevarla al espacio de intercambio en el disco. Luego, se puede emplear el espacio recién liberado para *traer de vuelta* la página requerida, y continuar con la ejecución del proceso.

Esto implica una *doble* transferencia al disco (una para grabar la página víctima y una para traer la página de reemplazo), y por tanto, a una doble demora.

Se puede, con un mínimo de *burocracia* adicional (aunque requiere de apoyo de la MMU): implementar un mecanismo que disminuya la probabilidad de tener que realizar esta doble transferencia: agregar un *bit de modificación* o *bit de página sucia* (*dirty bit*) a la tabla de páginas. Este bit se marca como apagado siempre que se carga una página a memoria, y es automáticamente encendido por hardware cuando se realiza un acceso de escritura a dicha página.

Cuando el sistema operativo elige una página víctima, si su *bit de página sucia* está encendido, es necesario grabarla al disco, pero si está apagado, se garantiza que la información en disco es idéntica a su copia en memoria, y permite ahorrar la mitad del tiempo de transferencia.

Ahora bien, ¿cómo decidir qué páginas reemplazar marcándolas como *víctimas* cuando hace falta? Para esto se debe implementar un *algoritmo de reemplazo de páginas*. La característica que se busca en este algoritmo es que, para una patrón de accesos dado, permita obtener el menor número de fallos de página.

De la misma forma como se realizó la descripción de los algoritmos de planificación de procesos, para analizar los algoritmos de reemplazo se usará una *cadena de referencia*, esto es, una lista de referencias a memoria. Estas cadenas modelan el comportamiento de un conjunto de procesos en el sistema, y, obviamente, diferentes comportamientos llevarán a resultados distintos.

Hacer un volcado y trazado de ejecución en un sistema real puede dar una enorme cantidad de información, del orden de un millón de accesos por segundo. Para reducir esta información en un número más

tratable, se puede simplificar basado en que no interesa cada referencia a una *dirección* de memoria, sino cada referencia a una *página* diferente.

Además, varios accesos a direcciones de memoria en la misma página no causan efecto en el estado. Se puede tomar como un sólo acceso a todos aquellos que ocurren de forma consecutiva (esto es, sin llamar a ninguna otra página, no es necesario que sean en instrucciones consecutivas) a una misma página.

Para analizar a un algoritmo de reemplazo, si se busca la cantidad de fallos de página producidos, además de la cadena de referencia, es necesario conocer la cantidad de páginas y marcos del sistema que se está modelando. Por ejemplo, considérese la cadena de 12 solicitudes:

1, 4, 3, 4, 1, 2, 4, 2, 1, 3, 1, 4

Al recorrerla en un sistema con cuatro o más marcos, sólo se presentarían cuatro fallos (el fallo inicial que hace que se cargue por primera vez cada una de las páginas). Si, en el otro extremo, se cuenta con sólo un marco, se presentarían 12 fallos, dado que a cada solicitud se debería reemplazar el único marco disponible. El rendimiento evaluado sería en los casos de que se cuenta con dos o tres marcos.

34.3.1. Anomalía de Belady

Un fenómeno interesante que se presenta con algunos algoritmos es la *anomalía de Belady*, publicada en 1969: si bien la lógica indica que a mayor número de marcos disponibles se tendrá una menor cantidad de fallos de página, como lo ilustra la figura 34.3(a), con algunas de cadenas de referencia y bajo ciertos algoritmos puede haber una *regresión* o degradación, en la cual la cantidad de fallos aumenta aún con una mayor cantidad de marcos, como se puede ver en la figura 34.3(b).

Es importante recalcar que si bien la anomalía de Belady se presenta como un problema importante ante la evaluación de los algoritmos, en (La Red 2001: 559-569) se puede observar que en simulaciones con características más cercanas a las de los patrones reales de los programas, su efecto observado es prácticamente nulo.

Para los algoritmos que se presentan a continuación, se asumirá una memoria con tres marcos, y con la siguiente cadena de referencia:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

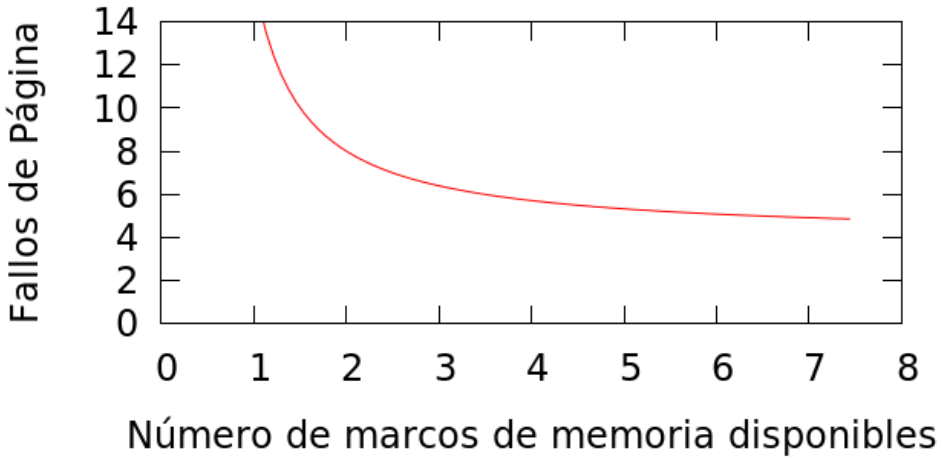
34.3.2. Primero en entrar, primero en salir (FIFO)

El algoritmo de más simple y de obvia implementación es, nuevamente, el FIFO: al cargar una página en memoria, se toma nota de en qué momento fue cargada, y cuando sea necesario reemplazar una página, se elige la que haya sido cargada hace más tiempo.

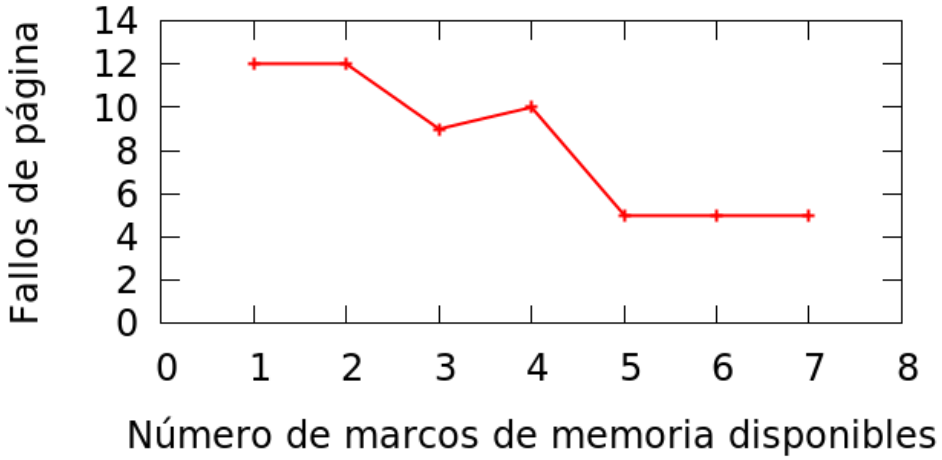
Partiendo de un estado inicial en que las tres páginas están vacías, necesariamente las tres primeras referencias a distintas páginas de memoria (7, 0, 1) causarán fallos de página. La siguiente (2) causará uno, pero la quinta referencia (0) puede ser satisfecha sin requerir una nueva transferencia.

La principal ventaja de este algoritmo es, como ya se ha mencionado, la simplicidad, tanto para programarlo como para comprenderlo. Su implementación puede ser tan simple como una lista ligada circular, cada elemento que va recibiendo se agrega en el último elemento de la lista, y se "empuja" el apuntador para convertirlo en la cabeza. Su desventaja, claro está, es que no toma en cuenta la historia de las últimas solicitudes, por lo que puede causar un bajo rendimiento. Todas las páginas tienen la misma probabilidad de ser reemplazadas, sin importar su frecuencia de uso.

Con las condiciones aquí presentadas, un esquema FIFO causará 15 fallos de página en un total de 20 accesos requeridos.



(a) Relación ideal entre el número de marcos y fallos de página.



(b) Comportamiento del algoritmo *fifo* exhibiendo la anomalía de Belady al pasar de tres a cuatro marcos.

Figura 34.3: Anomalía de Belady, empleando la cadena de referencia *1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5* (Belady, 1969).

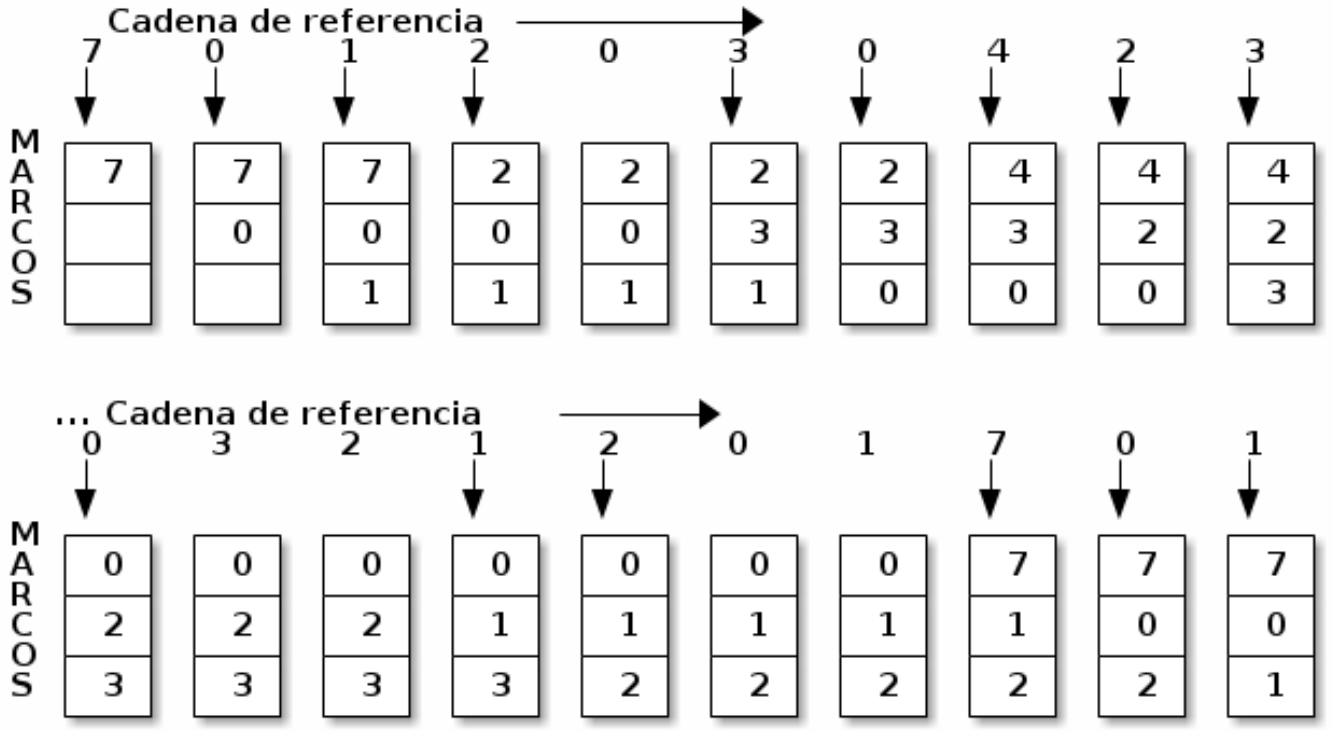


Figura 34.4: Algoritmo FIFO de reemplazo de páginas.

El algoritmo FIFO es vulnerable a la anomalía de Belady. La figura 34.3(b) ilustra este fenómeno al pasar de tres a cuatro marcos.

La prevalencia de cadenas que desencadenan la anomalía de Belady fue uno de los factores principales que llevaron al diseño de nuevos algoritmos de reemplazo de páginas.

34.3.3. Reemplazo de páginas óptimo (OPT, MIN)

Un segundo algoritmo, de interés puramente teórico, fue propuesto, y es típicamente conocido como OPT o MIN. Bajo este algoritmo, el enunciado será elegir como página víctima a aquella página que *no vaya a ser utilizada* por un tiempo máximo (o nunca más).

Si bien este algoritmo está demostrado como óptimo o mínimo, se mantiene como curiosidad teórica porque requiere conocimiento *a priori* de las necesidades a futuro del sistema —y si esto es impracticable ya en los algoritmos de despachadores, lo será mucho más con un recurso de reemplazo tan dinámico como la memoria.

Su principal utilidad reside en que ofrece una cota mínima: calculando el número de fallos que se presentan al seguir OPT, es posible ver qué tan cercano resulta otro algoritmo respecto al caso óptimo. Para esta cadena de referencia, y con tres páginas, se tiene un total de nueve fallos.

34.3.4. Menos recientemente utilizado (LRU)

Este esquema se ha revisado en diversos mecanismos relacionados con la administración de memoria. Busca acercarse a OPT *prediciendo* cuándo será la próxima vez en que se emplee cada una de las páginas que tiene en memoria basado en la *historia reciente* de su ejecución.

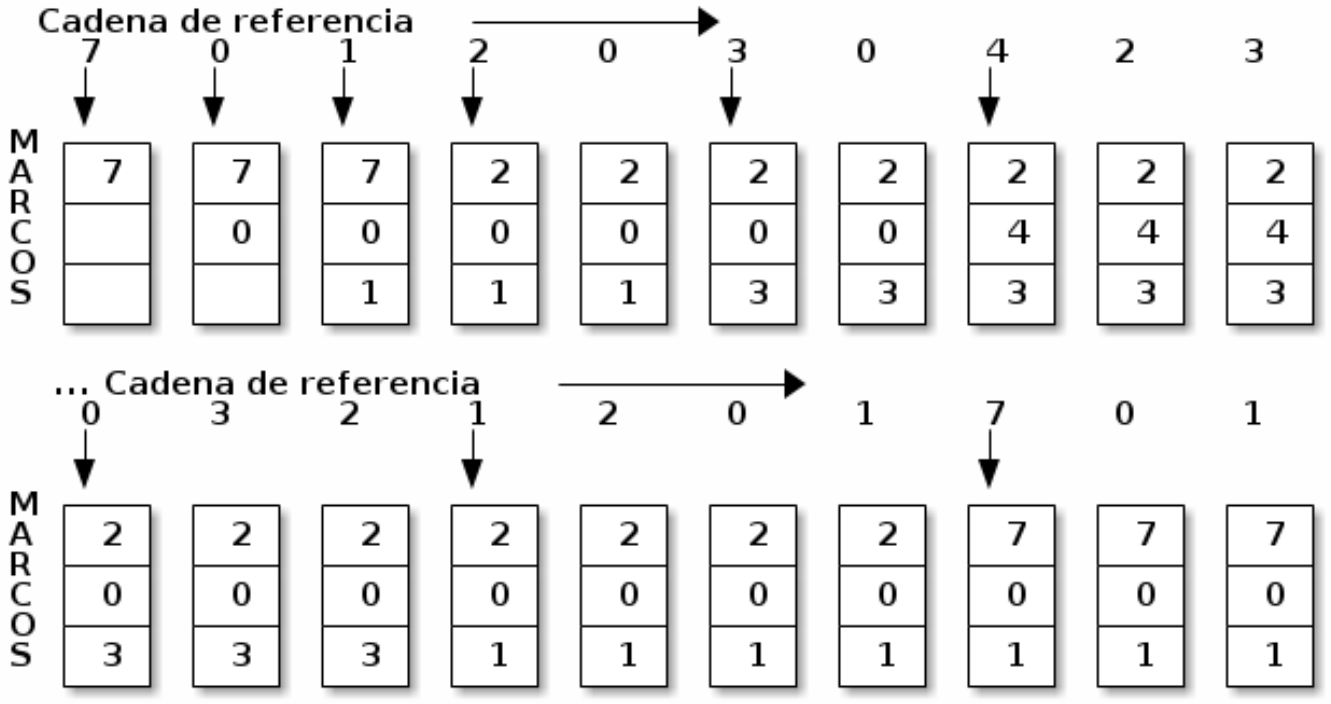


Figura 34.5: Algoritmo óptimo de reemplazo de páginas (OPT).

Cuando necesita elegir una página víctima, LRU elige la página que no ha sido empleada hace más tiempo.

Para la cadena de referencia, LRU genera 12 fallos, en el punto medio entre OPT y FIFO.

Una observación interesante puede ser que para una cadena S y su *cadena espejo* (invertida) R^S, el resultado de evaluar S empleando LRU es igual al de evaluar R^S con OPT, y viceversa.

La principal debilidad de LRU es que para su implementación requiere apoyo en hardware[3] sensiblemente más complejo que FIFO. Una implementación podría ser agregar un contador a cada uno de los marcos, actualizarlo siempre al hacer una referenciar a dicha página, y elegir como víctima a la página con un menor conteo. Este mecanismo tiene la desventaja de que, en presencia de una gran cantidad de páginas, tiene que recorrerlas todas para buscar la más *envejecida*.

Otro mecanismo es emplear una lista doblemente ligada con dos métodos de acceso: lista y *stack*. Cada vez que se haga referencia a una página, ésta se mueve a la cabeza del *stack*, y cada vez que se busque una página víctima, se selecciona a aquella que esté en el extremo *inferior* del *stack* (tomándolo como lista). Este mecanismo hace un poco más cara la actualización (pueden requerirse hasta seis modificaciones), pero encuentra la página víctima en tiempo constante.

Se ha demostrado que LRU y OPT están libres de la anomalía de Belady, dado que, para n marcos, las páginas que estarían en memoria son un subconjunto estricto de las que estarían con $n + 1$ marcos.

34.3.5. Más frecuentemente utilizada (MFU)/Menos frecuentemente utilizada (LFU)

Estos dos algoritmos se basan en mantener un contador, tal como lo hace LRU, pero en vez de medir el tiempo, miden la *cantidad* de referencias que se han hecho a cada página.

[3]Dada la frecuencia con que se efectúan referencias a memoria, emplear un mecanismo puramente en software para actualizar las entradas de los marcos resultaría inaceptablemente lento.

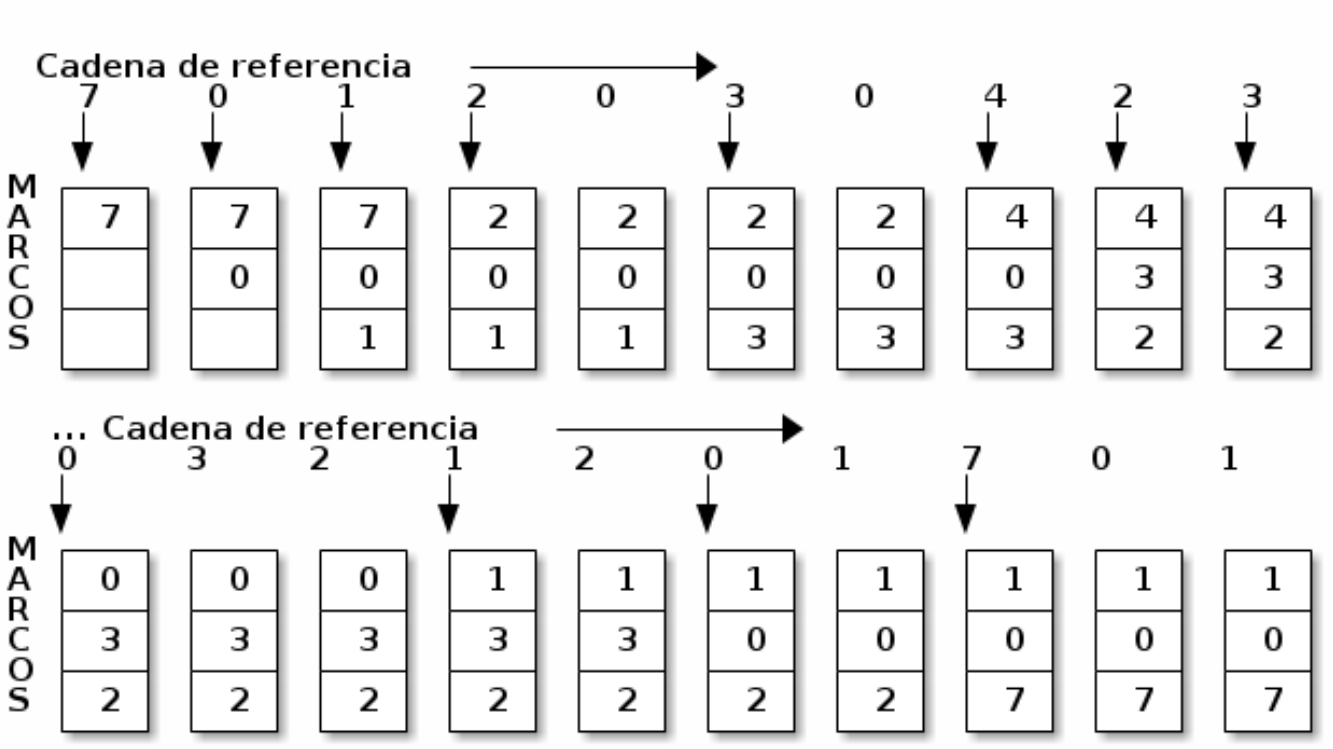


Figura 34.6: Algoritmo reemplazo de páginas menos recientemente utilizadas (LRU).

El MFU parte de la lógica que, si una página fue empleada muchas veces, probablemente vuelva a ser empleada muchas veces más; LFU parte de que una página que ha sido empleada pocas veces es probablemente una página recién cargada, y va a ser empleada en el futuro cercano.

Estos dos algoritmos son tan caros de implementar como LRU, y su rendimiento respecto a OPT no es tan cercana, por lo cual casi no son empleados.

34.3.6. Aproximaciones a LRU

Dada la complejidad que presenta la implementación de LRU en hardware, los siguientes sistemas buscan una *aproximación* a éste.

Bit de referencia Esta es una aproximación bastante común. Consiste en que todas las entradas de la tabla de páginas tengan un bit adicional, al que llamaremos *de referencia* o *de acceso*. Al iniciar la ejecución, todos los bits de referencia están apagados (0). Cada vez que se referencia a un marco, su bit de referencia se enciende (esto, en general, lo realiza el hardware).

El sistema operativo invoca periódicamente a que se apaguen nuevamente todos los bits de referencia. En caso de presentarse un fallo de página, se elige por FIFO sobre el subconjunto de marcos que no hayan sido referenciados en el periodo actual (esto es, entre todos aquellos para los cuales el bit de referencia sea 0).

Columna de referencia Una mejoría casi trivial sobre la anterior consiste en agregar *varios* bits de referencia, conformándose como una *columna*: en vez de descartar su valor cada vez que transcurre el periodo determinado, el valor de la columna de referencia es desplazado a la derecha, descartando el bit más

bajo (una actualización sólo modifica el bit más significativo). Por ejemplo, con una implementación de cuatro bits, un marco que no ha sido empleado en los últimos cuatro periodos tendría el valor 0 000, mientras que un marco que sí ha sido referenciado los últimos cuatro periodos tendría 1 111. Un marco que fue empleado hace cuatro y tres periodos, pero a partir entonces ya no, tendría el 0011.

Cuando el sistema tenga que elegir a una nueva página víctima, lo hará de entre el conjunto que tenga un número más bajo.

La parte de mantenimiento de este algoritmo es muy simple; recorrer una serie de bits es una operación muy sencilla. Seleccionar el número más bajo requiere una pequeña búsqueda, pero sigue resultando mucho más sencillo que LRU.

Segunda oportunidad (o *reloj*) El algoritmo de la segunda oportunidad trabaja también basado en un bit de referencia y un recorrido tipo FIFO. La diferencia en este caso es que, al igual que hay eventos que *encienden* a este bit (efectuar una referencia al marco), hay otros que lo *apagan*:

se mantiene un apuntador a la *próxima víctima*, y cuando el sistema requiera efectuar un reemplazo, éste verificará si el marco al que apunta tiene el bit de referencia encendido o apagado. En caso de estar apagado, el marco es seleccionado como víctima, pero en caso de estar encendido (indicando que fue utilizado recientemente), se le da una *segunda oportunidad*: el bit de referencia se apaga, el apuntador de víctima potencial avanza una posición, y vuelve a intentarlo.

A este algoritmo se le llama también *de reloj* porque puede implementarse como una lista ligada circular, y el apuntador puede ser visto como una manecilla. La manecilla avanza sobre la lista de marcos buscando uno con el bit de referencia apagado, y apagando a todos a su paso.

En el peor caso, el algoritmo de *segunda oportunidad* degenera en FIFO.

Segunda oportunidad mejorada El bit de referencia puede ampliarse con un *bit de modificación*, dándonos las siguientes combinaciones, en orden de preferencia:

(0, 0) No ha sido utilizado ni modificado recientemente. Candidato ideal para su reemplazo.

(0,1) No ha sido utilizada recientemente, pero está modificada. No es tan buena opción, porque es necesario escribir la página a disco antes de reemplazarla, pero puede ser elegida.

(1,0) El marco está *limpio*, pero fue empleado recientemente, por lo que probablemente se vuelva a requerir pronto.

(1,1) Empleada recientemente y *sucia* —sería necesario escribir la página a disco antes de reemplazar, y probablemente vuelva a ser requerida pronto. Hay que evitar reemplazarla.

La lógica para encontrar una página víctima es similar a la *segunda oportunidad*, pero busca reducir el costo de E/S. Esto puede requerir, sin embargo, dar hasta cuatro vueltas (por cada una de las listas) para elegir la página víctima.

34.3.7. Algoritmos con manejo de buffers

Un mecanismo que se emplea cada vez con mayor frecuencia es que el sistema no espere a enfrentarse a la necesidad de reemplazar un marco, sino que proactivamente busque tener siempre espacio vacío en memoria. Para hacerlo, conforme la carga lo permite, el sistema operativo busca las páginas *sucias* más proclives a ser llevadas a disco y va actualizando el disco (y marcándolas nuevamente como *limpias*). De este modo, cuando tenga que traer de vuelta una página, siempre habrá espacio donde ubicarla sin tener que esperar a que se transfiera una para liberarla.

34.4. Asignación de marcos

Abordando el problema prácticamente por el lado opuesto al del reemplazo de páginas, ¿cómo se asignan los marcos existentes a los procesos del sistema? Esto es, ¿qué esquemas se pueden definir para que la asignación inicial (y, de ser posible, en el transcurso de la ejecución) sea adecuada?

Por ejemplo, usando esquema sencillo: un sistema con 1 024 KB de memoria, compuesta de 256 páginas de 4096 bytes cada una, y basado en paginación puramente sobre demanda.

Si el sistema operativo ocupa 248 KB, el primer paso será reservar las 62 páginas que éste requiere, y destinar las 194 páginas restantes para los procesos a ejecutar.

Conforme se van lanzando y comienzan a ejecutar los procesos, cada vez que uno de ellos genere un fallo de página, se le irá asignando uno de los marcos disponibles hasta causar que la memoria entera esté ocupada. Claro está, cuando un proceso termine su ejecución, todos los marcos que tenía asignados volverán a la lista de marcos libres.

Una vez que la memoria esté completamente ocupada (esto es, que haya 194 páginas ocupadas por procesos), el siguiente fallo de página invocará a un algoritmo de reemplazo de página, que elegirá una de las 194.[4]

Este esquema, si bien es simple, al requerir una gran cantidad de fallos de página explícitos puede penalizar el rendimiento del sistema —el esquema puede resultar *demasiado flojo*, no le vendría mal ser un poco más *ansioso* y asignar, de inicio, un número determinado como mínimo utilizable de marcos.

34.4.1. Mínimo de marcos

Si un proceso tiene asignados muy pocos marcos, su rendimiento indudablemente se verá afectado. Hasta ahora se ha supuesto que cada instrucción puede causar un sólo fallo de página, pero la realidad es más compleja. Cada instrucción del procesador puede, dependiendo de la arquitectura, desencadenar varias solicitudes y potencialmente varios fallos de página.

Todas las arquitecturas proporcionan instrucciones de referencia directa a memoria (instrucciones que permiten especificar una dirección de memoria para leer o escribir) — esto significa que todas requerirán que, para que un proceso funcione adecuadamente, tenga por lo menos dos marcos asignados: en caso de que se le permitiera sólo uno, si la instrucción ubicada en `0x00A2C8` solicita la carga de `0x043F00`, ésta causaría dos fallos: el primero, cargar al marco la página `0x043`, y el segundo, cargar nuevamente la página `0x00A`, necesario para leer la siguiente instrucción a ejecutar del programa (`0x00A2CC`, asumiendo palabras de 32 bits).

Algunas arquitecturas, además, permiten *referencias indirectas a memoria*, esto es, la dirección de carga puede solicitar la dirección *que está referenciada* en `0x043F00`. El procesador tendría que recuperar esta dirección, y podría encontrarse con que hace referencia a una dirección en otra página (por ejemplo, `0x010F80`). Cada nivel de indirección que se permite aumenta en uno el número de páginas que se deben reservar como mínimo por proceso.

Algunas arquitecturas, particularmente las más antiguas,[5] permiten que tanto los operandos de algunas instrucciones aritméticas como su resultado sean direcciones de memoria (y no operan estrictamente sobre los registros, como las arquitecturas RISC). En éstas, el mínimo debe también tener este factor en cuenta: si en una sola instrucción es posible sumar dos direcciones de memoria y guardar el resultado en una adicional, el mínimo a reservar es de cuatro marcos: uno para el flujo del programa, otro para el primer operando, uno para el segundo operando, y el último para el resultado.

[4] En realidad, dentro de la memoria del sistema operativo, al igual que la de cualquier otro proceso, hay regiones que deben mantenerse residentes y otras que pueden paginarse. Se puede, simplificando, omitir por ahora esa complicación y asumir que el sistema operativo completo se mantendrá siempre en memoria

[5] Aquellas diseñadas antes de que la velocidad del procesador se distanciara tanto del tiempo de acceso a memoria.

34.4.2. Esquemas de asignación

Ahora, una vez establecido el número mínimo de marcos por proceso, ¿cómo determinar el nivel *deseable*?

Partiendo de que el rendimiento de un proceso será mejor entre menos fallos de paginación cause, se podría intentar otorgar a cada proceso el total de marcos que solicita, pero esto tendría como resultado disminuir el grado de multiprogramación y, por tanto, reducir el uso efectivo total del procesador.

Otra alternativa es la *asignación igualitaria*: se divide el total de espacio en memoria física entre todos los procesos en ejecución, en partes iguales. Esto es, volviendo a la computadora hipotética que se presentó al inicio de esta sección, si hay cuatro procesos que requieren ser ejecutados, de los 194 marcos disponibles, el sistema asignará 48 (192 KB) a dos de los procesos y 49 (196 KB) a los otros dos (es imposible asignar fracciones de marcos). De este modo, el espacio será compartido por igual.

La asignación igualitaria resulta ser un esquema deficiente para casi todas las distribuciones de procesos: bajo este esquema, si P_1 es un gestor de bases de datos que puede estar empleando 2 048 KB (512 páginas) de memoria virtual (a pesar de que el sistema tiene sólo 1 MB de memoria física) y P_2 es un lector de texto que está empleando un usuario, requiriendo apenas 112 KB (28 páginas), con lo cual incluso dejaría algunos de sus marcos sin utilizar.

Un segundo esquema, que resuelve mejor esta situación, es la *asignación proporcional*: dar a cada proceso una porción del espacio de memoria física proporcional a su uso de memoria virtual.

De tal suerte que, si además de los procesos anteriores se tiene a P_3 empleando 560 KB (140 páginas) y a P_4 con 320 KB (80 páginas) de memoria virtual, el uso total de memoria virtual sería de $V_T = 512 + 28 + 140 + 80 = 760$ páginas, esto es, el sistema tendría comprometido mediante la memoria virtual un sobreuso cercano a 4:1 sobre la memoria física.[6]

Cada proceso recibirá entonces $F_P = \frac{V_P}{V_T} \times m$, donde F_P indica el espacio de memoria física que el proceso recibirá, V_P la cantidad de memoria virtual que está empleando, y m la cantidad total de marcos de memoria disponibles. De este modo, P_1 recibirá 130 marcos, P_2 7, P_3 35 y P_4 20, proporcionalmente a su uso de memoria virtual.

Cabe apuntar que este mecanismo debe observar ciertos parámetros mínimos: por un lado, si el mínimo de marcos definido para esta arquitectura es de cuatro, por más que entrara en ejecución un proceso de 32 KB (ocho páginas) o aumentara al doble el grado de multiprocesamiento, ningún proceso debe tener asignado menos del mínimo definido.

La asignación proporcional también debe cuidar no sobre-asignar recursos a un proceso *obeso*: P_1 es ya mucho más grande que todos los procesos del sistema. En caso de que esta creciera mucho más, por ejemplo, si multiplicara por cuatro su uso de memoria virtual, esto llevaría a que se *castigara* desproporcionadamente a todos los demás procesos del sistema.

Por otro lado, este esquema ignora por completo las prioridades que hoy en día manejan todos los sistemas operativos; si se quisiera considerar, podría incluirse como factor la prioridad, multiplicando junto con V_P.

El esquema de asignación proporcional sufre, sin embargo, cuando cambia el nivel de multiprogramación, esto es, cuando se inicia un nuevo proceso o finaliza un proceso en ejecución, deben recalcularse los espacios en memoria física asignados a cada uno de los procesos restantes. Si finaliza un proceso, el problema es menor, pues sólo se asignan los marcos y puede esperarse a que se vayan poblando por paginación sobre demanda, pero si inicia uno nuevo, es necesario reducir de golpe la asignación de todos los demás procesos hasta abrir suficiente espacio para que quepa.

Por último, el esquema de la asignación proporcional también tiende a desperdiciar recursos: si bien hay procesos que mantienen un patrón estable de actividad a lo largo de su ejecución, muchos otros pueden tener periodos de mucho menor requisitos. Por ejemplo, un proceso servidor de documentos pasa la mayor parte

[6] Ya que de los 1 024 KB, o 256 páginas, que tiene el sistema descrito, descontando los 248 KB, o 62 páginas, que ocupa el sistema operativo, quedan 194 páginas disponibles para los procesos.

de su tiempo simplemente esperando solicitudes, y podría reducirse a un uso mínimo de memoria física, sin embargo, al solicitársele un documento, se le deberían poder asignar más marcos (para trabajar en una *ráfaga*) hasta que termine con su tarea. En la sección 34.5.1 se retomará este tema.

34.4.3. Ámbitos del algoritmo de reemplazo de páginas

Para atender los problemas no resueltos que se describieron en la sección anterior, se puede discutir el ámbito en que operará el algoritmo de reemplazo de páginas.

Reemplazo local Mantener tan estable como sea posible el cálculo hecho por el esquema de asignación empleado. Esto significa que cuando se presente un fallo de página, las únicas páginas que se considerarán para su intercambio serán aquellas pertenecientes *al mismo proceso* que el que causó el fallo.

Un proceso tiene asignado su espacio de memoria física, y se mantendrá estable mientras el sistema operativo no tome alguna decisión por cambiarlo.

Reemplazo global Los algoritmos de asignación determinan el espacio asignado a los procesos al ser inicializados, e influyen a los algoritmos de reemplazo (por ejemplo, dando mayor peso para ser elegidas como páginas víctima a aquellas que pertenezcan a un proceso que excede de su asignación en memoria física).

Los algoritmos de reemplazo de páginas operan sobre el espacio completo de memoria, y la asignación física de cada proceso puede variar según el estado del sistema momento a momento.

Reemplazo global con prioridad Es un esquema mixto, en el que un proceso puede *sobrepasar* su límite siempre que le *robe* espacio en memoria física exclusivamente a procesos de prioridad inferior a él. Esto es consistente con el comportamiento de los algoritmos planificadores, que siempre dan preferencia a un proceso de mayor prioridad por sobre de uno de prioridad más baja.

El reemplazo local es más rígido y no permite mejorar el rendimiento que tendría el sistema si aprovechara los periodos de inactividad de algunos de los procesos. En contraposición, los esquemas basados en reemplazo global pueden llevar a rendimiento inconsistente: dado que la asignación de memoria física sale del control de cada proceso puede que la misma sección de código presente tiempos de ejecución muy distintos si porciones importantes de su memoria fueron paginadas a disco.

34.5. Hiperpaginación

Es un fenómeno que se puede presentar por varias razones: cuando (bajo un esquema de reemplazo local) un proceso tiene asignadas pocas páginas para llevar a cabo su trabajo, y genera fallos de página con tal frecuencia que le imposibilita realizar trabajo real. Bajo un esquema de reemplazo global, cuando hay demasiados procesos en ejecución en el sistema y los constantes fallos y reemplazos hacen imposible a todos los procesos involucrados avanzar, también se presenta hiperpaginación.[7]

Hay varios escenarios que pueden desencadenar la hiperpaginación, y su efecto es tan claro e identificable que prácticamente cualquier usuario de cómputo lo sabrá reconocer. A continuación se presentará un escenario ejemplo en que las malas decisiones del sistema operativo pueden conducirlo a este estado.

Suponga un sistema que está con una carga media normal, con un esquema de reemplazo global de marcos. Se lanza un nuevo proceso, que como parte de su inicialización requiere poblar diversas estructuras a lo largo de su espacio de memoria virtual. Para hacerlo, lanza una serie de fallos de página, a las que el sistema operativo responde reemplazando a varios marcos pertenecientes a otros procesos.

[7] Una traducción literal del término *thrashing*, empleado en inglés para designar a este fenómeno, resulta más gráfica: *paliza*.

Casualmente, a lo largo del periodo que toma esta inicialización (que puede parecer una eternidad: el disco es entre miles y millones de veces más lento que la memoria) algunos de estos procesos solicitan los espacios de memoria que acaban de ser enviados a disco, por lo cual lanzan nuevos fallos de página.

Cuando el sistema operativo detecta que la utilización del procesador decrece, puede aprovechar la situación para lanzar procesos de mantenimiento. Se lanzan estos procesos, reduciendo aún más el espacio de memoria física disponible para cada uno de los procesos preexistentes.

Se ha formado ya toda una cola de solicitudes de paginación, algunas veces contradictorias. El procesador tiene que comenzar a ejecutar `NOOP` (esto es, no tiene trabajo que ejecutar), porque la mayor parte del tiempo lo pasa en espera de un nuevo marco por parte del disco duro. El sistema completo avanza cada vez más lento.

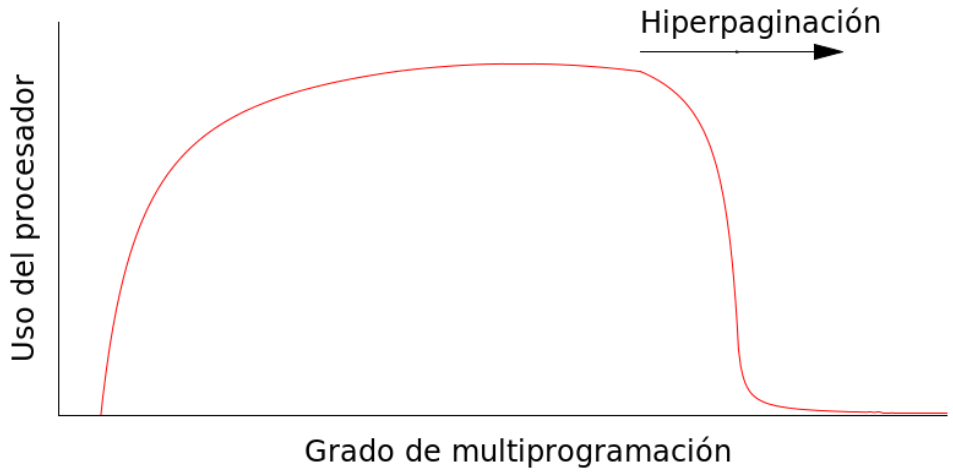


Figura 34.7: Al aumentar demasiado el grado de multiprogramación, el uso del CPU cae abruptamente, entrando en hiperpaginación (Silberschatz, Galvin y Gagné 2010: 349).

Los síntomas de la hiperpaginación son muy claros, y no son difíciles de detectar. ¿Qué estrategia puede emplear el sistema operativo una vez que se da cuenta que se presentó esta situación?

Una salida sería reducir el nivel de multiprogramación —si la paginación se presentó debido a que los requisitos de memoria de los procesos actualmente en ejecución no pueden ser satisfechos con la memoria física disponible, el sistema puede seleccionar uno (o más) de los procesos y suspenderlos por completo hasta que el sistema vuelva a un estado normal. Podría seleccionarse, por ejemplo, al proceso con menor prioridad, al que esté causando más cantidad de fallos, o al que esté ocupando más memoria.

34.5.1. Modelando el *conjunto activo*

Un pico en la cantidad de fallos de página no necesariamente significa que se va a presentar una situación de hiperpaginación —muchas veces indica que el proceso cambió su *atención* de un conjunto de páginas a otro, o dicho de otro modo, que cambió el *conjunto activo* del proceso, y resulta natural que, al cambiar el conjunto activo, el proceso accese de golpe una serie de páginas que no había referenciado en cierto tiempo.

El *conjunto activo* es, pues, la aproximación más clara a la *localidad de referencia* de un proceso dado: el conjunto de páginas sobre los que está iterando en un momento dado.

Idealmente, para evitar los problemas relacionados con la hiperpaginación, el sistema debe asignar a cada proceso suficientes páginas como para que mantenga en memoria física su conjunto activo —y si no es posible hacerlo, el proceso es un buen candidato para ser suspendido. Sin embargo, detectar con suficiente claridad como para efectuar este diagnóstico *cuál* es el conjunto activo es una tarea muy compleja, que típicamente implica rastrear y verificar del orden de los últimos miles a decenas de miles de accesos a memoria.

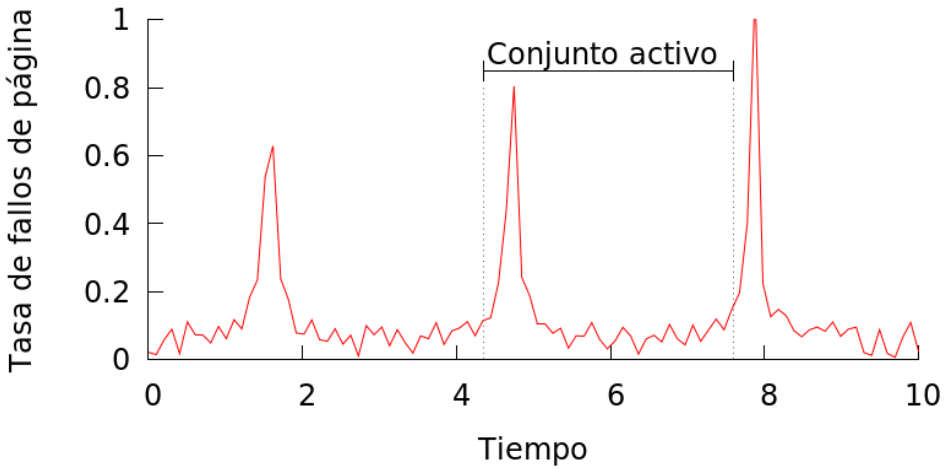


Figura 34.8: Los picos y valles en la cantidad de fallos de página de un proceso definen su *conjunto activo*.

Capítulo 35

Consideraciones de seguridad

Para una cobertura a mayor profundidad del material presentado en esta sección, se sugiere estudiar los siguientes textos:

- Smashing The Stack For Fun And Profit (Aleph One 1996)
- The Tao of Buffer Overflows (Sánchez Inédito)

35.1. Desbordamientos de buffer (*buffer overflows*)

Una de las funciones principales de los sistemas operativos en la que se ha insistido a lo largo del libro es la de implementar protección entre los procesos pertenecientes a diferentes usuarios, o ejecutándose con distinto nivel de privilegios. Y si bien el enfoque general que se ha propuesto es el de analizar por separado subsistema por subsistema, al hablar de administración de memoria es necesario mencionar también las implicaciones de seguridad que del presente tema se pueden desprender.

En las computadoras de arquitectura von Neumann, todo dato a ser procesado (sean instrucciones o datos) debe pasar por la memoria, por el *almacenamiento primario*. Sólo desde ahí puede el procesador leer la información directamente.

A lo largo del presente capítulo se ha mencionado que la MMU incluye ya desde el hardware el concepto de *permisos*, separando claramente las regiones de memoria donde se ubica el código del programa (y son, por tanto, ejecutables y de sólo lectura) de aquéllas donde se encuentran los datos (de lectura y escritura). Esto, sin embargo, no los pone a salvo de los *desbordamientos de buffer* (*buffer overflows*), errores de programación (típicamente, la falta de verificación de límites) que pueden convertirse en vulnerabilidades.[1]

[1]Citando a Theo de Raadt, autor principal del sistema operativo OpenBSD, todo error es una vulnerabilidad esperando a ser

35.1.1. La *pila de llamadas* (stack)

Recordando lo mencionado en la sección 30.4, en que se presentó el espacio en memoria de un proceso, es conveniente profundizar un poco más acerca de cómo está estructurada la *pila de llamadas* (*stack*).

El *stack* es el mecanismo que brinda un sentido local a la representación del código estructurado. Está dividido en *marcos de activación* (sin relación con el concepto de marcos empleado al hablar de memoria virtual); durante el periodo en que es el marco *activo* (esto es, cuando no se ha transferido el control a ninguna otra función), está delimitado por dos valores, almacenados en registros:

Apuntador a la pila (*Stack pointer*, `SP`) Apunta al *final actual* (dirección inferior) de la pila. En arquitecturas x86, emplea el registro `ESP`; cuando se pide al procesador que actúe sobre el *stack* (con las operaciones `pushl` o `popl`), lo hace sobre este registro.

Apuntador del marco (*Frame pointer*, `FP`, o *Base local*, `LB`) Apunta al *inicio* del marco actual, o lo que es lo mismo, al final del marco anterior. En arquitecturas x86, emplea el registro `EBP`.

A cada función a la cual va entrando la ejecución del proceso, se va creando un *marco de activación* en el *stack*, que incluye:

- Los argumentos recibidos por la función.
- La dirección de retorno al código que la invocó.
- Las variables locales creadas en la función.

Con esto en mente, es posible analizar la traducción de una llamada a función en C a su equivalente en ensamblador, y en segundo término ver el marco del *stack* resultante:

```
void func(int a, int b, int c) {
   char buffer1[5];
   char buffer2[10];
}

void main() {
  func(1,2,3);
}
```

Y lo que el código resultante en ensamblador efectúa es:

1. El procesador *empuja* (`pushl`) los tres argumentos al *stack* (`ESP`). La notación empleada (`$1`, `$2`, `$3`) indica que el número indicado se expresa de forma literal. Cada uno de estos tres valores restará 4 bytes (el tamaño de un valor entero en x86-32) a `ESP`.
2. En ensamblador, los nombres asignados a las variables y funciones no significan nada. La llamada `call` no es lo que se entendería como una llamada a función en un lenguaje de alto nivel —lo que hace el procesador es *empujar* al *stack* la dirección de la siguiente instrucción, y cargar a éste la dirección en el fuente donde está la etiqueta de la función (esto es, transferir la ejecución hacia allá).
3. Lo primero que hace la función al ser invocada es asegurarse de saber a dónde volver: *empuja* al *stack* el viejo apuntador al marco (`EBP`), y lo reemplaza (`movl`) por el actual. A esta ubicación se le llama `SFP` (*Saved Frame Pointer*, *apuntador al marco grabado*)

descubierta.

4. Por último, con `subl`, resta el espacio necesario para alojar las variables locales, `buffer1` y `buffer2`. Notarán que, si bien éstas son de 5 y 10 bytes, está recorriendo 20 bytes —esto porque, en la arquitectura x86-32, los accesos a memoria deben estar *alineados a 32 bits*.

```
; main
        pushl $3
        pushl $2
        pushl $1
        call func

func:
        pushl %ebp
        movl %esp,%ebp
        subl $20,%esp
```

La figura 35.1 ilustra cómo queda la región inferior del *stack* (el espacio de trabajo de la función actual) una vez que tuvieron lugar estos cuatro pasos.

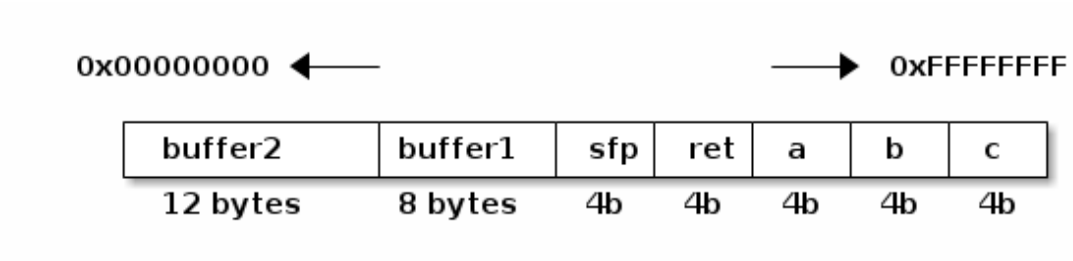


Figura 35.1: Marco del *stack* con llamada a `func(1,2,3)` en x86-32.

35.1.2. C y las funciones de manejo de cadenas

El lenguaje de programación C fue creado con el propósito de ser tan simple como sea posible, manteniéndose tan cerca del hardware como se pudiera, para que pudiera ser empleado como un lenguaje de programación para un sistema operativo portable. Y si bien en 1970 era visto como un lenguaje relativamente de alto nivel, hoy en día puede ubicarse como el más bajo nivel en que programa la mayor parte de los desarrolladores del mundo.

C no tiene soporte nativo para *cadenas* de caracteres. El soporte es provisto mediante *familias* de funciones en la biblioteca estándar del lenguaje, que están siempre disponibles en cualquier implementación estándar de C. Las familias principales son `strcat`, `strcpy`, `printf` y `gets`. Estas funciones trabajan con cadenas que siguen la siguiente estructura:

- Son arreglos de 1 o más caracteres (`char`, 8 bits).
- *Deben* terminar con el byte de terminación NUL (`\0`).

El problema con estas funciones es que sólo algunas de las funciones derivadas implementan verificaciones de límites, y algunas son incluso capaces de crear cadenas ilegales (que no concluyan con el terminador `\0`).

El problema aparece cuando el programador no tiene el cuidado necesario al trabajar con datos de los cuales no tiene *certeza*. Esto se demuestra con el siguiente código vulnerable:

```
#include <stdio.h>
int main(int argc, char **argv) {
        char buffer[256];
        if(argc > 1) strcpy(buffer, argv[1]);
        printf("Escribiste %s\n", buffer);
        return 0;
}
```

El problema con este código reside en el `strcpy(buffer, argv[1])` —dado que el código es recibido del usuario, no se tiene la *certeza* de que el argumento que recibe el programa por línea de comandos (empleando `argv[1]`) quepa en el arreglo `buffer[256]`. Esto es, si se ejecuta el programa ejemplo con una cadena de 120 caracteres:

```
$ ./ejemplo1 `perl -e 'print "A" x 120'`
Escribiste: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAA
$
```

La ejecución resulta exitosa. Sin embargo, si se ejecuta el programa con un parámetro demasiado largo para el arreglo:

```
$ ./ejemplo1 `perl -e 'print "A" x 500'`
Escribiste: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAA
```

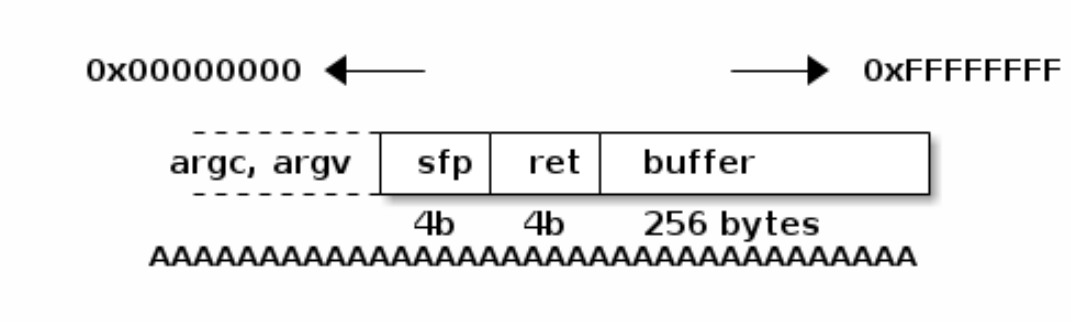


Figura 35.2: Estado de la memoria después del `strcpy()`.

para alterar el flujo del programa, esto es, sobreescribir `RET` con una dirección válida. Para esto, es necesario conocer la longitud desde el inicio del buffer hasta donde terminan `RET` y `SFP`, en este caso particular, 264 bytes (256 del buffer más cuatro de `RET` más cuatro de `SFP`).

Citando al texto de Enrique Sánchez,

> ¿Por qué ocurre un desbordamiento de *stack*? Imagina un vaso y una botella de cerveza. ¿Qué ocurre si sirves la botella completa en el vaso? Se va a derramar. Imagina que tu variable es el vaso, y la entrada del usuario es la cerveza. Puede ocurrir que el usuario sirva tanto líquido como el que cabe en el vaso, pero puede también seguir sirviendo hasta que se derrame. La cerveza se derramaría en todas direcciones, pero la memoria no crece de esa manera, es sólo un arreglo bidimensional, y sólo crece en una dirección.
>
> Ahora, ¿qué más pasa cuando desbordas un contenedor? El líquido sobrante va a mojar la botana, los papeles, la mesa, etc. En el caso de los papeles, destruirá cualquier cosa que hubieras apuntado (como el teléfono que acabas de anotar de esa linda chica). Cuando tu variable se desborde, ¿qué va a sobrescribir? Al `EBP`, al `EIP`, y lo que les siga, dependiendo de la función, y si es la última función, las variables de ambiente. Puede que el programa aborte y tu shell resulte inutilizado a causa de las variables sobreescritas.

Hay dos técnicas principales: *saltar* a un punto determinado del programa, y *saltar* hacia dentro del *stack*.

Un ejemplo de la primera técnica se muestra a continuación. Si el atacante está intentando burlar la siguiente validación simple de nombre de usuario y contraseña,

```
if (valid_user(usr, pass)) {
  /* (...) */
} else {
  printf("Error!\n");
  exit 1;
}
```

Y detecta que `valid_user()` es susceptible a un desbordamiento, le bastaría con incrementar en cuatro la dirección de retorno. La conversión de este `if` a ensamblador es, primero, saltar hacia la etiqueta `valid_user`, e ir (empleando al valor que ésta regrese en`%EBX`) a la siguiente instrucción, o saltar a la etiqueta `FAIL`. Esto puede hacerse con la instrucción `BNE $0,%EBX, FAIL` (*Branch if Not Equal*, *saltar si no es igual*, que recibe como argumentos dos valores a ser comparados, en este caso el registro`%EBX` y el número 0, y la etiqueta destino, `FAIL`). Cambiar la dirección destino significa burlar la verificación.

Por otro lado, el atacante podría usar la segunda técnica para lograr que el sistema haga algo más complejo —por ejemplo, que ejecute código arbitrario que él proporcione. Para esto, el ataque más frecuente es saltar *hacia adentro del stack*.

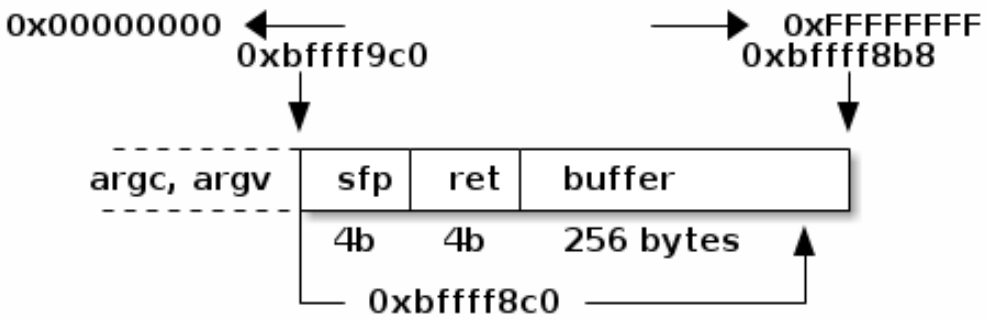


Figura 35.3: Ejecutando el código arbitrario inyectado al buffer.

Para hacerlo, si en vez de proporcionar simplemente una cadena suficientemente grande para sobrepasar el buffer se *inyecta* una cadena con código ejecutable válido, y sobreescribiera la dirección de retorno con la dirección de su código *dentro del buffer*, tendría 256 bytes de espacio para especificar código arbitrario. Este código típicamente se llama *shellcode*, pues se emplea para obtener un *shell* (un intérprete de comandos) que ejecuta con los privilegios del proceso explotado. Este escenario se ilustra en la figura 35.3.

35.1.4. Mecanismos de mitigación

Claro está, el mundo no se queda quieto. Una vez que estos mecanismos de ataque se dieron a conocer, comenzó un fuerte trabajo para crear mecanismos de mitigación de daños.

La principal y más importante medida es crear una cultura de programadores conscientes y prácticas seguras. Esto cruza necesariamente el no emplear funciones que no hagan verificación de límites. La desventaja de esto es que hace falta cambiar al *factor humano*, lo cual resulta prácticamente imposible de lograr con suficiente profundidad.[2] Muchos desarrolladores esgrimen argumentos en contra de estas prácticas, como la pérdida de rendimiento que estas funciones requieren, y muchos otros sencillamente nunca se dieron por enterados de la necesidad de programar correctamente.

Por esto, se han ido creando diversos mecanismos automatizados de protección ante los desbordamientos de buffer. Ninguno de estos mecanismos es *perfecto*, pero sí ayudan a reducir los riesgos ante los atacantes menos persistentes o habilidosos.

35.1.5. Secciones de datos no ejecutables

En secciones anteriores se describió la protección que puede imponer la MMU por regiones, evitando la modificación de código ejecutable.

En la arquitectura x86, dominante en el mercado de computadoras personales desde hace muchos años, esta característica existía en varios procesadores basados en el modelo de segmentación de memoria, pero desapareció al cambiarse el modelo predominante por uno de memoria plana paginada, y fue hasta alrededor

[2] El ejemplo más claro de este problema es la función *gets*, la cual sigue siendo enseñada y usada en los cursos básicos de programación en C.

del 2001 en que fue introducida de vuelta, bajo los nombres *bit NX* (*Never eXecute*, nunca ejecutar) o *bit XD* (*eXecute Disable*, deshabilitar ejecución), como una característica particular de las extensiones PAE.

Empleando este mecanismo, la MMU puede evitar la ejecución de código en el área de *stack*, lo cual anula la posibilidad de *saltar al stack*. Esta protección desafortunadamente no es muy efectiva: una vez que tiene acceso a un buffer vulnerable, el atacante puede *saltar a libc*, esto es, por ejemplo, proporcionar como parámetro el nombre de un programa a ejecutar, e indicar como retorno la dirección de la función `system` o `execve` de la `libc`.

Las secciones de datos no ejecutables son, pues, un obstáculo ante un atacante, aunque no representan una dificultad mucho mayor.

35.1.6. Aleatorización del espacio de direcciones

Otra técnica es que, en tiempo de carga y a cada ejecución, el proceso reciba diferentes direcciones base para sus diferentes áreas. Esto hace más difícil para el atacante poder indicar a qué dirección destino se debe saltar.

Un atacante puede emplear varias técnicas para ayudarse a *adivinar* detalles acerca del acomodo en memoria de un proceso, y, con un buffer suficientemente grande, es común ver *cadenas de NOP*, esto es, una extensión grande de operaciones nulas, seguidas del *shellcode*, para aumentar las probabilidades de que el control se transfiera a un punto útil.

35.1.7. Empleo de *canarios*

Se llama *canario* a un valor aleatorio de protección,[3] insertado entre los buffers y la dirección de retorno, que es verificado antes de regresar de una función. Si se presentó un desbordamiento de buffer, el valor del *canario* será reemplazado por basura, y el sistema podrá detener la ejecución del proceso comprometido antes de que brinde privilegios elevados al atacante. La figura 35.4 ilustra este mecanismo.

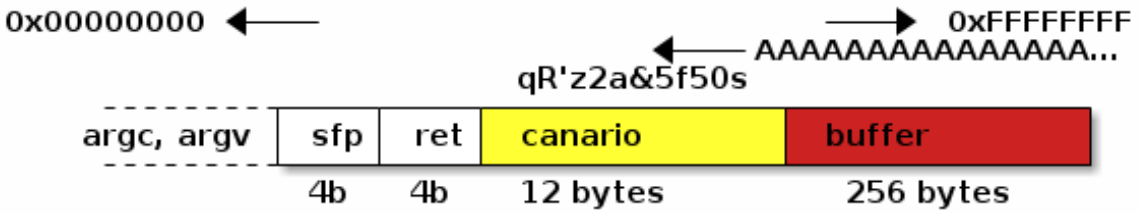


Figura 35.4: Marco de *stack* con un *canario* aleatorio protector de 12 bytes (`qR'z2a&5f50s`): si este es sobreescrito por un buffer desbordado, se detendrá la ejecución del programa.

Un atacante tiene dos mecanismos ante un sistema que requiere del canario: uno es el atacar no directamente a la función en cuestión, sino al *manejador de señales* que es notificado de la anomalía, y otro es, ya que se tiene acceso a la memoria del proceso, *averiguar el valor del canario*. Esto requiere ataques bastante más sofisticados que los vistos en esta sección, pero definitivamente ya no fuera del alcance de los atacantes.

[3] Este uso proviene de la costumbre antigua de los mineros de tener un canario en una jaula en las minas. Como el canario es muy sensible ante la falta de oxígeno, si el canario moría servía como indicador a los mineros de que debían abandonar la mina de inmediato, antes de correr la misma suerte.

35.2. Ligado estático y dinámico de bibliotecas

Las *bibliotecas de código* (o simplemente *bibliotecas*) implementan el código de una serie de funcionalidades generales, que pueden ser usadas en diferentes programas y contextos. Un ejemplo clásico sería la biblioteca estándar de C, la cual ofrece funciones básicas de entrada/salida, manejo de cadenas, entre otras.

A medida que el software crece en complejidad, los programadores recurren a la *reutilización de código* para aprovechar la implementación de la funcionalidad que ofrecen las distintas bibliotecas. De esta forma, evitan "reinventar la rueda", y se concentran en la funcionalidad específica del software que están construyendo.

El concepto de *ligado* se refiere al proceso mediante el cual, se toma el *código objeto* de un programa junto con el código de las bibliotecas que éste usa para crear un archivo ejecutable. De forma general hay dos tipos de ligado, que se explican a continuación.

El *ligado estático* consiste en tomar el código de una biblioteca e integrarlo al código del programa para generar el archivo ejecutable. Lo anterior implica que *cada programa* tiene su propia copia del código de la biblioteca, lo cual puede causar un desperdicio de memoria y disco si hay muchos programas que usan la misma versión de ésta.

Por su parte, en el *ligado dinámico* el código de las bibliotecas no se copia dentro de la imagen ejecutable del programa, pero requiere establecer algún mecanismo para informar que el programa necesita un código externo. Esto se puede implementar de diferentes formas. Por ejemplo, se puede incluir un fragmento de código dentro del programa que usa la biblioteca denominado *stub*, el cual en tiempo de ejecución solicita que se cargue la biblioteca requerida. Otra estrategia que se puede utilizar consiste en incluir algunas indicaciones que le permiten al sistema operativo, en el momento de crear el proceso, ubicar las bibliotecas que este requerirá para su ejecución. En cualquier caso, el ligado dinámico busca que las bibliotecas sólo sean cargadas cuando se las requiera.

La figura 30.4 (p. 149) ilustra el momento en que ocurre cada uno de estos ligados: el ligado estático es realizado por el *editor de ligado*, uniendo en un solo *módulo cargable* al programa compilado (*módulo objeto*) con las bibliotecas (*otros objetos*); el ligado dinámico es realizado parcialmente en tiempo de carga (para las *bibliotecas del sistema*) y parcialmente en tiempo de ejecución (para las *bibliotecas de carga dinámica*).[4]

35.2.1. Las bibliotecas y la seguridad

El ligado dinámico puede traer consigo una serie de problemas, entre los cuales se destacan el manejo de versiones de las bibliotecas y potenciales vulnerabilidades. El primer problema es conocido, en ambientes Windows, como el *infierno de las DLL*. éste se puede causar de muchas formas. Por ejemplo, si al instalar un nuevo programa, se instala también una versión incompatible de una biblioteca que es usada por otros programas. Esto causa que los demás programas no se puedan ejecutar, y lo que es más, hace que la depuración del fallo sea muy difícil. Por otro lado, si no se tienen los controles suficientes, al desinstalar un programa se puede borrar una biblioteca compartida, lo cual puede llevar a que otros programas dejen de funcionar.

El *infiero de las DLL* puede ser prevenido mediante estrategias como el *versionamiento* de las biblioteca de ligado dinámico (esto es, hacer que cada componente de las bibliotecas lleve la versión que implementa o *nivel de compatibilidad* que implementa),[5] y mediante el uso de scripts de instalación o *gestores de dependencias* que verifican si en el sistema hay una versión compatible de la biblioteca. Teniendo esta información, la biblioteca en cuestión se instalará únicamente en caso necesario.

[4] Refiérase al libro *Linkers and Loaders* (ligadores y cargadores) de John R. Levine (1999) para mayores detalles respecto a este proceso.

[5] Este nivel de compatibilidad incluye no sólo a la *interfaz de aplicación al programador* (API, definida en las secciones 15 y 15.1), sino también la *interfaz de aplicación binaria* (ABI), esto es, no sólo la información del nombre de las funciones que expone y los tipos de argumentos que reciben, sino también la ubicación en memoria de su definición en un archivo ya compilado.

El ligado dinámico puede presentar problemas o vulnerabilidades debido a que el programa usa un código proporcionado por terceros, y *confía* en que la biblioteca funciona tal como se espera sin incluir código malicioso. Por tal razón, desde el punto de vista teórico bastaría que un atacante instale su propia versión de una biblioteca para que pueda tener el control de los programas que la usan e incluso del mismo sistema operativo.[6] En el caso de bibliotecas ligadas *estáticamente*, dado que estas forman ya parte del programa, un atacante tendría que modificar al archivo objeto mismo del programa para alterar las bibliotecas.

Así las cosas, más allá de la economía de espacio en memoria, ¿cómo se explica que sea tanto más popular el ligado dinámico en los sistemas operativos modernos?

Parte muy importante de la respuesta es la *mantenibilidad*: si es encontrado un fallo en una biblioteca de carga dinámica, basta con que los desarrolladores lo corrijan una vez (cuidando, claro, de mantener la compatibilidad binaria) y reemplazar a dicha biblioteca en disco *una sola vez*. Todos los programas que liguen dinámicamente con esta biblioteca tendrán disponible de inmediato la versión actualizada. En mucho sistemas operativos, el *gestor de paquetes* puede detectar cuáles procesos en ejecución emplean a determinada biblioteca dinámica, y reiniciarlos de forma transparente al administrador.

En contraste, de estar el fallo en una biblioteca de ligado estático, el código afectado estaría incluido como parte de *cada uno de los programas* ligados con ella. Como consecuencia, para corregir este defecto, cada uno de los programas afectados tendría que ser recompilado (o, por lo menos, *religado*) antes de poderse beneficiar de las correcciones.

Y si bien este proceso resulta manual y tedioso para un administrador de sistemas con acceso a las fuentes de los programas que tiene instalados, resulta mucho más oneroso aún para quienes emplean software *propietario* (en la sección A.3 se aborda con mayor detenimiento lo que significa el software propietario en contraposición al software libre).

Cabe mencionar que el comportamiento del sistema ante la actualización de una biblioteca descrita ilustra una de las diferencias semánticas entre sistemas Windows y sistemas Unix que serán abordadas en el capítulo VI: mientras un sistema Unix permite la eliminación de un archivo *que está siendo utilizado*, Windows no la permite. Esto explica por qué las actualizaciones de bibliotecas en sistemas Windows se aplican *durante el proceso de apagado*: mientras haya procesos que tienen abierta una biblioteca, ésta no puede ser reemplazada. Caso contrario en sistemas Unix, en que el archivo puede ser sustituido, pero mientras no sean reiniciados los procesos en cuestión, éstos seguirán ejecutando la versión de la biblioteca con el error.

Capítulo 36

Ejercicios

36.1. Preguntas de autoevaluación

1. Diagrame el acomodo del espacio en memoria de un proceso. ¿Qué diferencias principales y qué similitudes tienen la *sección de datos* con el *espacio de libres* (*heap*), y entre el *espacio de libres* con la *pila* (*stack*)?

[6] Esto se puede lograr, por ejemplo, alterando la configuración del entorno en la cual el sistema busca las bibliotecas.

2. Un proceso en un sistema con arquitectura de memoria basada en la *segmentación* tiene la siguiente tabla de segmentos:

Segmento	Inicio	Tamaño	Permisos
0	240	600	RX
1	2 300	16	R
2	90	100	RW
3	1 320	950	RW
4	-	96	RX

   Para cada una de las siguientes solicitudes, indique qué dirección física correspondería, y –de ser el caso– qué excepción se genera.

   a) Lectura, 0-430

   b) Escritura, 0-150

   c) Lectura, 1-15

   d) Escritura, 2-130

   e) Ejecución, 4-25

3. El buffer de traducción adelantada (TLB) de un sistema en particular presenta una efectividad de 95%. Obtener un valor del TLB toma 10ns. La memoria principal tarda 120ns en recuperar un valor. ¿Cuál es el tiempo promedio para completar una operación a memoria?

4. Con la siguiente cadena de referencia, y empleando cuatro marcos de memoria física, desarrolle la asignación bajo los esquemas FIFO, OPT y LRU:

   1, 3, 2, 1, 5, 3, 4, 1, 5, 2, 6, 7, 5, 7, 2, 5, 3, 5, 3, 1

   Asumiendo que cada fallo de página toma ocho milisegundos en ser atendido, ¿qué diferencia en rendimiento puede observarse?

5. Suponga un sistema paginado con un rango de direcciones de 4 GB (4 294 967 296 direcciones).

   - ¿Cuántas páginas tendrá el sistema si se utilizan páginas de 4 096 bytes?
   - ¿Qué tamaño (en bits) tendrá una entrada de la tabla de traducción? Suponga que sólo se guarda el número de marco físico.
   - ¿Qué tamaño tendrá la tabla de paginación si se desea cubrir todo el rango?
   - Suponga que el tamaño de la tabla de paginación fuera demasiado grande. Proponga dos soluciones explicando ventajas y desventajas de cada una.

6. Explique la relación que hay entre direcciones virtuales y direcciones físicas. Indique cómo se realiza la traducción en los siguientes casos:

   - Una computadora con TLB y tabla de paginación de un nivel, con la entrada cargada en la TLB.
   - Una computadora con TLB y tabla de paginación de un nivel y sin la entrada cargada.
   - Una computadora sin TLB y con tabla de paginación de dos niveles.
   - Una computadora sin paginación ni TLB pero con segmentación.

7. Un equipo presenta rendimiento muy deficiente. Ante un análisis de utilización a lo largo de un día, se encuentra que el promedio de uso de CPU está a 20% y el uso de la interfaz al disco duro que aloja a la memoria virtual a 95%. ¿En qué condición está el sistema?

   Elija de la siguiente lista las dos respuestas que *mayor efectividad* tendrían para mejorar el rendimiento del sistema. Fundamente con una breve explicación.

   a) Reemplazar al CPU por uno 20% más rápido (pasar de 1 GHz a 1.2 GHz).

   b) Destinar un área del disco 25% más grande para la memoria virtual (pasar de 4 a 5 GB).

   c) Aumentar el grado de multiprogramación en 10% (aumentar de 20 a 22 los procesos en la cola de ejecución).

   d) Reducir el grado de multiprogramación en 10% (reducir de 20 a 18 los procesos en la cola de ejecución).

   e) Instalar un 25% más de memoria principal (pasar de 2 a 2.5 GB).

   f) Instalar un disco duro un 33% más rápido (cambiar un disco de 5 400 RPM por uno de 7 200 RPM).

8. Describa en qué consiste un ataque de *desbordamiento de pila* (*stack overflow*), y un mecanismo de protección del sistema para contrarrestarlos. ¿Puede sugerir una manera en que un atacante podría burlar dicha protección?

36.2. Temas de investigación sugeridos

Esquemas de asignación de memoria en una realidad NUMA La realidad que se expuso en el capítulo respecto al multiprocesamiento simétrico como fuertemente dominante en relación a los sistemas NUMA se mantiene cierta... Pero va cambiando rápidamente, y los sistemas NUMA son cada vez más comunes.

Claro está, la popularización de los sistemas NUMA tiene un alto efecto en cómo se manejan los esquemas de administración de memoria. Al entrar en juego la *afinidad* de cada proceso a un CPU dado, la administración de la memoria y el planificador (despachador) de procesos quedan fuertemente interrelacionados.

En el número de septiembre del 2013 de la revista "Communications of the ACM" aparece un artículo corto, conciso y bastante interesante: "An overview of non-uniform memory access" (Lameter 2013). Sugerimos emplearlo como punto de partida.

Cargado y ligado de programas Este tema ocupa un punto intermedio entre el estudio de los sistemas operativos y el de los compiladores. Para los lectores del presente libro, comprender este punto común puede resultar de interés.

Al convertir un programa escrito en determinado lenguaje de programación por un humano en código ejecutable, no sólo hay que convertir las instrucciones y proveer las abstracciones mínimas, también es necesario que preparar al código para poder ser *reubicado* en la memoria.

El proceso de *cargado* se refiere a cómo la imagen binaria del programa que está grabada en un medio de almacenamiento es modificada al llevarse a memoria para adecuarse al espacio que le asigna el sistema operativo; el proceso de *ligado* es cómo se maneja su integración con las bibliotecas compartidas.

En este proceso entran temas como la gestión de *memoria compartida*, las *indicaciones al compilador* para generar *código independiente de su posición*, etcétera.

Mecanismos para mantener la coherencia en caché En un entorno multiprocesador, el acceso concurrente a las mismas posiciones de memoria se vuelve muy complicado, por decir lo menos. Los distintos niveles de memoria caché tienen que sincronizar su operación (a altísima velocidad) y permitir que avancen los procesos compartiendo datos.

Hay varios mecanismos para mantener la coherencia. Algunas líneas para comenzar la investigación pueden ser, para computadoras razonablemente pequeñas, los mecanismos *fisgones* (*snoopy*) de invalidación y de actualización, y para equipos más grandes, los mecanismos por hardware de *directorio*; los protocolos *fisgones* son también conocidos por sus siglas, o por las universidades donde fueron diseñados: MSI, MESI (Illinois), MOSI (Berkeley), MOESI y MESIF.

Relación con sistemas operativos en uso Este capítulo es probablemente el que mejor permite apreciar de forma directa la relación entre los temas cubiertos y su manejo en un sistema operativo actual. Una manera sencilla de comprender el efecto de los diversos parámetros del subsistema de memoria virtual es la empírica: ¿Cómo reacciona un sistema operativo de propósito general ante cambios en los distintos valores y umbrales; cuáles serían las condiciones límite; qué comportamientos esperaríamos llevando al sistema a estos extremos, y cuáles encontramos en la realidad; cómo es la interfaz al administrador del sistema?

Prácticamente todos los sistemas operativos que se emplean hoy permiten ajustar estos valores, y la comparación entre sistemas operativos distintos seguramente resultará también interesante al lector; en el caso de Linux, se sugiere revisar la documentación de los parámetros de *sysctl* relativos a la memoria virtual. Éstos están documentados en (van Riel y Morreale 1998-2008).

Otras categorías de vulnerabilidad Este capítulo presentó el *desbordamiento de pila*, una de las vulnerabilidades clásicas y que más se siguen explotando al día de hoy. Se presentaron algunos mecanismos de mitigación, pero la conclusión es lapidaria: la correcta protección de límites es de responsabilidad exclusiva e ineludible del desarrollador.

Sin embargo, el desbordamiento de pila no es la única vulnerabilidad. Hay varias vulnerabilidades relacionadas con el acomodo o el tamaño específico de los datos en la memoria. Dos ejemplos:

- En agosto del 2013, se descubrió lo que se popularizó como una *cadena Unicode de la muerte*: un conjunto de caracteres que causa que cualquier programa que intente desplegarlos en pantalla en la línea de productos Apple se *caiga*. El artículo publicado por The Register (Williams 2013) relata el proceso que llevó a averiguar, a partir del *reporte de falla* generado por el sistema y analizando el contenido de la memoria que éste reporta, cómo se puede encontrar un *desbordamiento de entero*.
- En abril de 2014 se dio a conocer un fallo en la biblioteca criptográfica OpenSSL, particularmente en la rutina que implementa el *Heartbeat*. Dada la gran base instalada de usuarios de OpenSSL, y lo sensible de la información que expone al atacante, desde su descubrimiento se dio por hecho que ésta es una de las vulnerabilidades con mayor efecto en escala mundial. Dado el renombre que implica descubrir una vulnerabilidad de este tamaño, los descubridores de este fallo montaron el sitio http://heartbleed.com/ donde hay todo tipo de ligas describiendo esta problemática; otro artículo,también publicado por The Register (Williams 2014), explica la problemática explotada muy claramente.

36.3. Lecturas relacionadas

- Jonathan Corbet (2004a). *The Grumpy Editor goes 64-bit*. URL: `https://lwn.net/Articles/79036/`. Experiencia del editor de Linux Weekly News al migrar a una arquitectura de 64 bits en 2004. Lo más

interesante del artículo son los comentarios, ilustran buena parte de los pros y contras de una migración de 32 a 64 bits.

- Ian J. Campbell (2013). *Using Valgrind to debug Xen Toolstacks*. URL: http://www.hellion.org.uk/blog/posts/using-valgrind-on-xen-toolstacks/. Presenta un ejemplo de uso de la herramienta *Valgrind*, para encontrar problemas en la asignación, uso y liberación de memoria en un programa en C.
- John L. Males (2014). *Process memory usage*. URL: http://troysunix.blogspot.com/2011/07/process-memory-usage.html presenta ejemplos del uso de pmap en diferentes sistemas Unix.
- NetBSD Project (2009). *pmap – display process memory map*. URL: http://www.daemon-systems.org/man/pmap.1.html. Más allá de simplemente mostrar la operación de una herramienta del sistema en Unix, esta página de manual ilustra claramente la estructura de la organización de la memoria.
- John R. Levine (1999). *Linkers and Loaders*. Morgan-Kauffman. ISBN: 1-55860-496-0. URL: http://www.iecc.com/linker/
- L. A. Belady, R. A. Nelson y G. S. Shedler (1969). «An Anomaly in Space-time Characteristics of Certain Programs Running in a Paging Machine». En: *Commun. ACM* 12.6, págs. 349-353. ISSN: 0001-0782. DOI: 10.1145/363011.363155. URL: http://doi.acm.org/10.1145/363011.363155
- Mel Gorman (2004). *Understanding the Linux Virtual Memory Manager*. Bruce Perens' Open Source Series. Pearson Education. ISBN: 0-13-145348-3. Libro de libre descarga y redistribución. Aborda a profundidad los mecanismos y algoritmos relativos a la memoria empleados por el sistema operativo Linux. Entra en detalles técnicos a profundidad, presentándolos poco a poco, por lo que no resulta demasiado complejo de leer. El primer tercio del libro describe los mecanismos, y los dos tercios restantes siguen el código comentado que los implementa.
- William Swanson (2003). *The art of picking Intel registers*. URL: http://www.swansontec.com/sregisters.html
- Enrique Sánchez (Inédito). *The Tao of Buffer Overflows*. URL: http://sistop.gwolf.org/biblio/The_Tao_of_Buffer_Overflows_-_Enrique_Sanchez.pdf
- Aleph One (1996). «Smashing the stack for fun and profit». En: *Phrack Magazine*. URL: http://phrack.org/issues/49/14.html,
- Kingcopes (2013). *Attacking the Windows 7/8 Address Space Randomization*. URL: http://kingcope.wordpress.com/2013/01/24/ Explica cómo puede burlarse la protección basada en aleatorización de direcciones (ALSR) en Windows 7 y 8, logrando una dirección predecible de memoria hacia la cual saltar.
- Christoph Lameter (2013). «An Overview of Non-uniform Memory Access». En: *Commun. ACM* 56.9, págs. 59-54. ISSN: 0001-0782. DOI: 10.1145/2500468.2500477. URL: http://doi.acm.org/10.1145/2500468.2500477

Parte VI

Organización de archivos

Capítulo 37

Introducción

De los papeles que cumple el sistema operativo, probablemente el que más consciente tengan en general sus usuarios es el de la gestión del espacio de almacenamiento, esto es, la organización de la información en un *sistema de archivos*. Al día de hoy, todos los usuarios de equipo de cómputo dan por sentado y comprenden a grandes rasgos la organización del espacio de almacenamiento en un *directorio jerárquico*, con unidades de almacenamiento llamadas *archivos*, de diferentes tipos según su función. En el presente capítulo se revisará la semántica que compone a este modelo, para que en el capítulo VII se continúe con los detalles de la gestión del espacio físico donde éstos están alojados.

La abstracción que hoy se conoce como *sistemas de archivos* es una de las que más tiempo ha vivido y se ha mantenido a lo largo de la historia de la computación, sobreviviendo a lo largo de prácticamente todas las generaciones de sistemas operativos. Sin embargo, para poder analizar cómo es que el sistema operativo representa la información en el dispositivo físico, el presente capítulo inicia discutiendo cómo es que esta información es comprendida por los niveles más altos —por los programas en espacio de usuario.

La información *cruda* tiene que pasar una serie de transformaciones. Yendo de niveles superiores a los más bajos, un programa estructura sus datos en *archivos*, siguiendo el *formato* que resulte más pertinente al tipo de información a representar. Un conjunto de archivos hoy en día es típicamente representado en una estructura de *directorios*.[1]

Cada dispositivo empleado para almacenar archivos tiene un directorio. Cuando un sistema opera con más de un dispositivo físico, hay principalmente dos mecanismos para integrar a dichos dispositivos en un *sistema de archivos virtual*,[2] brindando al usuario una interfaz uniforme. Por último, los archivos son una estructura meramente lógica; deben ser convertidos para ser representados en un *dispositivo de bloques* como los diversos tipos de unidades –aunque esta nomenclatura es a veces incorrecta– como *discos*. Este último paso será abordado en el capítulo VII.

Del diagrama presentado en la figura 37.1, toca al objeto de estudio de esta obra –el sistema operativo–

[1] Se tienen otros mecanismos para su organización, pero éstos no están tan ampliamente difundidos.

[2] Esto será abordado en la sección 39.3.

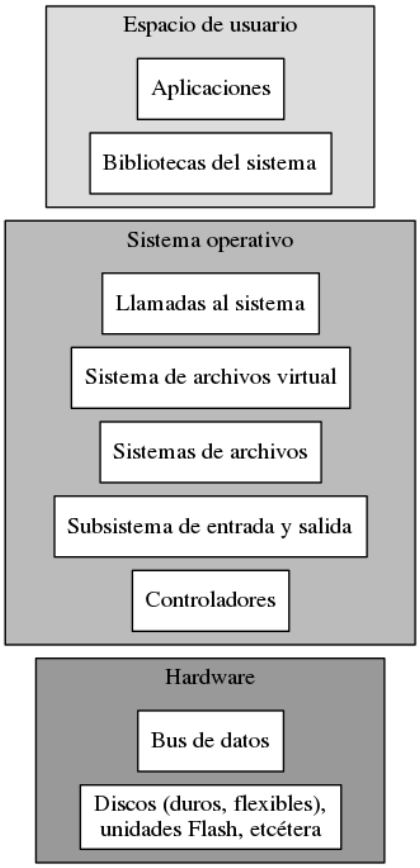


Figura 37.1: Capas de abstracción para implementar los sistemas de archivos.

recibir del espacio de usuario las llamadas al sistema que presentan la interfaz de archivos y directorios, integrar el sistema de archivos virtual y traducir la información resultante a un sistema de archivos.

Cabe mencionar que varias de las capas aquí presentadas podrían perfectamente ser subdivididas, analizadas por separado, e incluso tratarse de forma completamente modular —de hecho, este es precisamente el modo en que se implementan de forma transparente características hoy en día tan comunes como sistemas de archivos en red, o compresión y cifrado de la información. Una referencia más detallada acerca de ventajas, desventajas, técnicas y mecanismos de la división y comunicación entre capas puede ubicarse en (Heidemann y Popek 1994).

Capítulo 38

Concepto de archivo

En primer término, un archivo es un *tipo de datos abstracto* —esto es, podría verse como una estructura que exclusivamente permite la manipulación por medio de una interfaz *orientada a objetos*: los procesos en el sistema sólo pueden tener acceso a los archivos por medio de la interfaz ofrecida por el sistema operativo.[1] La siguiente sección describe las principales operaciones provistas por esta interfaz.

Para el usuario, los archivos son la *unidad lógica mínima* al hablar de almacenamiento: todo el almacenamiento *persistente* (que sobrevive en el tiempo, sea a reinicios del sistema, a pérdida de corriente o a otras circunstancias en el transcurso normal de ejecución) en el sistema al que tiene acceso, se efectúa dentro de archivos; el espacio libre en los diferentes dispositivos no tiene mayor presencia fuera de saber que está *potencialmente* disponible.

Dentro de cada *volumen* (cada medio de almacenamiento), los archivos disponibles conforman un *directorio*, y son típicamente identificados por un *nombre* o una *ruta*. Más adelante se presentarán de las diferentes construcciones semánticas que pueden conformar a los directorios.

38.1. Operaciones con archivos

Cada sistema operativo definirá la interfaz de archivos acorde con su semántica, pero en líneas generales, las operaciones que siempre estarán disponibles con un archivo son:

Borrar Elimina al archivo del directorio y, de ser procedente, libera el espacio del dispositivo

Abrir Solicita al sistema operativo verificar si el archivo existe o puede ser creado (dependiendo del modo requerido) y se cuenta con el acceso para el *modo de acceso* al archivo indicado y si el medio lo soporta (por ejemplo, a pesar de contar con todos los permisos necesarios, el sistema operativo no debe permitir abrir para escritura un archivo en un CD-ROM u otro medio de sólo lectura). En C, esto se hace con la función `fopen()`.

Al abrir un archivo, el sistema operativo asigna un *descriptor de archivo* que identifica la relación entre el proceso y el archivo en cuestión; éstos serán definidos propiamente en la sección 38.2.

Todas las operaciones descritas a continuación operan sobre el descriptor de archivo, no con su nombre o ruta.

Cerrar Indica al sistema que *el proceso en cuestión* terminó de trabajar con el archivo; el sistema entonces debe escribir los buffers a disco y eliminar la entrada que representa a esta combinación archivo-proceso de las tablas activas, invalidando al *descriptor de archivo*. En C, para cerrar un descriptor de archivo se usa `fclose()`.

Dado que todas las operaciones se realizan por medio del descriptor de archivo, si un proceso cierra un archivo y requiere seguir utilizándolo, tendrá que abrirlo de nuevo para obtener un nuevo descriptor.

[1] Como se verá en la sección 43.3.1, esto no es *necesariamente* así, sin embargo, el uso de los dispositivos *en crudo* es muy bajo. Este capítulo está enfocado exclusivamente al uso estructurado en sistemas de archivos.

Leer Si se solicita al sistema la lectura de un archivo hacia determinado buffer, éste copia el siguiente *pedazo* de información. Este *pedazo* podría ser una línea o un bloque de longitud definida, dependiendo del modo en que se solicite la lectura. El sistema mantiene un apuntador a la última posición leída, para poder *continuar* con la lectura de forma secuencial.

La función que implementa la lectura en C es `fread()`. Cabe mencionar que `fread()` entrega el *número de caracteres* especificado; para trabajar con líneas de texto hace falta hacerlo mediante bibliotecas que implementen esta funcionalidad, como `readline`.

Escribir Teniendo un archivo abierto, guarda información en él. Puede ser que escriba desde su primer posición (*truncando* al archivo, esto es, borrando toda la información que pudiera ya tener), o *agregando* al archivo, esto es, iniciando con el apuntador de escritura al final del mismo. La función C para escribir a un descriptor de archivo es `fwrite()`.

Reposicionar Tanto la lectura como la escritura se hacen siguiendo un *apuntador*, que indica cuál fue la última posición del archivo a la que accesó el proceso actual. Al reposicionar el apuntador, se puede *saltar* a otro punto del archivo. La función que reposiciona el apuntador dentro de un descriptor de archivos es `fseek()`.

Hay varias otras operaciones comunes que pueden implementarse con llamadas compuestas a estas operaciones (por ejemplo, *copiar* un archivo puede verse como *crear* un archivo nuevo en modo de escritura, abrir en modo de lectura al archivo fuente, e ir *leyendo* de éste y *escribiendo* al nuevo hasta llegar al fin de archivo fuente).

Las operaciones aquí presentadas no son todas las operaciones existentes; dependiendo del sistema operativo, habrá algunas adicionales; estas se presentan como una base general común a los principales sistemas operativos.

Vale la pena mencionar que esta semántica para el manejo de archivos presenta a cada archivo como si fuera una *unidad de cinta*, dentro de la cual la cabeza lectora/escritora simulada puede avanzar o retroceder.

38.2. Tablas de archivos abiertos

Tanto el sistema operativo como cada uno de los procesos mantienen normalmente *tablas de archivos abiertos*. Éstas mantienen información acerca de todos los archivos actualmente abiertos, presentándolos hacia el proceso por medio de un *descriptor de archivo*; una vez que un archivo fue abierto, las operaciones que se realizan dentro de éste no son empleando su nombre, sino su descriptor de archivo.

En un sistema operativo multitareas, más de un proceso podría abrir el mismo archivo a la vez; lo que cada uno de ellos pueda hacer, y cómo esto repercute en lo que vean los demás procesos, depende de la semántica que implemente el sistema; un ejemplo de las diferentes semánticas posibles es el descrito en la sección 38.3.

Ahora, ¿por qué estas tablas se mantienen tanto por el sistema operativo como por cada uno de los procesos, no lleva esto a una situación de información redundante?

La respuesta es que la información que cada uno debe manejar es distinta. El sistema operativo necesita:

Conteo de usuarios del archivo Requiere saberse cuántos procesos están empleando en todo momento a determinado archivo. Esto permite, por ejemplo, que cuando el usuario solicite *desmontar* una partición (puede ser para expulsar una unidad removible) o eliminar un archivo, el sistema debe poder determinar cuándo es momento de declarar la solicitud como *efectuada*. Si algún proceso tiene abierto un archivo, y particularmente si tiene cambios pendientes de guardar, el sistema debe hacer lo posible por evitar que el archivo *desaparezca* de su visión.

Modos de acceso Aunque un usuario tenga permisos de acceso a determinado recurso, el sistema puede determinar negarlo si llevaría a una inconsistencia. Por ejemplo, si dos procesos abren un mismo archivo en modo de escritura, es probable que los cambios que realice uno sobreescriban a los que haga el otro.

Ubicación en disco El sistema mantiene esta información para evitar que cada proceso tenga que consultar las tablas en disco para encontrar al archivo, o cada uno de sus fragmentos.

Información de bloqueo En caso de que los modos de acceso del archivo requieran protección mutua, puede hacerlo por medio de un bloqueo.

Por otro lado, el proceso necesita:

Descriptor de archivo Relación entre el nombre del archivo abierto y el identificador numérico que maneja internamente el proceso. Un archivo abierto por varios procesos tendrá descriptores de archivo distintos en cada uno de ellos.

Al detallar la implementación, el descriptor de archivo otorgado por el sistema a un proceso es simplemente un número entero, que podría entenderse como *el n-ésimo archivo empleado por el proceso*.[2]

Permisos Los modos válidos de acceso para un archivo. Esto no necesariamente es igual a los permisos que tiene el archivo en cuestión en disco, sino que el *subconjunto* de dichos permisos bajo los cuales está operando para este proceso en particular —si un archivo fue abierto en modo de sólo lectura, por ejemplo, este campo únicamente permitirá la lectura.

38.3. Acceso concurrente: bloqueo de archivos

Dado que los archivos pueden emplearse como mecanismo de comunicación entre procesos que no guarden relación entre sí, incluso a lo largo del tiempo, y para emplear un archivo basta indicar su nombre o ruta, los sistemas operativos multitarea implementan mecanismos de bloqueo para evitar que varios procesos intentando emplear de forma concurrente a un archivo se corrompan mutuamente.

Algunos sistemas operativos permiten establecer bloqueos sobre determinadas regiones de los archivos, aunque la semántica más común es operar sobre el archivo entero.

En general, la nomenclatura que se sigue para los bloqueos es:

Compartido (*Shared lock*) Podría verse como equivalente a un bloqueo (o *candado*) para realizar lectura — varios procesos pueden adquirir al mismo tiempo un bloqueo de lectura, e indica que todos los que posean dicho *candado* tienen la expectativa de que el archivo no sufrirá modificaciones.

Exclusivo (*Exclusive lock*) Un bloqueo o *candado* exclusivo puede ser adquirido por un sólo proceso, e indica que realizará operaciones que modifiquen al archivo (o, si la semántica del sistema operativo permite expresarlo, a la *porción* del archivo que indica).

Respecto al *mecanismo* de bloqueo, hay también dos tipos, dependiendo de qué tan explícito tiene que ser su manejo:

Mandatorio u obligatorio (*Mandatory locking*) Una vez que un proceso adquiere un candado obligatorio, el sistema operativo se encargará de imponer las restricciones corresponidentes de acceso a todos los demás procesos, independientemente de si éstos fueron programados para considerar la existencia de dicho bloqueo o no.

[2] No sólo los archivos reciben descriptores de archivo. Por ejemplo, en todos los principales sistemas operativos, los descriptores 0, 1 y 2 están relacionados a *flujos de datos*: respectivamente, la entrada estándar (`STDIN`), la salida estándar (`STDOUT`) y el error estándar (`STDERR`); si el usuario no lo indica de otro modo, la terminal desde donde fue ejecutado el proceso.

Consultivo o asesor (*Advisory locking*) Este tipo de bloqueos es manejado cooperativamente entre los procesos involucrados, y depende del programador de *cada uno* de los programas en cuestión el solicitar y respetar dicho bloqueo.

Haciendo un paralelo con los mecanismos presentados en el capítulo III, los mecanismos que emplean mutexes, semáforos o variables de condición serían *consultivos*, y únicamente los que emplean monitores (en que la única manera de llegar a la información es por medio del mecanismo que la protege) serían *mandatorios*.

No todos los sistemas operativos implementan las cuatro posibles combinaciones (compartido mandatorio, o compartido consultivo, exclusivo mandatorio y exclusivo consultivo). Como regla general, en los sistemas Windows se maneja un esquema de bloqueo obligatorio, y en sistemas Unix es de bloqueo consultivo.[3]

Cabe mencionar que el manejo de bloqueos con archivos requiere del mismo cuidado que el de bloqueo por recursos cubierto en la sección 22: dos procesos intentando adquirir un candado exclusivo sobre dos archivos pueden caer en un bloqueo mutuo tal como ocurre con cualquier otro recurso.

38.4. Tipos de archivo

Si los archivos son la *unidad lógica mínima* con la que se puede guardar información en almacenamiento secundario, naturalmente sigue que hay archivos de diferentes tipos: cada uno podría ser un documento de texto, un binario ejecutable, un archivo de audio o video, o un larguísimo etcetera, e intentar emplear un archivo como uno de un tipo distinto puede resultar desde una frustración al usuario porque el programa no responde como éste quiere, hasta en pérdidas económicas.[4]

Hay tres estrategias principales para que el sistema operativo reconozca el tipo de un archivo:

Extensión En los sistemas CP/M de los setenta, el nombre de cada archivo se dividía en dos porciones, empleando como elemento separador al punto: el nombre del archivo y su extensión. El sistema mantenía una lista de extensiones conocidas, para las cuales sabría cómo actuar, y este diseño se propagaría a las aplicaciones, que sólo abrirían a aquellos archivos cuyas extensiones supieran manejar.

Esta estrategia fue heredada por VMS y MS-DOS, de donde la adoptó Windows; ya en el contexto de un entorno gráfico, Windows agrega, más allá de las extensiones directamente ejecutables, la relación de extensiones con los programas capaces de trabajar con ellas, permitiendo invocar a un programa con sólo dar "doble clic" en un archivo.

Como nota, este esquema de asociación de tipo de archivo permite ocultar las extensiones toda vez que ya no requieren ser del conocimiento del usuario, sino que son gestionadas por el sistema operativo, abre una vía de ataque automatizado que se popularizó en su momento: el envío de correos con extensiones engañosas duplicadas, esto es, el programa maligno (un *programa troyano*) se envía a todos los contactos del usuario infectado, presentándose por ejemplo como una imágen, con el nombre `inocente.png.exe`. Por el esquema de ocultamiento mencionado, éste se presenta al usuario como `inocente.png`, pero al abrirlo, el sistema operativo lo reconoce como un ejecutable, y lo ejecuta en vez de abrirlo en un visor de imágenes.

[3] Esto explica el que en Windows sea tan común que el sistema mismo rechace hacer determinada operación porque *el archivo está abierto por otro programa* (bloqueo mandatorio compartido), mientras que en Unix esta responsabilidad recae en cada uno de los programas de aplicación.

[4] Por ejemplo, imprimir un archivo binario resulta en una gran cantidad de hojas inútiles, particularmente tomando en cuenta que hay caracteres de control como el ASCII 12 (avance de forma, *form feed*), que llevan a las impresoras que operan en modo texto a iniciar una nueva página; llevar a un usuario a ejecutar un archivo ejecutable *disfrazado* de un documento inocuo, como se verá a continuación, fue un importante vector de infección de muchos virus.

Números mágicos La alternativa que emplean los sistemas Unix es, como siempre, simple y *elegante*, aunque indudablemente presenta eventuales lagunas: el sistema mantiene una lista compilada de las *huellas digitales* de los principales formatos que debe manejar,[5] para reconocer el contenido de un archivo basado en sus primeros bytes.

Casi todos los formatos de archivo incluyen lo necesario para que se lleve a cabo este reconocimiento, y cuando no es posible hacerlo, se intenta por medio de ciertas reglas *heurísticas*. Por ejemplo, todos los archivos de imágen en *formato de intercambio gráfico* (GIF) inician con la cadena `GIF87a` o `GIF89a`, dependiendo de la versión; los archivos del lenguaje de descripción de páginas PostScript inician con la cadena`%!`, el *Formato de Documentos Portátiles* (PDF) con`%PDF`, un documento en formatos definidos alrededor de XML inicia con `<!DOCTYPE`, etcétera. Algunos de estos formatos no están *anclados* al inicio, sino en un punto específico del primer bloque.

Un caso especial de números mágicos es el llamado *hashbang* (`#!`). Esto indica a un sistema Unix que el archivo en cuestión (típicamente un archivo de texto, incluyendo código fuente en algún lenguaje de *script*) debe tratarse como un ejecutable, y empleando como *intérprete* al comando indicado inmediatamente después del *hashbang*. Es por esto que se pueden ejecutar directamente, por ejemplo, los archivos que inician con `#!/usr/bin/bash`: el sistema operativo invoca al programa `/usr/bin/bash`, y le especifica como argumento al archivo en cuestión.

Metadatos externos Los sistemas de archivos empleado por las Apple Macintosh desde 1984 separan en dos *divisiones* (*forks*) la información de un archivo: los datos que propiamente constituyen al archivo en cuestión son la *división de datos* (*data fork*), y la información *acerca del archivo* se guardan en una estructura independiente llamada *división de recursos* (*resource fork*).

Esta idea resultó fundamental para varias de las características *amigables al usuario* que presentó Macintosh desde su introducción, particularmente, para presentar un entorno gráfico que respondiera ágilmente, sin tener que buscar los datos base de una aplicación dentro de un archivo de mucho mayor tamaño. La *división de recursos* cabe en pocos sectores de disco, y si se toma en cuenta que las primeras Macintosh funcionaban únicamente con discos flexibles, el tiempo invertido en leer una lista de iconos podría ser demasiada.

La división de recursos puede contener todo tipo de información; los programas ejecutables son los que le dan un mayor uso, dado que incluyen desde los aspectos gráficos (icono a mostrar para el archivo, ubicación de la ventana a ser abierta, etc.) hasta aspectos funcionales, como la traducción de sus cadenas al lenguaje particular del sistema en que está instalado. Esta división permite una gran flexibilidad, dado que no es necesario tener acceso al fuente del programa para crear traducciones y temas.

En el tema particular que concierne a esta sección, la división de recursos incluye un campo llamado *creador*, que indica cuál programa fue el que generó al archivo. Si el usuario solicita ejecutar un archivo de datos, el sistema operativo lanzaría al programa *creador*, indicándole que abra al archivo en cuestión.

Las versiones actuales de MacOS ya no emplean esta técnica, sino que una llamada *appDirectory*, para propósitos de esta discusión, la técnica base es la misma.

38.5. Estructura de los archivos y métodos de acceso

La razón principal del uso de sistemas de archivos son, naturalmente, *los archivos*. En estos se almacena información de *algún tipo*, con o sin una estructura predeterminada.

[5] Una de las ventajas de este esquema es que cada administrador de sistema puede ampliar la lista con las huellas digitales que requiera localmente.

La mayor parte de los sistemas operativos maneja únicamente archivos *sin estructura* —cada aplicación es responsable de preparar la información de forma congruente, y la responsabilidad del sistema operativo es únicamente entregarlo como un conjunto de bytes. Históricamente, hubo sistemas operativos, como IBM CICS (1968), IBM MVS (1974) o DEC VMS (1977), que administraban ciertos tipos de datos en un formato básico de *base de datos*.

El hecho de que el sistema operativo no imponga estructura a un archivo no significa, claro está, que la aplicación que lo genera no lo haga. La razón por la que los sistemas creados en los últimos 30 años no han implementado este esquema de base de datos es que le *resta* flexibilidad al sistema: el que una aplicación tuviera que ceñirse a los tipos de datos y alineación de campos del sistema operativo impedía su adecuación, y el que la funcionalidad de un archivo tipo base de datos dependiera de la versión del sistema operativo creaba un *acoplamiento* demasiado rígido entre el sistema operativo y las aplicaciones.

Esta práctica ha ido cediendo terreno para dejar esta responsabilidad en manos de procesos independientes en espacio de usuario (como sería un gestor de bases de datos tradicional), o de bibliotecas que ofrezcan la funcionalidad de manejo de archivos estructurados (como en el caso de SQL*ite*, empleado tanto por herramientas de adquisición de datos de bajo nivel como *systemtap* como por herramientas tan de escritorio como el gestor de fotografías *shotwell* o el navegador *Firefox*).

En los sistemas derivados de MS-DOS puede verse aún un remanente de los archivos estructurados: en estos sistemas, un archivo puede ser *de texto* o *binario*. Un archivo de texto está compuesto por una serie de caracteres que forman *líneas*, y la separación entre una línea y otra constituye de un *retorno de carro* (caracter ASCII 13, CR) seguido de un *salto de línea* (caracter ASCII 10, LF).[6]

El acceso a los archivos puede realizarse de diferentes maneras:

Acceso secuencial Mantiene la semántica por medio de la cual permite leer de los archivos, de forma equivalente a como lo harían las unidades de cinta mencionadas en la sección 38.1, y como lo ilustra la figura 38.1: el mecanismo principal para leer o escribir es ir avanzando consecutivamente por los bytes que conforman al archivo hasta llegar a su final.

Típicamente se emplea este mecanismo de lectura para leer a memoria código (programas o bibliotecas) o documentos, sean enteros o fracciones de los mismos. Para un contenido estructurado, como una base de datos, resultaría absolutamente ineficiente, dado que no se conoce el punto de inicio o finalización de cada uno de los registros, y probablemente sería necesario hacer *barridos secuenciales* del archivo completo para cada una de las búsquedas.



Figura 38.1: Archivo de acceso secuencial.

Acceso aleatorio El empleo de gestores como *SQLite* u otros muchos motores de base de datos más robustos no exime al usuario de pensar en el archivo como una tabla estructurada, como lo ilustra la figura 38.2.

[6]Esta lógica es herencia de las máquinas de escribir manuales, en que el *salto de línea* (avanzar el rodillo a la línea siguiente) era una operación distinta a la del *retorno de carro* (devolver la cabeza de escritura al inicio de la línea). En la época de los teletipos, como medida para evitar que se perdieran caracteres mientras la cabeza volvía hasta la izquierda, se decidió separar el inicio de nueva línea en los dos pasos que tienen las máquinas de escribir, para inducir una demora que evitara la pérdida de información.

Si la única semántica por medio de la cual el sistema operativo permitiera trabajar con los archivos fuera la equivalente a una unidad de cinta, implementar el acceso a un punto determinado del archivo podría resultar demasiado costoso.

Afortunadamente, que el sistema operativo no imponga registros de longitud fija no impide que *el programa gestor* lo haga. Si en el archivo al cual apunta el descriptor de archivo `FD` hay 2 000 registros de 75 bytes cada uno y el usuario requiere recuperar el registro número 65 hacia el buffer `registro`, puede *reposicionar* el apuntador de lectura al byte $65 \times 75 = 4\,875$ (`seek(FD, 4875)`) y leer los siguientes 75 bytes en `registro` (`read(FD, *registro, 75)`).

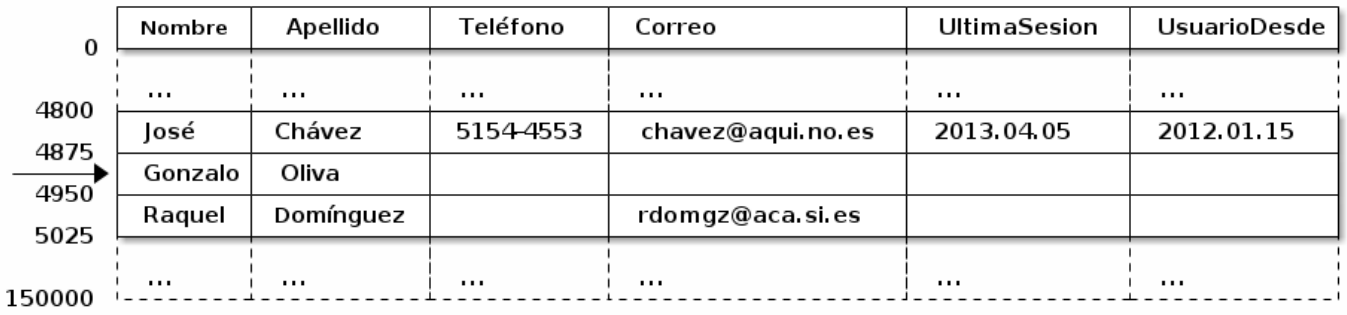


Figura 38.2: Archivo de acceso aleatorio.

Acceso relativo a índice En los últimos años se han popularizado los gestores de base de datos *débilmente estructurados* u *orientados a documentos*, llamados genéricamente *No*SQL. Estos gestores pueden guardar registros de tamaño variable en disco, por lo que, como lo ilustra la figura 38.3, no pueden encontrar la ubicación correcta por medio de los mecanismos de acceso aleatorio.

Para implementar este acceso, se divide al conjunto de datos en dos secciones (incluso, posiblemente, en dos archivos independientes): la primer sección es una lista corta de identificadores, cada uno con el punto de inicio y término de los datos a los que apunta. Para leer un registro, se emplea acceso aleatorio sobre el índice, y el apuntador se avanza a la ubicación específica que se solicita.

En el transcurso de un uso intensivo de esta estructura, dado que la porción de índice es muy frecuentemente consultada y relativamente muy pequeña, muy probablemente se mantenga completa en memoria, y el acceso a cada uno de los registros puede resolverse en tiempo muy bajo.

La principal desventaja de este modelo de indexación sobre registros de longitud variable es que sólo resulta eficiente para contenido *mayormente de lectura*: cada vez que se produce una escritura y cambia la longitud de los datos almacenados, se va generando fragmentación en el archivo, y para resolverla frecuentemente se hace necesario suspender un tiempo la ejecución de todos los procesos que lo estén empleando (e invalidar, claro, todas las copias en caché de los índices). Ahora bien, para los casos de uso en que el comportamiento predominante sea de lectura, este formato tendrá la ventaja de no desperdiciar espacio en los campos nulos o de valor irrelevante para algunos de los registros, y de permitir la flexibilidad de registrar datos originalmente no contemplados sin tener que modificar la estructura.

Es importante recalcar que la escritura en ambas partes de la base de datos (índice y datos) debe mantenerse con garantías de atomicidad — si se pierde la sincronía entre ellas, el resultado será una muy probable corrupción de datos.

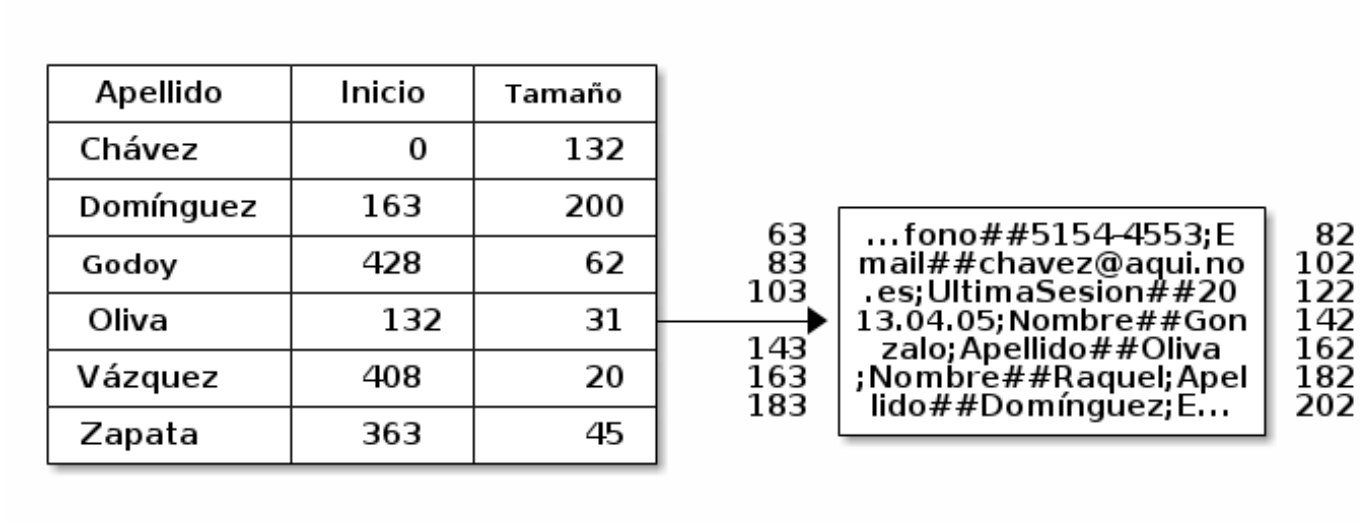


Figura 38.3: Acceso relativo a índice: tabla con apuntadores al lugar justo en un archivo sin estructura.

38.6. Archivos especiales

Los sistemas de archivos son estructuras de tan fácil manejo y comprensión que resulta natural que los diseñadores de sistemas operativos buscaran aprovecharlos para presentar todo tipo de estructuras, no sólo archivos. En este sentido, la máxima Unix, *todo es un archivo*, resulta el ejemplo natural.

Este concepto fue brevemente presentado en la sección 16; complementando dicha sección, en un sistema Unix estándar, un archivo puede pertenecer a las siguientes categorías:

Archivo estándar Aunque suene obvio, el tipo básico de archivo es, precisamente, el archivo. Representa directamente la información que lo conforma. Su semántica de uso es la presentada en el transcurso de este capítulo.

Objetos del sistema de archivos La información acerca del sistema de archivos se almacena también dentro de archivos, aunque su tratamiento es especial (con una reminiscencia de los sistemas históricos mencionados en la sección 38.5): El sistema operativo oculta al archivo *real*, y gestiona su información procesada para que lo emplee el usuario. Este tipo de archivos puede referirse a directorios o a ligas simbólicas. Ambos se detallan en la sección 39.1.

Dispositivos Permiten el acceso directo a un dispositivo externo. Como fue presentado en la sección 16, pueden ser *de bloques* o *de caracteres*. El contenido del archivo son dos números, el *mayor* y el *menor*; tradicionalmente, el mayor indica de qué clase de dispositivo se trata, y el menor, la instancia de que se trata.

Por ejemplo, tanto los discos duros como sus particiones llevan por número mayor el ocho; los números menores para los duros son 0, 16, 32, 48. Cada uno de los discos puede tener hasta 15 particiones: 1 a 15, 17 a 31, etcétera.

Comunicación entre procesos Los archivos pueden representar *tuberías nombradas* y *sockets*. Ambos son mecanismos que permiten intercambiar información entre distintos procesos; la tubería nombrada es más sencilla de emplear, pero permite únicamente la comunicación unidireccional, en tanto que el socket permite comunicación bidireccional como si se tratara de una conexión por red.

38.7. Transferencias orientadas a bloques

Un sistema de archivos es la representación que se da a un conjunto de archivos y directorios sobre un *dispositivo de bloques*, esto es, un dispositivo que, para cualquier transferencia solicitada desde o hacia él, responderá con un bloque de tamaño predefinido.

Esto es, si bien el sistema operativo presenta una abstracción por medio de la cual la lectura (`read()`) puede ser de un tamaño arbitrario, todas las transferencias de datos desde cualquiera de los discos serán de un múltiplo del tamaño de bloques, definido por el hardware (típicamente 512 bytes).

Al leer, como en el ejemplo anterior, sólamente un registro de 75 bytes, el sistema operativo lee el bloque completo y probablemente lo mantiene en un caché en la memoria principal; si en vez de una lectura, la operación efectuada fue una de escritura (`write()`), y el sector a modificar no ha sido leído aún a memoria (o fue leído hace mucho, y puede haber sido expirado del caché), el sistema tendrá que leerlo nuevamente, modificarlo en memoria, y volver a guardarlo a disco.

Capítulo 39

Organización de archivos

Hasta este punto, el enfoque ha sido en qué es y cómo se maneja un archivo. Sin embargo, no tendría sentido hablar de *sistemas de archivos* si no hubiera una gran cantidad de archivos. Es común que un sólo medio de almacenamiento de un equipo de uso casero aloje *decenas de miles* de archivos, y en equipos dedicados, no está fuera de lugar tener cientos o miles de veces tanto. Por lo tanto, se tiene que ver también cómo organizar una gran cantidad de archivos.

39.1. Evolución del concepto de *directorio*

El concepto dominante en almacenamiento hoy en día es el de *directorios jerárquicos*. Esto no siempre fue así; conviene revisar brevemente su historia para explicar la razón de ciertos detalles de implementación del esquema actualmente dominante.

39.1.1. Convenciones de nomenclatura

Cada sistema de archivos puede determinar cuántos y qué caracteres son válidos para designar a uno de sus elementos, y cuáles son separadores válidos. El caracter que se emplea para separar los elementos de un directorio no es un estándar a través de todos los sistemas operativos —los más comunes en uso hoy en día son la diagonal (`/`), empleada en sistemas tipo Unix y derivados (incluyendo MacOS X y Android), y la diagonal invertida (`\`), empleada en CP/M y derivados, incluyendo MS-DOS y Windows.

Diversos sistemas han manejado otros caracteres (por ejemplo, el MacOS histórico empleaba los dos puntos, `:`), y aunque muchas veces los mantenían ocultos del usuario mediante una interfaz gráfica rica, los programadores siempre tuvieron que manejarlos explícitamente.

A lo largo del presente texto se empleará la diagonal (/) como separador de directorios.

39.1.2. Sistema de archivos *plano*

Los primeros sistemas de archivos limitaban el concepto de directorio a una representación plana de los archivos que lo conformaban, sin ningún concepto de *jerarquía de directorios* como el que hoy resulta natural a los usuarios. Esto se debía, en primer término, a lo limitado del espacio de almacenamiento de las primeras computadoras en implementar esta metáfora (por lo limitado del espacio de almacenamiento, los usuarios no dejaban sus archivos a largo plazo en el disco, sino que los tenían ahí meramente mientras los requerían), y en segundo término, a que no se había aún desarrollado un concepto de separación, permisos y privilegios como el que poco después aparecería.

En las computadoras personales los sistemas de archivos eran también planos en un primer momento, pero por otra razón: en los sistemas *profesionales* ya se había desarrollado el concepto; al aparecer la primer computadora personal en 1975, ya existían incluso las primeras versiones de Unix diseñadas para trabajo en red. La prioridad en los sistemas personales era mantener el código del sistema operativo simple, mínimo. Con unidades de disco capaces de manejar entre 80 y 160 KB, no tenía mucho sentido implementar directorios — si un usuario quisiera llevar a cabo una división temática de su trabajo, lo colocaría en distintos *discos flexibles*. El sistema operativo CP/M nunca soportó jerarquías de directorios, como tampoco lo hizo la primer versión de MS-DOS.[1]

El sistema de archivos original de la Apple Macintosh, MFS, estaba construido sobre un modelo plano, pero presentando la *ilusión* de directorios de una forma comparable a las etiquetas: eran representados bajo *ciertas* vistas (aunque notoriamente no en los diálogos de abrir y grabar archivos), pero el nombre de cada uno de los archivos tenía que ser único, dado que el directorio al que pertenecía era básicamente sólo un atributo del archivo.

Y contrario a lo que dicta la intuición, el modelo de directorio plano no ha desaparecido: el sistema de *almacenamiento en la nube* ofrecido por el servicio *Amazon S3* (*simple storage service*, *servicio simple de almacenamiento*) maneja únicamente *objetos* (comparable con la definición de *archivos* que se ha venido manejando) y *cubetas* (de cierto modo comparables con las *unidades* o *volúmenes*), y permite referirse a un objeto o un conjunto de éstos basado en *filtros* sobre el total que conforman una cubeta.

Conforme se desarrollen nuevas interfaces al programador o al usuario, probablemente se popularicen más ofertas como la que hoy hace Amazon S3. Al día de hoy, sin embargo, el esquema jerárquico sigue, con mucho, siendo el dominante.

39.1.3. Directorios de profundidad fija

Las primeras implementaciones de directorios eran *de un sólo nivel*: el total de archivos en un sistema podía estar dividido en directorios, fuera por tipo de archivo (separando, por ejemplo, programas de sistema, programas de usuario y textos del correo), por usuario (facilitando una separación lógica de los archivos de un usuario de pertenecientes a los demás usuarios del sistema)

El directorio *raíz* (base) se llama en este esquema MFD (*master file directory*, *directorio maestro de archivos*), y cada uno de los directorios derivados es un UFD (*user file directory*, *directorio de archivos de usuario*).

Este esquema resuelve el problema principal del nombre global único: antes de los directorios, cada usuario tenía que cuidar que los nombres de sus archivos fueran únicos en el sistema, y ya teniendo cada uno su propio espacio, se volvió una tarea mucho más simple. La desventaja es que, si el sistema restringe a cada usuario a escribir en su UFD, se vuelve fundamentalmente imposible trabajar en algún proyecto conjunto: no

[1] El soporte de jerarquías de directorios fue introducido apenas en la versión 2.0 de dicho sistema operativo, junto con el soporte a discos duros de 10 MB, acompañando al lanzamiento de la IBM PC modelo XT.

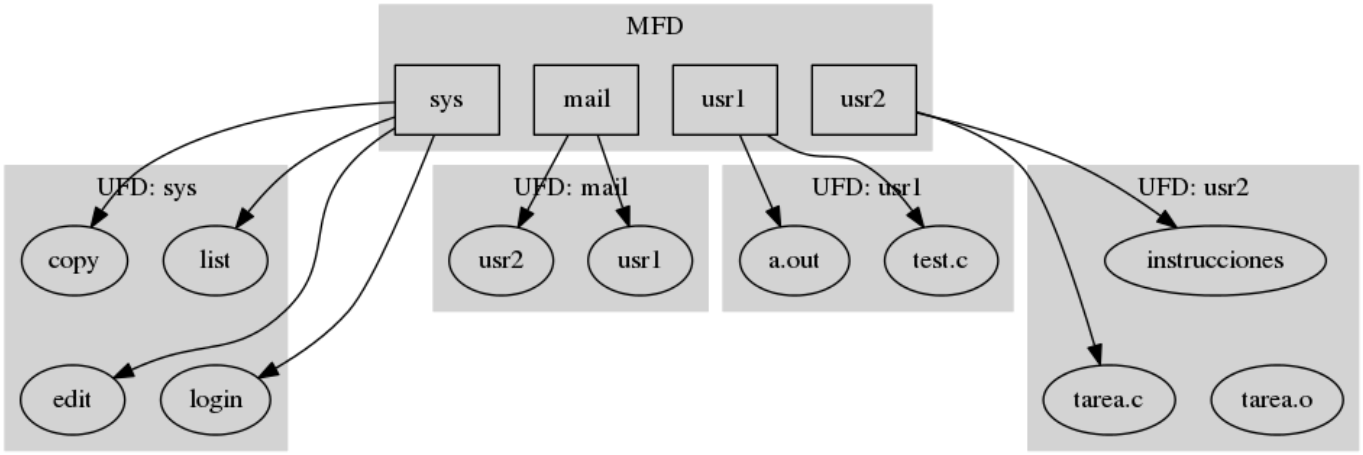


Figura 39.1: Directorio simple, limitado a un solo nivel de profundidad.

puede haber un directorio que esté tanto dentro de `usr1` como de `usr2`, y los usuarios encontrarán más dificil llevar un proyecto conjunto.

39.1.4. Directorios estructurados en árbol

El siguiente paso natural para este esquema es permitir una *jerarquía ilimitada*: en vez de exigir que haya una capa de directorios, se le puede *dar la vuelta* al argumento, y permitir que cada directorio pueda contener a otros archivos o directorios anidados arbitrariamente. Esto permite que cada usuario (y que el administrador del sistema) estructure su información siguiendo criterios lógicos y piense en el espacio de almacenamiento como un espacio a largo plazo.

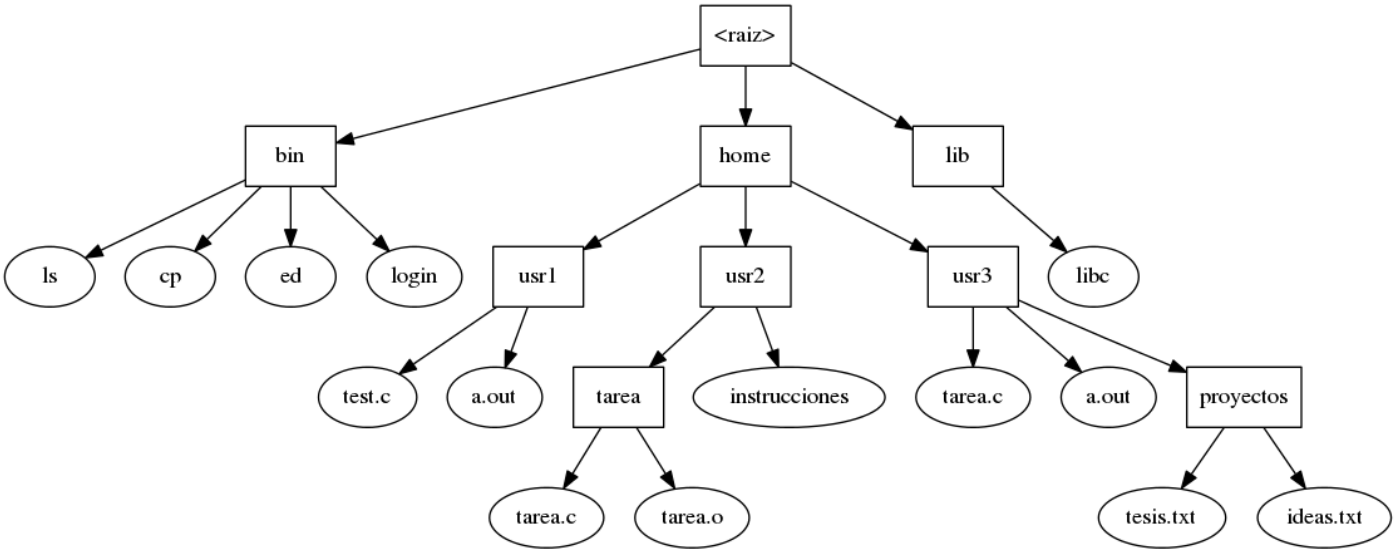


Figura 39.2: Directorio estucturado en árbol.

Junto con esta estructura nacen las *rutas de búsqueda* (*search path*): tanto los programas como las bibliotecas de sistema ahora pueden estar en cualquier lugar del sistema de archivos. Al definirle al sistema una *ruta de búsqueda*, el usuario operador puede desentenderse del lugar exacto en el que está determinado programa

—el sistema se encargará de buscar en todos los directorios mencionados los programas o bibliotecas que éste requiera.[2]

39.1.5. El directorio como un *grafo dirigido*

Si bien parecería que muchos de los sistemas de archivos empleados hoy en día pueden modelarse suficientemente con un árbol, donde hay un sólo nodo raíz, y donde cada uno de los nodos tiene un sólo nodo padre, la semántica que ofrecen es en realidad un *superconjunto estricto* de ésta: la de un grafo dirigido.

En un grafo dirigido como el presentado en la figura 39.3, un mismo nodo puede tener varios directorios *padre*, permitiendo, por ejemplo, que un directorio de trabajo común sea parte del directorio personal de dos usuarios. Esto es, *el mismo objeto* está presente en más de un punto del árbol.

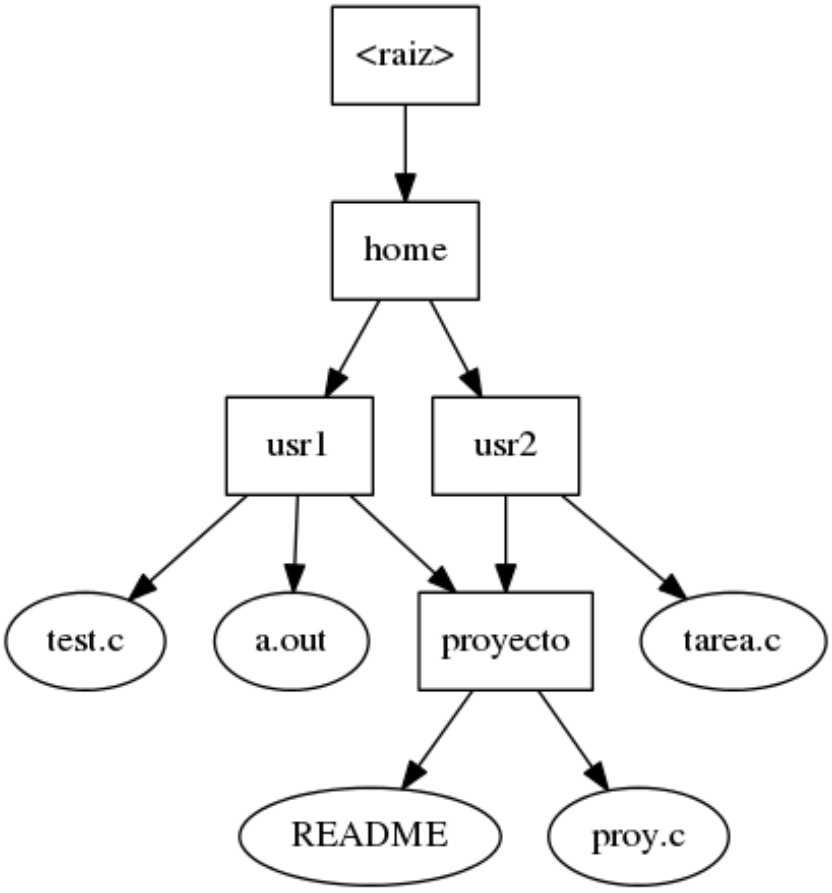


Figura 39.3: Directorio como un *grafo dirigido acíclico*: el directorio `proyecto` está tanto en el directorio `/home/usr1` como en el directorio `/home/usr2`.

Un sistema de archivos puede permitir la organización como un *grafo dirigido*, aunque es común que la interfaz que presenta al usuario[3] se restrinja a un *grafo dirigido acíclico*: las ligas múltiples son permitidas, siempre y cuando no generen un ciclo.

[2]La *ruta de búsqueda* refleja la organización del sistema de archivos en el contexto de la instalación específica. Es común que la ruta de búsqueda de un usuario estándar en Unix sea similar a `/usr/local/bin:/usr/bin:/bin:~/bin` — esto significa que cualquier comando que sea presentado es buscado, en el orden indicado, en los cuatro directorios presentados (separados por el caracter `:`, la notación `~` indica el directorio personal del usuario activo). En Windows, es común encontrar la ruta de búsqueda `c:\WINDOWS\system32;c:\WINDOWS`

[3]Esta simplificación es simplemente una abstracción, y contiene una pequeña mentira, que será desmentida en breve.

La semántica de los sistemas Unix implementa directorios como grafos dirigidos por medio de dos mecanismos:

Liga o enlace duro La entrada de un archivo en un directorio Unix es la relación entre la ruta del archivo y el *número de i-nodo* en el sistema de archivos.[4] Si a partir de un archivo en cualquier punto del árbol se crea una *liga dura* a él, ésta es sencillamente otra entrada en el directorio apuntando al mismo *i-nodo*. Ambas entradas, pues, son el mismo archivo —no hay uno *maestro* y uno *dependiente*.

En un sistema Unix, este mecanismo tiene sólo dos restricciones:

1. Sólo se pueden hacer ligas duras dentro del mismo volumen.
2. No pueden hacerse ligas duras a directorios, sólo a archivos.[5]

Liga o enlace simbólico Es un archivo *especial*, que meramente indica a dónde apunta. El encargado de seguir este archivo a su destino (esto es, de *resolver* la liga simbólica) es el sistema operativo mismo; un proceso no tiene que hacer nada especial para seguir la liga.

Una liga simbólica puede *apuntar* a directorios, incluso creando ciclos, o a archivos en otros volúmenes.

Cuando se crea una liga simbólica, la liga y el archivo son dos entidades distintas. Si bien cualquier proceso que abra al archivo destino estará trabajando con la misma entidad, en caso de que éste sea renombrado o eliminado, la liga quedará *rota* (esto es, apuntará a una ubicación inexistente).

Si bien estos dos tipos de liga son válidos también en los sistemas Windows,[6] en dichos sistemas sigue siendo más común emplear los *accesos directos*. Se denomina así a un archivo (identificado por su extensión, `.lnk`), principalmente creado para poder *apuntar* a los archivos desde el escritorio y los menúes; si un proceso solicita al sistema abrir el *acceso directo*, no obtendrá al archivo destino, sino al acceso directo mismo.

Si bien el API de Win32 ofrece las funciones necesarias para emplear las ligas, tanto duras como simbólicas, éstas no están reflejadas desde la interfaz usuario del sistema — y son sistemas donde el usuario promedio no emplea una interfaz programador, sino que una interfaz gráfica. Las ligas, pues, no son más empleadas por *cuestión cultural*: en sus comunidades de usuarios, nunca fueron frecuentes, por lo cual se mantienen como conceptos empleados sólo por los *usuarios avanzados*.

Ya con el conocimiento de las ligas, y reelaborando la figura 39.3 con mayor apego a la realidad: en los sistemas operativos (tanto Unix como Windows), todo directorio tiene dos entradas especiales: los directorios `.` y `..`, que aparecen tan pronto como el directorio es creado, y resultan fundamentales para mantener la *navegabilidad* del árbol.

Como se puede ver en la figura 39.4, en todos los directorios, `.` es una liga dura al mismo directorio, y `..` es una liga al directorio *padre* (de nivel jerárquico inmediatamente superior). Claro está, como sólo puede haber una liga `..`, un directorio enlazado desde dos lugares distintos sólo apunta hacia uno de ellos con su enlace `..`; en este caso, el directorio común `proyecto` está dentro del directorio `/home/usr2`. La figura representa la *liga simbólica* desde `/home/usr1` como una línea punteada.

Hay una excepción a esta regla: el directorio raíz. En este caso, tanto `.` como `..` apuntan al mismo directorio.

Esta es la razón por la cual no se puede tomar rigurosamente un árbol de archivos como un *grafo dirigido acíclico*, ni en Windows ni en Unix: tanto las entradas `.` (al apuntar al mismo directorio donde están contenidas) como las entradas `..` (al apuntar al directorio padre) crean ciclos.

[4] El significado y la estructura de un i-nodo se abordan en el capítulo VII.

[5] Formalmente, puede haberlas, pero sólo el administrador puede crearlas; en la sección 39.2.1 se cubre la razón de esta restricción al hablar de recorrer los directorios.

[6] Únicamente en aquellos que emplean el sistema de archivos que introdujo Windows NT, el NTFS, no en los que utilizan alguna de las variantes de FAT.

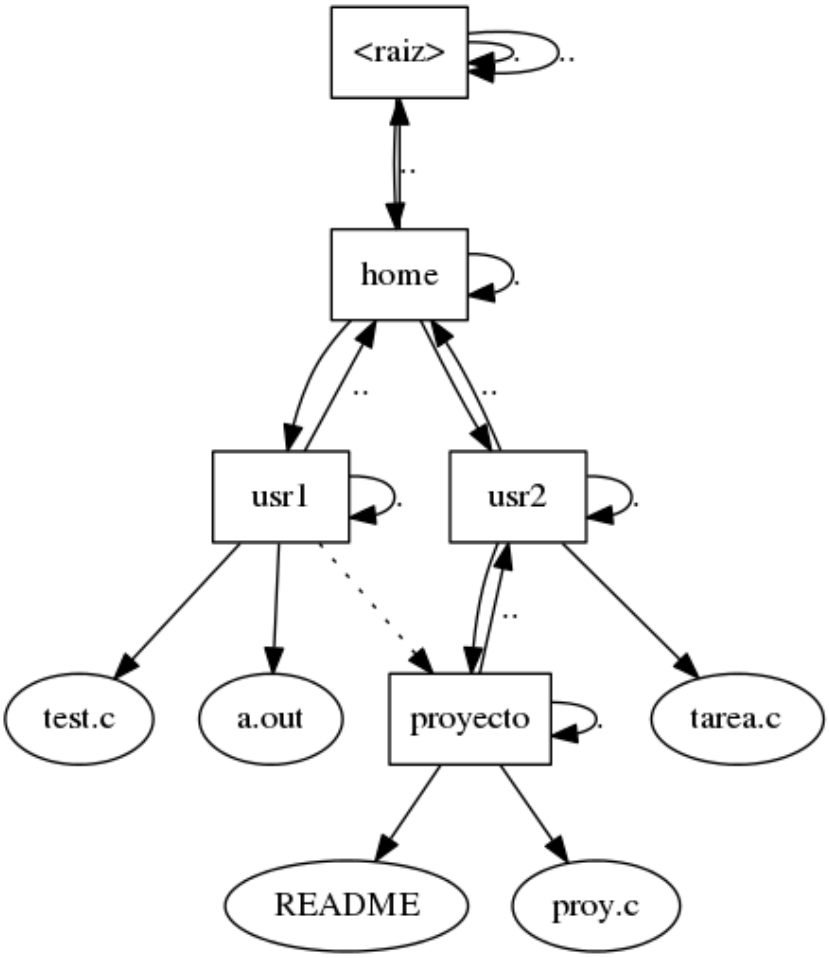


Figura 39.4: Directorio como un *grafo dirigido*, mostrando los *enlaces ocultos* al directorio actual `.` y al directorio *padre* `..`

39.2. Operaciones con directorios

Al igual que los archivos, los directorios tienen una semántica básica de acceso. Éstos resultan también tipos de datos abstractos con algunas operaciones definidas. Muchas de las operaciones que pueden realizarse con los directorios son análogas a las empleadas para los archivos.[7] Las operaciones básicas a presentar son:

Abrir y cerrar Como en el caso de los archivos un programa que requiera trabajar con un directorio deberá *abrirlo* para hacerlo, y *cerrarlo* cuando ya no lo requiera. Para esto, en C, se emplean las funciones `opendir()` y `closedir()`. Estas funciones trabajan asociadas a un *flujo de directorio* (*directory stream*), que funciona de forma análoga a un descriptor de archivo.

Listado de archivos Para mostrar los archivos que conforman a un directorio, una vez que se abrió el directorio se *abre*, el programa *lee* (con `readdir()`) cada una de sus entradas. Cada uno de los resultados es una estructura `dirent` (*directory entry*, esto es, *entrada de directorio*), que contiene su nombre en `d_name`, un apuntador a su *i-nodo* en `d_ino`, y algunos datos adicionales del archivo en cuestión.

Para presentar al usuario la lista de archivos que conforman un directorio, podría hacerse:

[7] De hecho, en muchos sistemas de archivos los directorios son meramente archivos de tipo especial, que son presentados al usuario de forma distinta. En la sección 43.4 se presenta un ejemplo.

```
#include <stdio.h>
#include <dirent.h>
#include <sys/types.h>

int main(int argc, char *argv[]) {
  struct dirent *archivo;
  DIR *dir;
  if (argc != 2) {
    printf("Indique el directorio a mostrar\n");
    return 1;
  }
  dir = opendir(argv[1]);
  while ((archivo = readdir(dir)) != 0) {
    printf("%s\t", archivo->d_name);
  }
  printf("\n");
  closedir(dir);
}
```

Al igual que en al hablar de archivos se puede *reposicionar* (`seek()`) al descriptor de archivo, para *rebobinar* el descriptor del directorio al principio del listado se emplea `rewinddir()`.

Buscar un elemento Si bien en el transcurso del uso del sistema operativo resulta una operación frecuente que el usuario solicite el listado de archivos dentro de un directorio, es mucho más frecuente buscar a un archivo en particular. La llamada `fopen()` antes descrita efectúa una búsqueda similar a la presentada en el ejemplo de código anterior, claro está, deteniéndose cuando encuentra al archivo en cuestión.

Crear, eliminar o renombrar un elemento Si bien estas operaciones se llevan a cabo sobre el directorio, son invocadas por medio de la semántica orientada a archivos: un archivo es creado con `fopen()`, eliminado con `remove()`, y renombrado con `rename()`.

39.2.1. Recorriendo los directorios

Es frecuente tener que aplicar una operación a todos los archivos dentro de cierto directorio, por ejemplo, para agrupar un directorio completo en un archivo comprimido, o para copiar todos sus contenidos a otro medio; procesar todas las entradas de un directorio, incluyendo las de sus subdirectorios, se denomina *recorrer el directorio* (en inglés, *directory traversal*).

Si se trabaja sobre un sistema de archivos plano, la operación de recorrido completo puede realizarse con un programa tan simple como el presentado en la sección anterior.

Al hablar de un sistema de profundidad fija, e incluso de un directorio estructurado en árbol, la lógica se complica levemente, dado que para recorrer el directorio es necesario revisar, entrada por entrada, si ésta es a su vez un directorio (y en caso de que así sea, entrar y procesar a cada uno de sus elementos). Hasta aquí, sin embargo, se puede recorrer el directorio sin mantener estructuras adicionales en memoria representando el estado.

Sin embargo, al considerar a los grafos dirigidos, se vuelve indispensable mantener en memoria la información de todos los nodos que ya han sido tocados —en caso contrario, al encontrar ciclo (incluso si éste es creado por mecanismos como las *ligas simbólicas*), se corre el peligro de entrar en un bucle infinito.

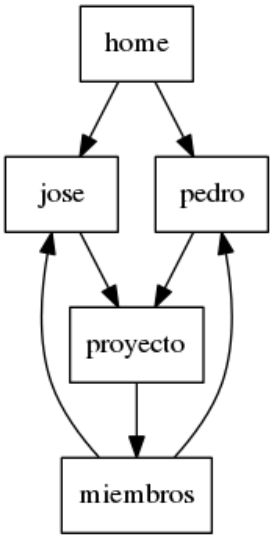


Figura 39.5: Directorio basado en grafo dirigido que incluye ciclos.

Para recorrer al directorio ilustrado como ejemplo en la figura 39.5, no bastaría tomar nota de las rutas de los archivos conforme son recorridas —cada vez que los sean procesados, su ruta será distinta. Al intentar respaldar el directorio `/home/jose/proyecto`, por ejemplo, el recorrido resultante podría ser:

- `/home/jose/proyecto`
- `/home/jose/proyecto/miembros`
- `/home/jose/proyecto/miembros/jose`
- `/home/jose/proyecto/miembros/jose/proyectos`
- `/home/jose/proyecto/miembros/jose/proyectos/miembros`
- ... Y un etcétera infinito.

Para resolver esta situación, los programas que recorren directorios en los sistemas de archivos *reales* deben emplear un indexado basado en *i-nodo*,[8] que identifica sin ambigüedad a cada uno de los archivos. En el caso presentado, si el i-nodo de `jose` fuera 10 543, al consultar a los miembros de `miembros` el sistema encontrará que su primer entrada apunta al i-nodo 10 543, por lo cual la registraría sólo como un apuntador a datos *ya archivados*, y continuaría con la segunda entrada del directorio, que apunta a `pedro`.

39.2.2. Otros esquemas de organización

Por más que el uso de sistemas de archivos basados en directorios jerárquicos parece universal y es muy ampliamente aceptado, hay cada vez más casos de uso que apuntan a que se pueda estar por dar la bienvenida a una nueva metáfora de organización de archivos.

Hay distintas propuestas, y claro está, es imposible aún saber cuál dirección obtendrá el favor del mercado —o, dado que no necesariamente siga habiendo un modelo apto para todos los usos, de *qué* segmento del mercado.

[8] Que si bien no ha sido definido aún formalmente, para esta discusión bastará saber que es un número único por volumen.

39.3. *Montaje* de directorios

Para trabajar con el contenido de un sistema de archivos, el sistema operativo tiene que *montarlo*: ubicarlo en algún punto del árbol de archivos visible al sistema y al usuario.

Es muy común, especialmente en los entornos derivados de Unix, que un sistema operativo trabaje con distintos sistemas de archivos al mismo tiempo. Esto puede obedecer a varias causas, entre las cuales se encuentran:

Distintos medios físicos Si la computadora tiene más de una unidad de almacenamiento, el espacio dentro de cada uno de los discos se maneja como un sistema de archivos indepentiente. Esto es especialmente cierto en la presencia de unidades removibles (CD, unidades USB, discos duros externos, etc.)

Diferentes usos esperados Como se verá más adelante, distintos *esquemas de organización* (esto es, distintos sistemas de archivos) presentan ventajas para diversos patrones de uso. Por ejemplo, tiene sentido que una base de datos resida sobre una organización distinta a la de los programas ejecutables (binarios) del sistema.

Abstracciones de sistemas no-físicos El sistema operativo puede presentar diversas estructuras *con una estructura* de sistema de archivos. El ejemplo más claro de esto es el sistema de archivos virtual `/proc`, presente en los sistemas Unix, que permite ver diversos aspectos de los procesos en ejecución (y, en Linux, del sistema en general). Los archivos bajo `/proc` no están en ningún disco, pero se presentan como si fueran archivos estándar.

Razones administrativas El administrador del sistema puede emplear sistemas de archivos distintos para aislar espacios de usuarios entre sí: por ejemplo, para evitar que un exceso de mensajes enviados en la bitácora (típicamente bajo `/var/log`) saturen al sistema de archivos principal, o para determinar patrones de uso máximo por grupos de usuarios.

En los sistemas tipo Unix, el mecanismo para montar los archivos es el de un árbol con *puntos de montaje*. Esto es, *todos los archivos y directorios* del sistema operativo están estructurados en *un único árbol*. Cuando se solicita al sistema operativo *montar* un sistema de archivos en determinado lugar, éste se integra al árbol, ocultando todo lo que el directorio en cuestión previamente tuviera.[9]

La manera en que esto se presenta en sistemas Windows es muy distinta. Ahí, cada uno de los volumenes *detectados* recibe un *identificador de volumen*, y es montado automáticamente en un sistema de directorio estructurado como árbol de un sólo nivel representando a los dispositivos del sistema.[10] Este árbol es presentado a través de la interfaz gráfica (aunque este nombre no significa nada para el API del sistema) como *Mi Computadora*.

Para especificar la *ruta completa* a determinado archivo, se inicia con el identificador del volumen. De este modo, la especificación absoluta de un archivo es una cadena como `VOL:\Dir1\Dir2\Archivo.ext` — el caracter `:` separa al volumen del árbol del sistema de archivos, y el caracter `\` separa uno de otro a los directorios. Por convención, si no se especifica la unidad, el sistema asumirá que se está haciendo referencia a la *unidad actual* (a grandes rasgos, la última unidad en ser utilizada).

[9] Hay implementaciones que exigen que el montaje se realice exclusivamente en directorios vacíos; se tienen otras, como UnionFS, que buscan seguir presentando una interfaz *de lectura* a los objetos que existían en el directorio previo al montaje, pero realizan las escrituras únicamente en el sistema ya montado; estas complican fuertemente algunos aspectos semánticos, por lo cual resultan poco comunes.

[10] En realidad, este árbol no sólo incluye a los volúmenes de almacenamiento, sino que a los demás dispositivos del sistema, como los distintos puertos, pero los *oculta* de la interfaz gráfica.

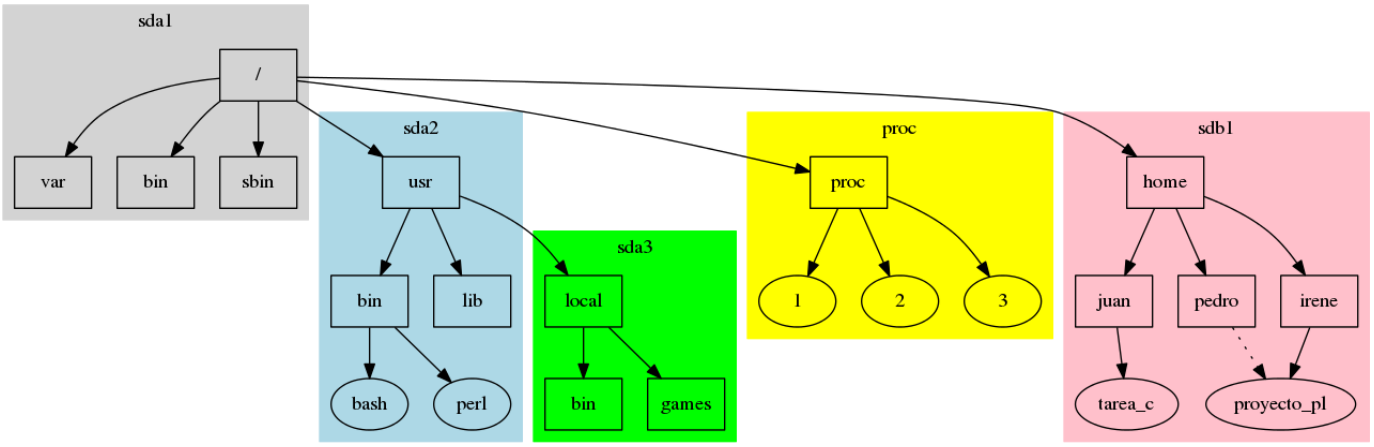


Figura 39.6: Árbol formado del montaje de `sda1` en la raíz, `sda2` como `/usr`, `sdb1` como `/home`, y el directorio virtual `proc`.

Los identificadores de volumen están preasignados, muchos de ellos según un esquema heredado desde la época de las primeras PC: los volúmenes `A` y `B` están reservados para las unidades de disco flexible; `C` se refiere al disco duro de arranque, y las unidades posteriores que va detectando el sistema son `D`, `E`, `F`, etcétera.

Es posible modificar esta nomenclatura y configurar a los discos para estar en otra ubicación, pero muchas aplicaciones dependen ya de este comportamiento y configuración específica.

Capítulo 40

Control de acceso

Parte importante de la información que se almacena respecto a cada uno de los archivos, directorios u otros objetos son sus permisos de acceso. Éstos indican al sistema qué puede hacerse y qué no con cada uno de los objetos, posiblemente diferenciando el acceso dependiendo del usuario o clase de usuarios de que se trate.

Hay muchos esquemas de control de acceso; a continuación se presentan tres esquemas, comenzando por el más simple, y avanzando hacia los más complejos.

40.1. Sistemas FAT

El sistema de archivos FAT[1] fue diseñado a fines de los setenta, y al día de hoy es muy probablemente el sistema de archivos más ampliamente utilizado del mundo. Su diseño es suficientemente simple para ser incluido en dispositivos muy limitados; esta simplicidad se demostrará al analizar sus principales estructuras

[1] Recibe su nombre de las siglas de su estructura fundamental, la *Tabla de Asignación de Archivos*, o *File Allocation Table*.

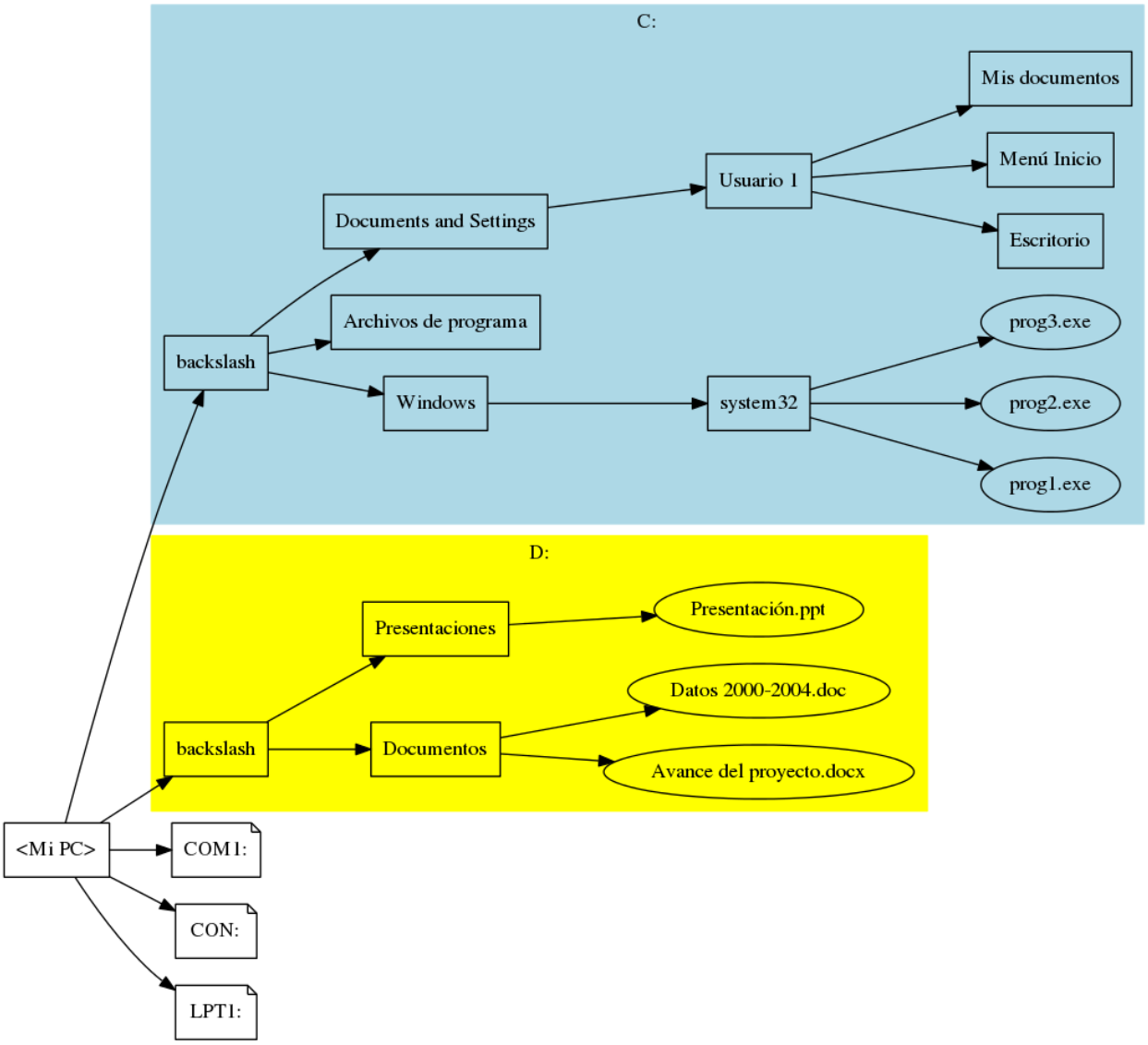


Figura 39.7: Jerarquía de dispositivos y volúmenes de archivos en un sistema Windows.

son analizadas en el capítulo VII. Y por otro lado, con sólo modificaciones relativamente menores a su diseño original, ha logrado extenderse de un sistema pensado para volúmenes de 150 KB hasta llegar a las decenas de gigabytes.

En cada una de las entradas del directorio en un sistema FAT, el byte número 12 almacena los siguientes *atributos*:

Oculto Si está encendido, el archivo no se deberá mostrar al usuario en listados de directorio.

Sólo lectura El sistema operativo debe evitar toda modificación al archivo, esto es, rechazar las llamadas al sistema que soliciten esta modificación.

Sistema Como se explicará en la sección 43.4.2, un sistema de archivos FAT tiende conforme se va utilizando a *fragmentar* los archivos que almacena. La carga del sistema operativo tiene que realizarse empleando código lo más compacto y sencillo posible: sin siquiera tener conocimiento del sistema de archivos. Este atributo indica que el sistema operativo debe cuidar no mover ni fragmentar al archivo en cuestión.

Archivado MS-DOS incluía herramientas bastante sencillas para realizar respaldos; estas basaban su operación en este atributo: Conforme se realizaba un respaldo, todos los archivos iban siendo marcados como *archivados*. Y cuando un archivo era modificado, el sistema operativo retiraba este atributo. De esta manera, el siguiente respaldo contendría únicamente los archivos que habían sido modificados desde el respaldo anterior.

Estos atributos eran suficientes para las necesidades de un sistema de uso personal de hace más de tres décadas. Sin embargo, y dado que está reservado todo un byte para los atributos, algunos sistemas extendieron los atributos cubriendo distintas necesidades. Estos cuatro atributos son, sin embargo, la base del sistema.

Puede verse que, bajo MS-DOS, no se hace diferencia entre *lectura* y *ejecución*. Como se presentó en la sección 38.4, el sistema MS-DOS basa la identificación de archivos en su *extensión*. Y, claro, siendo un sistema concebido como *monousuario*, no tiene sentido condicionar la *lectura* de un archivo: el sistema operativo siempre permitirá la lectura de cualquier archivo.

40.2. Modelo tradicional Unix

El sistema Unix precede a MS-DOS por una década. Sin embargo, su concepción es de origen para un esquema multiusuario, con las provisiones necesarias para que cada uno de ellos indique qué usuarios pueden o no tener diferentes tipos de acceso a los archivos.

En Unix, cada uno de los usuarios pertenece a uno o más *grupos*. Estos grupos son gestionados por el administrador del sistema.

Cada objeto en el sistema de archivos describe sus permisos de acceso por nueve bits, y con el identificador de su usuario y grupo propietarios.

Estos bits están dispuestos en tres grupos de tres; uno para el *usuario*, uno para el *grupo*, y uno para los *otros*. Por último, los tres grupos de tres bits indican si se otorga permiso de *lectura*, *escritura* y *ejecución*. Estos permisos son los mismos que fueron presentados en la sección 32.1, relativos al manejo de memoria.

Estos tres grupos de tres permiten suficiente *granularidad* en el control de acceso para expresar casi cualquier combinación requerida.

Por poner un ejemplo, si los usuarios `jose`, `pedro` y `teresa` están colaborando en el desarrollo de un sistema de facturación, pueden solicitar al administrador del sistema la creación de un grupo, `factura`. Una vez creado el sistema, y suponiendo que el desarrollo se efectúa en el directorio `/usr/local/factura/`, podrían tener los siguientes archivos:

```
$ ls -la /usr/local/factura/
drwxrwxr-x root   factura   4096 .
drwxr-xr-x root   root      4096 ..
-rw-r----- root   factura  12055 compilacion.log
-rwxrwxr-x pedro  factura 114500 factura
-rw-rw-r-- teresa factura  23505 factura.c
-rw-rw-r-- jose   factura   1855 factura.h
-rw-rw---- teresa factura  36504 factura.o
-rw-rw---- teresa factura  40420 lineamientos.txt
-rw-rw-r-- pedro  factura   3505 Makefile
```

Este ejemplo muestra que cada archivo puede pertenecer a otro de los miembros del grupo.[2] Para el ejemplo aquí mostrado, el usuario propietario es menos importante que el grupo al que pertenece.

[2] Los archivos pertenecen al usuario que los creó, a menos que su propiedad sea *cedida* por el administrador a otro usuario.

La ejecución sólo está permitida, para los miembros del grupo `factura`, en el archivo `factura`: los demás archivos son ya sea el código fuente que se emplea para *generar* dicho archivo ejecutable, o la información de desarrollo y compilación.

Respecto al acceso del resto de los usuarios (esto es, aquellos no pertenecientes al grupo `factura`), todos tienen permiso de lectura sobre todos los archivos a excepción del archivo intermedio de compilación, `factura.o`, el documento de diseño, `lineamientos.txt`, y la bitácora de compilación, `compilacion.log`

Este ejemplo muestra los directorios `.` y `..`, que fueron presentados en la sección 39.1.5. Para los directorios, se emplea el mismo sistema de permisos; el permiso de escritura indica quién puede crear o eliminar archivos dentro de él, el de lectura indica qué usuarios pueden averiguar la lista de archivos contenidos en dicho directorio, y el de ejecución indica qué usuarios pueden *entrar* al directorio. En este caso, todos los usuarios pueden listar (leer) y ejecutar (entrar) al directorio, pero únicamente el administrador `root` y los miembros del grupo `factura` pueden modificar los archivos que contiene.

Claro está, el directorio `..` (que apunta de vuelta al directorio padre, `/usr/local/`) es del sistema, y permite sólo la lectura y ejecución a todos los usuarios (incluyendo al grupo `factura`).

En este punto, cabe recordar que, en un sistema Unix, los dispositivos del sistema así como varios objetos virtuales (como las *tuberías* de comunicación entre procesos) son representados en el sistema de archivos. Esto significa que, por medio de estos permisos, el administrador del sistema puede indicar qué usuarios pueden tener permisos de lectura o modificación sobre los diferentes componentes hardware del sistema.

El esquema de permisos de Unix presenta algunas características adicionales, como los bits SUID/SGID o los *atributos extendidos* que presentan ciertas implementaciones; se sugiere al lector interesado referirse a las *páginas de manual* de Linux para los comandos `chmod` y `chattr`, respectivamente.

40.3. Listas de control de acceso

Una de las desventajas del modelo tradicional de usuarios Unix es que requiere de la intervención del administrador del sistema para cada nuevo proyecto: el administrador del sistema tuvo que crear al grupo `factura` antes de que pudiera realizarse ningún trabajo de forma colaborativa. Además, encontrar la representación justa para indicar un conjunto específico de permisos puede ser complicado. Dado que un archivo no puede pertenecer a más de un grupo, puede hacerse necesario crear conjuntos de grupos adicionales que intersectan aspectos específicos: una salida muy poco elegante.

Varios sistemas operativos, incluyendo particularmente a las versiones de Windows derivadas del NT, han agregado *listas de control de acceso* (o ACL, del inglés, *Access Control Lists*): cada uno de los archivos tiene asociada una lista donde se indican los permisos particulares que tendrá sobre éste cada usuario o grupo particular. La figura 40.1 muestra un ejemplo de la definición del control de acceso a un archivo en el sistema operativo *Windows 7*.

Si bien el modelo de listas de control de acceso brinda un control mucho más fino sobre cada uno de los archivos, no está exento de desventajas: en primer término, todos los permisos particulares que tenga un archivo tienen que almacenarse junto con su *i-nodo*. Dado que su longitud es variable, se representan como una estructura adicional adjunta a los datos del archivo, y esto se traduce en un tiempo de acceso ligeramente mayor.

En segundo término, resulta más difícil presentar un listado compacto y completo de los permisos para todos los archivos en determinado directorio. Realizar una evaluación *al vuelo* de los controles de acceso se vuelve más complejo para el administrador.

Por último, este esquema da lugar a ambigüedades, cuya política de resolución debe establecerse *a priori*. Por ejemplo, si un usuario pertenece a dos grupos distintos, uno de ellos con *aprobación explícita* para la escritura a cierto archivo, y otro con *denegación explícita* para la escritura al mismo, debe haber una política

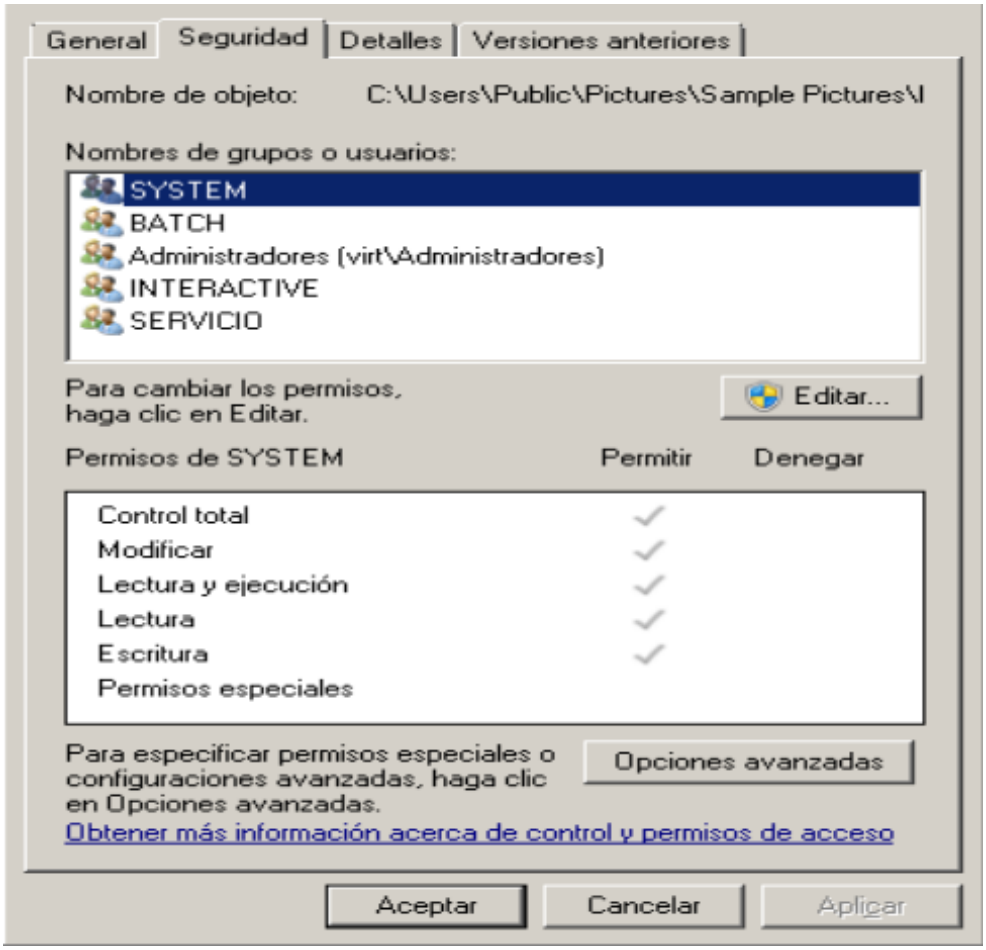


Figura 40.1: Lista de control de acceso a un archivo en Windows 7: en esta vista, muestra que el grupo `SYSTEM` tiene permiso para todo tipo de acceso, con excepción del definido por *Permisos especiales*.

global para resolver si el permiso será otorgado o rechazado. Estas políticas no siempre resultan fáciles de comprender por los usuarios del sistema.

Capítulo 41

Sistemas de archivos remotos

Uno de los principales y primeros usos que se dio a la comunicación en red fue el de compartir archivos entre computadoras independientes. En un principio, esto se realizaba de forma *explícita*, con transferencias manuales mediante de programas dedicados a ello, como sería hoy en día el FTP.

Por otro lado, desde mediados de los ochenta, es posible realizar estas transferencias de forma *implícita* y

automática, empleando *sistemas de archivos sobre la red* (o lo que es lo mismo, *sistemas de archivos remotos*). Éstos se presentan como caso particular de la *abstracción de sistemas no-físicos* que fueron mencionados en la sección anterior: si bien el sistema operativo no tiene acceso *directo* a los archivos y directorios que le solicitará el usuario, por medio de los módulos de red, sabe cómo obtenerlos y presentarlos *como si fueran locales*.

Al hablar de sistemas de archivos en red, casi siempre se hará siguiendo un *modelo cliente-servidor*. Estos términos no se refieren a las prestaciones relativas de una computadora, sino al papel que ésta desempeña *dentro de cada conexión*, esto es, se designa como *cliente* a la computadora que solicita un servicio, y como *servidor* a la que lo provee; es frecuente que dos computadoras sean tanto servidor como cliente la una de la otra en distintos servicios.

41.1. Network File System (NFS)

El *Sistema de Archivos en Red* (*Network File System*, mejor conocido como NFS) fue creado por Sun Microsystems, y desde 1984 forma parte de su sistema operativo —resultó una implementación tan exitosa que a los pocos años formaba parte de todos los sistemas tipo Unix.

Está construido sobre el mecanismo RPC (*remote procedure call*, *llamada a procedimientos remotos*), un mecanismo de mensajes y manejo básico de sesión que actúa como una capa superior a TCP/IP, incluyendo facilidades de *descubrimiento de recursos* y *abstracción*. RPC puede ser comparado con protocolos como DCE/RPC de OSF, DCOM de Microsoft, y hoy en día, SOAP y XML-RPC. Estos mecanismos permiten al programador delegar en un *servicio* el manejo de las conexiones de red, particularmente (en el caso particular aquí descrito) la persistencia de sesiones en caso de desconexión, y limitar su atención a una *conexión virtual establecida*.

La motivación de origen para la creación de NFS fue presentar una solución que aprovechara el hardware ya común en dicha época, y que centralizara la administración: ofrecer las facilidades para contar con redes donde hubiera un *servidor de archivos*, y donde las estaciones de trabajo tuvieran únicamente una instalación básica,[1] y el entorno de usuario completo estuviera disponible en cualquiera de las estaciones.

El sistema de archivos que ofrece sobre la red NFS cumple con la semántica Unix completa — para emplear un sistema remoto, basta montarlo[2] y usarlo como si fuera local. El manejo de permisos, usuarios, e incluso las ligas duras y simbólicas se manejan exactamente como se haría localmente.

El NFS viaja sobre un protocolo muy ligero; no implementa cifrado ni verificaciones adicionales. Al día de hoy, es uno de los mejores mecanismos para el envío de grandes cantidades de información — pero siempre en redes que sean *completamente confiables*.

Ahora, NFS se presenta como uno de los componentes de una solución completa. Dado que se espera que la información de usuarios y permisos sea *consistente* en todos los clientes; Sun ofrecía también un esquema llamado *Yellow Pages* (posteriormente renombrado a NIS, *Network Information System*) para compartir la información de autenticación y listas de usuarios.

La desventaja, en entornos sin NIS, es que los permisos se manejan según el identificador numérico del usuario. Si en diferentes sistemas el mismo usuario tiene distintos identificadores, los permisos no coincidirán. Es más, dado que el control de acceso principal es únicamente por dirección IP, para tener acceso irrestricto a los archivos de otros usuarios en NFS basta con tener control pleno de una computadora cualquiera en la red para poder *asumir o usurpar la identidad* de cualquier otro usuario.

Por último, para garantizar que las escrituras a archivos se llevaran a cabo cuando eran solicitadas (en contraposición a asumir éxito y continuar), todas las escrituras en un principio sobre NFS eran manejadas de forma *síncrona*, esto es, tras grabar un archivo, el cliente no continuaba con la ejecución hasta no tener

[1] Incluso manejando estaciones de trabajo *diskless*, esto es, computadoras sin disco duro, cuyo sistema de arranque tiene la capacidad de solicitar al servidor le envíe incluso el núcleo del sistema operativo que ejecutará.

[2] Para montar un sistema remoto, se emplea un comando como `mount archivos.unam.mx:/ext/home /home`, con lo cual el directorio `/ext/home` ubicado en el servidor `archivos.unam.mx` aparecerá montado como directorio `/home` local.

confirmación por parte del servidor de que los datos estaban ya guardados en disco. Esto, si bien asegura que el usuario recibirá retroalimentación confiable respecto a la operación que realizó, ocasiona demoras que muchas veces son percibidas como excesivas.

Versiones posteriores del protocolo mejoraron sobre los puntos débiles aquí mencionados. Al día de hoy, casi 30 años después de su presentación, NFS es aún un sistema de archivos en red muy ampliamente empleado.

41.2. Common Internet File System (CIFS)

El equivalente a NFS en los entornos donde predominan los sistemas Windows es el protocolo CIFS (*Common Internet File System*, Sistema de Archivos Común para Internet). Aparece en los sistemas primarios de Microsoft alrededor de 1990,[3] originalmente bajo el nombre SMB (*server message block*, *bloque de mensaje del servidor*).

Las primeras implementaciones estaban basadas en el protocolo NBF, frecuentemente conocido como NetBEUI, un protocolo no ruteable diseñado para redes pequeñas y entornos sencillos de oficina. A partir de Windows 2000 se ha reimplementado completamente para operar sobre TCP/IP. Es a partir de este momento que se le comienza a denominar CIFS, aunque el nombre SMB sigue siendo ampliamente utilizado.[4]

El sistema CIFS se ajusta mucho más a la semántica de los sistemas frecuentemente encontrados en las PC, MS-DOS y Windows, aunque dado el lapso transcurrido desde su introducción y el volumen de cambios en su mercado objetivo, ha pasado por varios cambios fundamentales, que al día de hoy complican su uso.

Para tener acceso a un volumen compartido por SMB se introdujo el comando `NET`;[5] basta indicar desde la línea de comando a DOS o Windows `NET USE W: \\servidor\directorio` para que el recurso compartido bajo el nombre `directorio` dentro del equipo conocido como `servidor` aparezca en el árbol *Mi PC*, y el usuario pueda emplear sus contenidos como si fuera un sistema de archivos local, con un volumen asignado de `W:`.

Cuando LAN Manager fue introducido al mercado, los sistemas Microsoft no manejaban aún el concepto de usuarios, por lo que la única medida de seguridad que implementaba SMB era el manejo de hasta dos contraseñas por directorio compartido: con una, el usuario obtenía acceso de sólo lectura, y con la otra, de lectura y escritura. Tras la aparición de Windows NT, se agregó un esquema de identificación por usuario/contraseña, que posibilita el otorgamiento de permisos con una *granularidad* mucho menor.[6]

El protocolo empleado por SMB fue pensado originalmente para una red *pequeña*, con hasta un par de decenas de equipos. La mayor parte de los paquetes eran enviados en modo *de difusión* (*broadcast*), por lo que era fácil llegar a la saturación, y no había un esquema centralizado de resolución de nombres, por lo que era frecuente *no encontrar* a determinado equipo.

Los cambios que CIFS presenta a lo largo de los años son muy profundos. Las primeras implementaciones presentan fuertes problemas de confiabilidad, rendimiento y seguridad, además de estar planteadas para su uso en un sólo tipo de sistema operativo; al día de hoy, todos estos puntos han mejorado fuertemente. En sistemas Unix, la principal implementación, *Samba*, fue creada haciendo ingeniería inversa sobre el protocolo; a lo largo de los años, se ha convertido en un esquema tan robusto que es hoy por hoy tomado como implementación refrencia.

[3] El desarrollo de SMB nació como LAN Manager, originalmente para OS/2.

[4] Es debido a este nombre que la implementación de CIFS para sistemas Unix, *Samba*, fue llamado de esta manera.

[5] Este comando es empleado en MS-DOS, pero está también disponible en Windows, y al día de hoy es una de las principales herramientas para administrar usuarios.

[6] Esto significa, que puede controlarse el acceso permitido más finamente, tanto por archivo como por usuario individual.

41.3. Sistemas de archivos distribuidos: Andrew File System

Los dos ejemplos de sistema de archivos en red presentados hasta ahora comparten una visión *tradicional* del modelo cliente-servidor: al ver el comando que inicializa una conexión, e incluso a ver la información que guarda el núcleo del cliente respecto a cualquiera de los archivos, resulta claro cuál es el servidor para cada uno de ellos.

Andrew File System, o AFS, desarrolaldo en la Carnegie Mellon University[7] y publicado en 1989, plantea presentar un verdadero *sistema de archivos distribuido*, en el cual los *recursos compartidos* no tengan que estar en un servidor en particular, sino que un conjunto de equipos se repartan la carga (esto es, *agnosticismo a la ubicación*). AFS busca también una *fácil escalabilidad*, la capacidad de agregar tanto espacio de almacenamiento como equipos con papeles de servidor. AFS permite inclusive migrar completamente un volumen mientras está siendo empleado de forma transparente.

Ante la complejidad e inestabilidad adicional que presentan con tanta frecuencia las redes grandes[8] (y lo hacían mucho más hace 30 años): AFS debe operar tan confiablemente como sea posible, *incluso sin la certeza de que la red opera correctamente*.

La autenticación en AFS se basa fuertemente sobre el modelo de *tickets* y credenciales de *Kerberos*,[9] pero se aleja sensiblemente de la semántica de operación de archivos que hasta ahora se han presentado. Muchos eventos, operaciones y estados van ligados al *momento en el tiempo* en que se presentan, mediante un *modelo de consistencia débil* (*weak consistency model*). Muy a grandes rasgos, esto significa que:

- Cuando se abre un archivo, éste se copia completo al cliente. Todas las lecturas *y escrituras* (a diferencia de los esquemas tradicionales, en que éstas son enviadas al servidor *lo antes posible* y de forma síncrona) se dirigen únicamente a la copia local.

- Al cerrar el archivo, éste se copia de vuelta al *servidor de origen*, el cual se *compromete* a notificar a los clientes si un archivo abierto fue modificado (esto es, a *hacer una llamada* o *callback*). Los clientes pueden entonces intentar incorporar los cambios a su versión de trabajo, o continuar con la copia ya obtenida —es *de esperarse* que si un segundo cliente realiza alguna modificación, incorpore los cambios hechos por el primero, pero esta responsabilidad se deja a la implementación del programa en cuestión.

Esto significa en pocas palabras que los cambios a un archivo abierto por un usuario no son visibles a los demás de inmediato; sólo una vez que se cierra un archivo, los cambios hechos a éste son puestos a disposición de las sesiones abiertas actuales, y sólo son enviados como *versión actual* a las sesiones abiertas posteriormente.

Con este cambio semántico, debe quedar claro que AFS no busca ser un sistema para todo uso ni un reemplazo universal de los sistemas de archivos locales, en contraposición de los sistemas de archivos centralizados. AFS no plantea en ningún momento una operación *diskless*. Bajo el esquema aquí descrito, las lecturas y escrituras resultan baratas, porque se realizan exclusivamente sobre el caché local, pero abrir y cerrar un archivo puede ser muy caro, porque debe transferirse el archivo completo.

Hay aplicaciones que verdaderamente sufrirían si tuvieran que implementarse sobre un sistema de archivos distribuido, por ejemplo, si una base de datos se distribuyera sobre AFS, la carencia de mecanismos de bloqueo sobre *secciones* del archivo, y el requisito de operar sobre *archivos completos* harían impracticable compartir un archivo de uso intensivo y aleatorio.

[7] Como parte del *Proyecto Andrew*, denominado así por el nombre de los fundadores de esta universidad: Andrew Carnegie y Andrew Mellon.

[8] El uso típico de AFS se planteaba para organizaciones grandes, del orden de decenas de miles de estaciones.

[9] Un sistema de autenticación y autorización centralizado para entornos corporativos.

Capítulo 42

Ejercicios

42.1. Preguntas de autoevaluación

1. Identifique diferentes sistemas de cómputo (computadoras, teléfonos, dispositivos móviles, dispositivos de función específica como cámaras digitales o reproductores de medios, etc.) a los que tiene acceso. ¿Cómo identifica cada uno de ellos el tipo de archivos? ¿Implementan alguna abstracción ocultando o traduciendo la presentación de determinados archivos de forma transparente al usuario?

2. Hay diferentes modos en que un programa puede accesar a la información contenida en un archivo; siendo los principales *secuencial*, *aleatorio* y *relativo a índice*. Indique qué casos de uso resuelve mejor cada uno de estos modos de acceso.

3. ¿Qué tipo de optimización podría llevar a cabo el sistema operativo si requiriera que todo programa declarara al momento de abrir un archivo si va a utilizarlo de forma mayormente secuencial o aleatoria?

4. De las siguientes afirmaciones, identifique cuáles se refieren a *ligas duras*, a *ligas simbólicas*, y a *enlaces directos*. Hay una sóla respuesta correcta para cada afirmación.

   a) Son dos (o más) entradas en el directorio apuntando al mismo i-nodo.
   b) Es un archivo normal e independiente en el sistema de archivos, que podría ser abierto directamente por los programas.
   c) Pueden apuntar a directorios, incluso creando ciclos.
   d) Su tipo de archivo en el directorio está indicado por la extensión `.LNK`.
   e) El sistema operativo *resuelve directamente* todas las operaciones *como si fueran* al archivo referido.
   f) No pueden referirse a archivos en sistemas de archivos distintos del propio.
   g) Pueden formar representarse en un sistema de archivos tipo FAT.
   h) Si un usuario elimina cualquiera de las referencias a un archivo empleando este esquema, el archivo sigue existiendo en las demás.

5. Cuando se habla acerca de los sistemas de archivos remotos o en red, ¿qué significa la *semántica de manejo de errores*? Presente un ejemplo de cómo se refleja una distinta semántica entre un sistema de archivos local y uno en red.

6. Suponga que hay un archivo `file0` en un sistema de archivos con semántica tipo Unix. Explique qué ocurre en cada situación (con su inodo, con sus sectores de contenido, entradas de directorio, etcétera):

   - Copiado: `cp file0 file1`
   - Liga dura: `ln file0 file1`
   - Liga simbólica: `ln -s file0 file1`
   - Eliminación: `rm file0`

7. La estructura i-nodo tiene, en casi todos los sistemas de archivos, un campo de *conteo de ligas*. Explique qué es. ¿Qué operaciones llevan a que se modifique, qué ocurre si llega a cero?

42.2. Temas de investigación sugeridos

Sistemas de archivos distribuidos En este capítulo se abordaron tres sistemas de archivos *remotos*, de los cuales sólo el último puede verse como un sistema de archivos *distribuido*: bajo AFS, los datos almacenados no están en una única computadora. Pero AFS tiene ya muchos años de haber sido desarrollado, por lo cual su diseño no consideró muchos de los aspectos de las redes de datos globales actuales.

En los últimos años se han presentado varias propuestas de sistemas de archivos verdaderamente distribuidos, aptos para la implementación de *nubes* de datos y de una distribución geográfica de la información; algunos ejemplos de estos sistemas de archivos son GlusterFS (desarrollado por RedHat), S3 (de Amazon), GoogleFS (de Google), Windows Distributed File System (de Microsoft), y CEPH (también de RedHat).

¿Qué semántica ofrecen estos sistemas de archivos, cómo se comparan con la nativa de las diferentes plataformas, cómo se opera en caso de desconexión o falla, parcial o total, qué características hacen a estos sistemas *especialmente* buenos para su uso distribuido, habría alguna desventaja de emplear esos sistemas en un entorno de red local?

42.3. Lecturas relacionadas

- Sandra Loosemore y col. (1993-2014). «File System Inteface: Functions for manipulating files». En: *The GNU C Library Reference Manual*. Free Software Foundation. Cap. 14, págs. 376-421. URL: https://www.gnu.org/software/libc/manual/pdf/libc.pdf
- Marshall Kirk McKusick (2012). «Disks from the Perspective of a File System». En: *ACM Queue* 10.9. URL: https://queue.acm.org/detail.cfm?id=2367378
  - Traducción al español: César Yáñez (2013). *Los discos desde la perspectiva de un Sistema de Archivos*. URL: http://cyanezfdz.me/2013/07/04/los-discos-desde-la-perspectiva-de-un-sistema-de-archivos-es.html
- John S. Heidemann y Gerald J. Popek (1994). «File-system Development with Stackable Layers». En: *ACM Trans. Comput. Syst.* 12.1, págs. 58-89. ISSN: 0734-2071. DOI: 10.1145/174613.174616. URL: http://doi.acm.org/10.1145/174613.174616
- Thomas E. Anderson y col. (1996). «Serverless Network File Systems». En: *ACM Trans. Comput. Syst.* 14.1, págs. 41-79. ISSN: 0734-2071. DOI: 10.1145/225535.225537. URL: http://doi.acm.org/10.1145/225535.225537
- Sean A. Walberg (2006). *Finding open files with lsof*. IBM DeveloperWorks. URL: http://www.ibm.com/developerworks/aix/library/au-lsof.html

Parte VII

Sistemas de archivos

Capítulo 43

Plasmando la estructura en el dispositivo

A lo largo del capítulo VI se abordaron los elementos del sistema de archivos tal como son presentados al usuario final, sin entrar en detalles respecto a cómo organiza toda esta información el sistema operativo en un *dispositivo persistente* —mencionamos algunas estructuras base, pero dejándolas explícitamente pendientes de definición. En este capítulo se tratarán las principales estructuras y mecanismos empleados para que un sistema de archivos sea ya no sólamente una estructura formal ideal, sino que una entidad almacenada en un dispositivo.

A lo largo de la historia del cómputo, el almacenamiento no siempre se realizó en discos (dispositivos giratorios de acceso aleatorio). Por varias décadas, los medios principales almacenamiento eran de acceso estrictamente secuencial (tarjetas perforadas, cintas de papel, cintas magnéticas); por más de 30 años, el medio primario de almacenamiento han sido los distintos tipos de discos magnéticos, y desde hace algunos años, estamos viendo una migración a *almacenamiento de estado sólido*, a dispositivos sin partes móviles que guardan la información en un tipo particular de memoria (tema que se abordará en la sección K.2). Volviendo a las categorías presentadas en la sección 16, los medios de acceso secuencial son *dispositivos de caracteres*, y tanto discos como unidades de estado sólido son *dispositivos de bloques*.

43.1. Conceptos para la organización

Los sistemas de archivo están en general desarrollados pensando en *discos*, y a lo largo de este capítulo, se hará referencia como *el disco* al medio de almacenamiento persistente en el cual esté plasmado el sistema de archivos. En el apéndice X se tocarán algunos de los aspectos que debemos considerar al hablar de sistemas de archivos respaldados en medios *distintos* a un disco.

Mientras tanto, conviene mantener una visión aún bastante idealizada y abstracta: un *disco* visto desde la perspectiva del sistema operativo será presentado a lo largo del presente capítulo[1] como un arreglo muy

[1] Para un punto de vista más riguroso de cómo se relaciona el sistema operativo con los discos y demás mecanismos de almacenamiento, refiérase al apéndice X.

grande de *bloques* de tamaño fijo, cada uno de ellos *directamente direccionable*; esto significa que el sistema operativo puede referirse por igual a cualquiera de los bloques del disco mediante una dirección física e inambigua dentro del disco entero. Partiendo de esto, se emplean los siguientes conceptos para almacenar, ubicar o recuperar la información:

Partición Una subdivisión de un disco, por medio de la cual el administrador o usuario del sistema puede definir la forma en que se emplea el espacio del disco, segmentándolo según haga falta.

Un disco puede tener varias particiones, y cada una de ellas puede tener un sistema de archivos independiente.

Volumen Colección de bloques *inicializados* con un sistema de archivos que pueden presentarse al usuario como una unidad. Típicamente un volumen coincide con una partición (pero no siempre es el caso, como se describirá en las secciones L y M).

El volumen se *describe* ante el sistema operativo en el *bloque de control de volumen*, también conocido como *superbloque* en Unix, o *tabla maestra de archivos* (*master file table*) en NTFS.

Sistema de archivos Esquema de organización que sigue un determinado *volumen*. Dependiendo del sistema de archivos elegido, cada uno de los componentes aquí presentados ocuparán un distinto lugar en el disco, presentando una semántica propia.

Para poder tener acceso a la información almacenada en determinado volumen, el sistema operativo debe tener soporte para el sistema de archivos particular en que éste esté estructurado.

Directorio raíz La estructura que relaciona cada nombre de archivo con su números de *i-nodo*. Típicamente sólo almacena los archivos que están en el *primer nivel jerárquico* del sistema, y los directorios derivados son únicamente referenciados desde éste.

En sistemas de archivos modernos, el directorio normalmente incluye sólo el nombre de cada uno de los archivos y el número de *i-nodo* que lo describe, todos los *metadatos* adicionales están en los respectivos *i-nodos*.

Metadatos Recibe este nombre toda la información *acerca de* un archivo que *no es* el contenido del archivo mismo. Por ejemplo, el nombre, tamaño o tipo del archivo, su propietario, el control de acceso, sus fechas de creación, último acceso y modificación, ubicación en disco, etcétera.

I-nodo Del inglés *i-node*, *information node* (nodo de información); en los sistemas tipo Windows, normalmente se le denomina *bloque de control de archivo* (FCB). Es la estructura en disco que guarda los *metadatos* de cada uno de los archivos, proporcionando un vínculo entre la *entrada en el directorio* y los datos que lo conforman.

La información almacenada incluye todos los metadatos relacionados con el archivo *a excepción del nombre* (mismo que radica únicamente en el *directorio*): los permisos y propietarios del archivo, sus fechas de creación, última modificación y último acceso, y la *relación de bloques* que ocupa en el disco. Más adelante se abordarán algunos de los esquemas más comunes para presentar esta relación de bloques.

Esta separación entre directorio e *i-nodo* permite a un mismo archivo formar parte de distintos directorios, como se explicó en la sección 39.1.5.

***Mapa de bits* de espacio libre** La función del bitmap es poder gestionar el espacio libre del disco. Recuérdese que el disco se presenta asignado por *bloques*, típicamente de 4 096 bytes — en el bitmap cada bloque se representa con un bit, con lo que aquí se puede encontrar de forma compacta el espacio ocupado y disponible, así como el lugar adecuado para crear un nuevo archivo.

El bitmap para un disco de 100 GB, de esta manera, se puede representar en 23 MB ($\frac{100\times10^9}{4\ 096}$), cantidad que puede razonablemente mantener en memoria un sistema de escritorio promedio hoy en día.[2]

Más adelante se verán algunas estructuras avanzadas que permiten mayor eficiencia en este sentido.

43.2. Diferentes sistemas de archivos

Un sistema operativo puede dar soporte a varios distintos sistemas de archivos; un administrador de sistemas puede tener muy diferentes razones que influyan para elegir cuál sistema de archivos empleará para su información; algunas razones para elegir a uno u otro son que el rendimiento de cada uno puede estar *afinado* para diferentes patrones de carga, necesidad de emplear un dispositivo portátil para intercambiar datos con distintos sistemas, e incluso restricciones de hardware.[3]

A lo largo de esta sección se revisará cómo los principales conceptos a abordar se han implementado en distintos sistemas de archivos; se hará referencia principalmente a una familia de sistema de archivos simple de comprender, aunque muestra claramente su edad: el sistema FAT. La razón para elegir al sistema de archivos FAT es la simplicidad de sus estructuras, que permiten comprender la organización general de la información. Donde sea pertinente, se mencionará en qué puntos principales estiba la diferencia con los principales sistemas de la actualidad.

El sistema FAT fue creado hacia finales de los setenta, y su diseño muestra claras evidencias de haber sido concebido para discos flexibles. Sin embargo, por medio de varias extensiones que se han presentado con el paso de los años (algunas con compatibilidad hacia atrás,[4] otras no), sigue siendo uno de los sistemas más empleados al día de hoy, a pesar de que ya no es recomendado como sistema primario por ningún sistema operativo de escritorio.

Si bien FAT tuvo su mayor difusión con los sistemas operativos de la familia MS-DOS, es un sistema de archivos nativo para una gran cantidad de otras plataformas (muchas de ellas dentro del mercado *embebido*), lo cual se hace obvio al revisar el soporte a atributos extendidos que maneja.

43.3. El volumen

Lo primero que requiere saber el sistema operativo para poder montar un volumen es su estructura general: en primer término, de qué *tipo* de sistema de archivos se trata, y acto seguido, la descripción básica del mismo: su extensión, el tamaño de los *bloques lógicos* que maneja, si tiene alguna *etiqueta* que describa su función ante el usuario, etc. Esta información está contenida en el *bloque de control de volumen*, también conocido como *superbloque* o *tabla maestra de archivos*.[5]

Tras leer la información del superbloque, el sistema operativo determina en primer término si puede proceder — si no sabe cómo trabajar con el sistema de archivos en cuestión, por ejemplo, no puede presentar información útil alguna al usuario (e incluso arriesgaría destruirla).

Se mencionó ya que el tamaño de bloques (históricamente, 512 bytes; el estándar *Advanced Format* en marzo del 2010 introdujo bloques de 4 096 bytes) es establecido por el hardware. Es muy común que, tanto por razones de eficiencia como para alcanzar a direccionar mayor espacio, el sistema de archivos *agrupe* a varios

[2] Esto explica porqué, incluso sin estar trabajando activamente con ningún archivo contenido en éste, el solo hecho de montar un volumen con gran cantidad de datos obliga al sistema a reservarle una cantidad de memoria.

[3] Por ejemplo, los *cargadores de arranque* en algunas plataformas requieren poder leer el volumen donde está alojada la imágen del sistema operativo —lo cual obliga a que esté en un sistema de archivos nativo a la plataforma.

[4] Se denomina *compatibilidad hacia atrás* a aquellos cambios que permiten interoperar de forma transparente con las versiones anteriores.

[5] Y aquí hay que aclarar: este bloque *no contiene a los archivos*, ni siquiera a las estructuras que apuntan hacia ellos, sólo describe al volumen para que pueda ser montado.

bloques físicos en uno lógico. En la sección 43.4 se revisará qué factores determinan el tamaño de bloques en cada sistema de archivos.

Dado que describir el volumen es la más fundamental de las operaciones a realizar, muchos sistemas de archivos mantienen *copias adicionales* del superbloque, a veces dispersas a lo largo del sistema de archivos, para poder recuperarlo en caso de que se corrompa.

En el caso de FAT, el volumen indica no sólo la *generación* del sistema de archivos que se está empleando (FAT12, FAT16 o FAT32, en los tres casos denominados así por la cantidad de bits para referenciar a cada uno de los bloques lógicos o *clusters*), sino el tamaño de los *clusters*, que puede ir desde los dos y hasta los 32 KB.

43.3.1. Volúmenes *crudos*

Si bien una de las principales tareas de un sistema operativo es la organización del espacio de almacenamiento en sistemas de archivos y su gestión para compartirse entre diversos usuarios y procesos, hay algunos casos en que un dispositivo orientado a bloques puede ser puesto a disposición de un proceso en particular para que éste lo gestione directamente. Este modo de uso se denomina *dispositivos crudos* o *dispositivos en crudo* (*raw devices*).

Pueden encontrarse dos casos de uso primarios hoy en día para dispositivos orientados a bloques no administrados mediante la abstracción de los sistemas de archivos:

Espacio de intercambio Como se vio en la sección 34.2.1, la gestión de la porción de la memoria virtual que está en disco es mucho más eficiente cuando se hace sin cruzar por la abstracción del sistema operativo — esto es, cuando se hace en un volumen en crudo. Y si bien el gestor de memoria virtual es parte innegable del sistema operativo, en un sistema *microkernel* puede estar ejecutándose como proceso de usuario.

Bases de datos Las bases de datos relacionales pueden incluir volúmenes muy grandes de datos estrictamente estructurados. Algunos gestores de bases de datos, como Oracle, MaxDB o DB2, recomiendan a sus usuarios el uso de volúmenes crudos, para optimizar los accesos a disco sin tener que cruzar por tantas capas del sistema operativo.

La mayor parte de los gestores de bases de datos desde hace varios años no manejan esta modalidad, por la complejidad adicional que supone para el administrador del sistema y por lo limitado de la ventaja en rendimiento que supone hoy en día, aunque es indudablemente un tema que se presta para discusión e investigación.

43.4. El directorio y los *i-nodos*

El directorio es la estructura que relaciona a los archivos como son presentados al usuario –identificados por una ruta y un nombre– con las estructuras que los describen ante el sistema operativo — los *i-nodos*.

A lo largo de la historia de los sistemas de archivos, se han implementado muy distintos esquemas de organización. Se presenta a continuación la estructura básica de la popular familia de sistemas de archivos FAT.

43.4.1. El directorio raíz

Una vez que el sistema de archivos está *montado* (véase 39.3), todas las referencias a archivos dentro de éste deben pasar a través del directorio. El directorio raíz está siempre en una ubicación *bien conocida* dentro

del sistema de archivos — típicamente al inicio del volumen, en los primeros sectores.[6] Un disco flexible tenía 80 *pistas* (típicamente denominadas *cilindros* al hablar de discos duros), con lo que, al ubicar al directorio en la pista 40, el tiempo promedio de movimiento de cabezas para llegar a él se reducía a la mitad. Si todas las operaciones de abrir un archivo tienen que pasar por el directorio, esto resultaba en una mejoría muy significativa.

El directorio es la estructura que determina el formato que debe seguir el nombre de cada uno de los archivos y directorios: es común que en un sistema moderno, el nombre de un archivo pueda tener hasta 256 caracteres, incluyendo espacios, caracteres internacionales, etc. Algunos sistemas de archivos son *sensibles a mayúsculas*, como es el caso de los sistemas nativos a Unix (el archivo `ejemplo.txt` es distinto de `Ejemplo.TXT`), mientras que otros no lo son, como es el caso de NTFS y VFAT (`ejemplo.txt` y `Ejemplo.TXT` son idénticos ante el sistema operativo).

Todas las versiones de FAT siguen para los nombres de archivos un esquema claramente arcaico: los nombres de archivo pueden medir hasta ocho caracteres, con una extensión opcional de tres caracteres más, dando un total de 11. El sistema no sólo no es sensible a mayúsculas y minúsculas, sino que todos los nombres deben guardarse completamente en mayúsculas, y permite sólo ciertos caracteres no alfanuméricos. Este sistema de archivos no implementa la separación entre directorio e i-nodo, que hoy es la norma, por lo que cada una de las entradas en el directorio mide exactamente 32 bytes. Como es de esperarse en un formato que ha vivido tanto tiempo y ha pasado por tantas generaciones como FAT, algunos de estos campos han cambiado sustancialmente sus significados. La figura 43.1 muestra los campos de una entrada del directorio bajo FAT32.

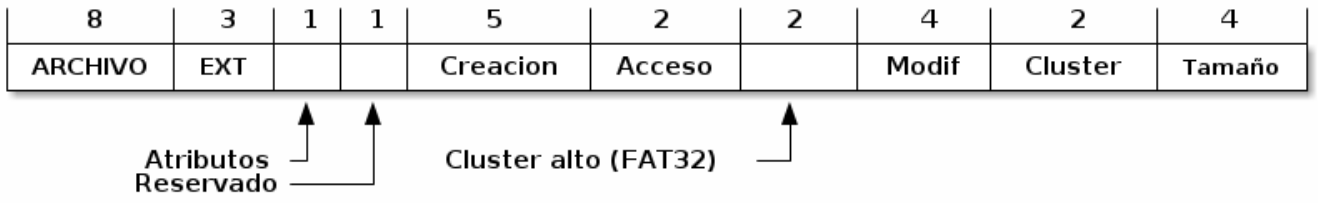


Figura 43.1: Formato de la entrada del directorio bajo FAT (Mohammed, 2007).

[6] Una excepción a esta lógica se presentó en la década de los 1980, cuando los diseñadores del sistema AmigaOS, decidieron ubicar al directorio en el sector *central* de los volúmenes, para reducir a la mitad el tiempo promedio de acceso a la parte más frecuentemente referida del disco.

La extensión VFAT fue agregada con el lanzamiento de Windows 95. Esta extensión permitía que, si bien el nombre *real* de un archivo seguiría estando limitada al formato presentado, pudieran agregarse entradas adicionales al directorio utilizando el atributo de *etiqueta de volumen* de maneras que un sistema MS-DOS debiera ignorar.[7]

Esto presenta una complicación adicional al hablar del directorio *raíz* de una unidad: si bien los directorios derivados no tienen este límite, al estar el directorio raíz ubicado en una sección fija del disco, tiene una longitud límite máxima: en un disco flexible (que hasta el día de hoy, por su baja capacidad, se formatea bajo FAT12), desde el bloque 20 y hasta el 33, esto es, 14 bloques. Con un tamaño de sector de 512 bytes, el directorio raíz mide $512 \times 14 = 7\,168$ bytes, esto es $\frac{7\,168}{32} = 224$ entradas como máximo. Y si bien esto puede no parecer muy limitado, ocupar cuatro entradas por archivo cuando, empleando VFAT, se tiene un nombre medianamente largo reduce fuertemente el panorama.

El problema no resulta tan severo como podría parecer: para FAT32, el directorio raíz ya no está ubicado en un espacio reservado, sino que como parte del espacio de datos, por lo cual es extensible en caso de requerirse.

Los primeros sistemas de archivos estaban pensados para unidades de muy baja capacidad; por mucho tiempo, las implementaciones del directorio eran simplemente listas lineales con los archivos que estaban alojados en el volumen. En muchos de estos primeros sistemas no se consideraban directorios jerárquicos, sino que presentaban un único espacio *plano* de nombres; cuando estos sistemas fueron evolucionando para soportar directorios anidados, por compatibilidad hacia atrás (y por consideraciones de rendimiento que se abordan a continuación) siguieron almacenando únicamente al directorio raíz en esta posición privilegiada, manejando a todos los directorios que derivaran de éste como si fueran archivos, repartidos por el disco.

En un sistema que implementa los directorios como listas lineales, entre más archivos haya, el tiempo que toma casi cualquier operación se incrementa linealmente (dado que potencialmente se tiene que leer el directorio completo para encontrar un archivo). Y las listas lineales presentan un segundo problema: cómo reaccionar cuando se *llena* el espacio que tienen asignado.

Como ya se presentó, FAT asigna un espacio fijo al directorio raíz, pero los subdirectorios pueden crecer abritrariamente. Un subdirectorio es básicamente una entrada con un tipo especial de archivo —si el doceavo byte de una entrada de directorio, que indica los *atributos del archivo* (véase la figura 43.1 y el cuadro 43.1) tiene al bit cuatro activado, la región de datos correspondientes a dicho archivo será interpretada como un subdirectorio.

43.4.2. La tabla de asignación de archivos

Queda claro que FAT es un sistema heredado, y que exhibe muchas prácticas que ya se han abandonado en diseños modernos de sistemas de archivos. Se vio que dentro de la entrada de directorio de cada archivo está prácticamente su *i-nodo* completo: la información de permisos, atributos, fechas de creación, y muy particularmente, el apuntador al *cluster* de inicio (bytes 26-28, más los 20-22 para FAT32). Esto resulta en una de las grandes debilidades de FAT: la tendencia a la fragmentación.

La familia FAT obtiene su nombre de la tabla de asignación de archivos (*file allocation table*), que aparece antes del directorio, en los primeros sectores del disco.[8] Cada byte de la FAT representa un *cluster* en el área de datos; cada entrada en el directorio indica, en su campo correspondiente, cuál es el primer *cluster* del archivo. Ahora bien, conforme se usa un disco, y los archivos crecen y se eliminan, y van llenando los espacios vacíos que van dejando, FAT va asignando espacio *conforme encuentra nuevos clusters libres*, sin cuidar que sea espacio continuo. Los apuntadores al *siguiente cluster* se van marcando en la tabla, *cluster* por *cluster*, y el último de cada archivo recibe el valor especial (dependiendo de la versión de FAT) `0xFFF`, `0xFFFF` o `0xFFFFFFFF`.

[7] La *etiqueta de volumen* estaba definida para ser empleada exclusivamente *a la cabeza* del directorio, dando una etiqueta global al sistema de archivos completo; el significado de una entrada de directorio con este atributo hasta antes de la incorporación de VFAT *no estaba definida*.

[8] Esta tabla es tan importante que, dependiendo de la versión de FAT, se guarda por duplicado, o incluso por triplicado.



Figura 43.2: Entradas representando archivos con (y sin) nombre largo bajo VFAT.

Ahora bien, si los directorios son sencillamente archivos que reciben un tratamiento especial, éstos son también susceptibles a la fragmentación. Dentro de un sistema Windows 95 o superior (empleando las extensiones VFAT), con directorios anidados a cuatro o cinco niveles como lo establece su jerarquía estándar,[9] la simple tarea de recorrerlos para encontrar determinado archivo puede resultar muy penalizado por la fragmentación.

43.4.3. La eliminación de entradas del directorio

Sólo unos pocos sistemas de archivos guardan sus directorios ordenados — si bien esto facilitaría las operaciones más frecuentes que se realizan sobre ellos (en particular, la búsqueda: cada vez que un directorio es recorrido hasta encontrar un archivo tiene que leerse potencialmente completo), mantenerlo ordenado ante

[9] Por ejemplo, `C:\Documents and Settings\Usuario\Menú Inicio\Programa Ejemplo\Programa Ejemplo.lnk`

Cuadro 43.1: Significado de cada uno de los bits del byte de *atributos del archivo* en el directorio FAT. La semántica que se presenta es la empleada por los sistemas MS-DOS y Windows; otros sistemas pueden presentar comportamientos adicionales.

Bit	Nombre	Descripción
0	Sólo lectura	El sistema no permitirá que sea modificado.
1	Oculto	No se muestra en listados de directorio.
2	Sistema	El archivo pertenece al sistema y no debe moverse de sus *clusters* (empleado, por ejemplo, para los componentes a cargar para iniciar al sistema operativo).
3	Etiqueta	Indica el nombre del volumen, no un archivo. En VFAT, expresa la *continuación* de un nombre largo.
4	Subdirectorio	Los *clusters* que componen a este archivo son interpretados como un subdirectorio, no como un archivo.
5	Archivado	Empleado por algunos sistemas de respaldo para indicar si un archivo fue modificado desde su última copia.
6	Dispositivo	Para uso interno del sistema operativo, no fue adoptado para los archivos.
7	Reservado	Reservado, no debe ser manipulado.

cualquier modificación resultaría mucho más *caro*, dado que tendría que reescribirse el directorio completo al crearse o eliminarse un archivo dentro de éste, y lo que es más importante, más *peligroso*, dado que aumentaría el tiempo que los datos del directorio están en un estado inconsistente, aumentando la probabilidad de que ante una interrupción repentina (fallo de sistema, corte eléctrico, desconexión del dispositivo, etc.) se presentara corrupción de la información llevando a pérdida de datos. Al almacenar las entradas del directorio sin ordenar, las escrituras que modifican esta crítica estructura se mantienen atómicas: un solo sector de 512 bytes puede almacenar 16 entradas básicas de FAT, de 32 bytes cada una.[10]

Ordenar las entradas del directorio teniendo sus contenidos ya en memoria y, en general, diferir las modificaciones al directorio resulta mucho más conveniente en el caso general. Esto vale también para la eliminación de archivos —a continuación se abordará la estrategia que sigue FAT. Cabe recordar que FAT fue diseñado cuando el medio principal de almacenamiento era el disco flexible, decenas de veces más lento que el disco duro, y con mucha menor confiabilidad.

Cuando se le solicita a un sistema de archivos FAT eliminar un archivo, éste no se borra del directorio, ni su información se libera de la tabla de asignación de archivos, sino que se *marca* para ser ignorado, reemplazando el primer caracter de su nombre por `0xE5`. Ni la entrada de directorio, ni la *cadena* de *clusters* correspondiente en las tablas de asignación,[11] son eliminadas —sólo son marcadas como *disponibles*. El espacio de almacenamiento que el archivo eliminado ocupa debe, entonces, ser *sumado* al espacio libre que tiene el volumen. Es sólo cuando se crea un nuevo archivo empleando esa misma entrada, o cuando otro archivo ocupa el espa-

[10] Aunque en el caso de VFAT, las diferentes entradas que componen un sólo nombre de archivo pueden quedar separadas en diferentes sectores.

[11] Este tema será abordado en breve, en la sección 44.4, al hablar de las tablas de asignación de archivos.

cio físico que ocupaba el que fue eliminado, que el sistema operativo marca *realmente* como desocupados los *clusters* en la tabla de asignación.

Es por esto por lo que desde los primeros días de las PC hay tantas herramientas de recuperación (o *des-borramiento*, *undeletion*) de archivos: siempre que no haya sido creado uno nuevo que ocupe la entrada de directorio en cuestión, recuperar un archivo es tan simple como volver a ponerle el primer caracter a su nombre.

Este es un ejemplo de un *mecanismo flojo* (en contraposición de los *mecanismos ansiosos*, como los vistos en la sección 34.1). Eliminar un archivo requiere de un trabajo mínimo, mismo que es *diferido* al momento de reutilización de la entrada de directorio.

43.5. Compresión y *desduplicación*

Los archivos almacenados en un área dada del disco tienden a presentar patrones comunes. Algunas situaciones ejemplo que llevarían a estos patrones comunes son:

- Dentro del directorio de trabajo de cada uno de los usuarios hay típicamente archivos creados con los mismos programas, compartiendo encabezado, estructura, y ocasionalmente incluso parte importante de los datos.
- Dentro de los directorios de binarios del sistema, habrá muchos archivos ejecutables compartiendo el mismo *formato binario*.
- Es muy común también que un usuario almacene versiones distintas del mismo documento.
- Dentro de un mismo documento, es frecuente que el autor repita en numerosas ocasiones las palabras que describen sus conceptos principales.

Conforme las computadoras aumentaron su poder de cómputo, desde finales de los ochenta se presentaron varios mecanismos que permitían aprovechar las regularidades en los datos almacenados en el disco para comprimir el espacio utilizable en un mismo medio. La compresión típicamente se hace por medio de mecanismos estimativos derivados del análisis del contenido,[12] que tienen como resultado un nivel variable de compresión: con tipos de contenido altamente regulares (como podría ser texto, código fuente, o audio e imágenes *en crudo*), un volumen puede almacenar frecuentemente mucho más de 200% de su espacio real.

Con contenido poco predecible o con muy baja redundancia (como la mayor parte de formatos de imágenes y audio, que incluyen ya una fase de compresión, o empleando cualquier esquema de cifrado) la compresión no ayuda, y sí reduce el rendimiento global del sistema en que es empleada.

43.5.1. Compresión de volumen completo

El primer sistema de archivos que se popularizó fue *Stacker*, comercializado a partir de 1990 por *Stac Electronics*. *Stacker* operaba bajo MS-DOS, creando un dispositivo de bloques virtual alojado en un disco estándar.[13] Varias implementaciones posteriores de esta misma época se basaron en este mismo principio.

Ahora, sumando la variabilidad derivada del enfoque probabilístico al uso del espacio con el ubicarse como una compresión orientada al volumen entero, resulta natural encontrar una de las dificultades resultantes

[12] Uno de los algoritmos más frecuentemente utilizados y fáciles de entender es la *Codificación Huffman*; éste y la familia de algoritmos *Lempel-Ziv* sirven de base para prácticamente la totalidad de las implementaciones.

[13] Esto significa que, al solicitarle la creación de una unidad comprimida de 30 MB dentro del volumen C (disco duro primario), ésta aparecería disponible como un volumen adicional. El nuevo volumen requería de la carga de un *controlador* especial para ser *montado* por el sistema operativo.

del uso de estas herramientas: dado que el sistema operativo estructura las operaciones de lectura y escritura por bloques de dimensiones regulares (por ejemplo, el tamaño típico de sector hardware de 512 bytes), al poder éstos traducirse a más o menos bloques reales tras pasar por una capa de compresión, es posible que el sistema de archivos tenga que reacomodar constantemente la información al *crecer* alguno de los bloques previos. Conforme mayor era el tiempo de uso de una unidad comprimida por *Stacker*, se notaba más degradación en su rendimiento.

Además, dado que bajo este esquema se empleaba el sistema de archivos estándar, las tablas de directorio y asignación de espacio resultaban también comprimidas. Estas tablas, como ya se ha expuesto, contienen la información fundamental del sistema de archivos; al comprimirlas y reescribirlas constantemente, la probabilidad de que resulten dañadas en alguna falla (eléctrica o lógica) aumenta. Y si bien los discos comprimidos por *Stacker* y otras herramientas fueron populares, principalmente durante la primera mitad de los noventa, conforme aumentó la capacidad de los discos duros fue declinando su utilización.

43.5.2. Compresión archivo por archivo

Dado el éxito del que gozó *Stacker* en sus primeros años, Microsoft anunció como parte de las características de la versión 6.0 de MS-DOS (publicada en 1993) que incluiría *DoubleSpace*, una tecnología muy similar a la de *Stacker*. Microsoft incluyó en sus sistemas operativos el soporte para DoubleSpace por siete años, cubriendo las últimas versiones de MS-DOS y las de Windows 95, 98 y Millenium, pero como ya se vio, la compresión de volumen completo presentaba importantes desventajas.

Para el entonces nuevo sistemas de archivos NTFS, Microsoft decidió incluir una característica distinta, más segura y más modular: mantener el sistema de archivos funcionando de forma normal, sin compresión, y habilitar la compresión *archivo por archivo* de forma transparente al usuario.

Este esquema permite al administrador del sistema elegir, por archivos o carpetas, qué areas del sistema de archivos desea almacenar comprimidas; esta característica viene como parte de todos los sistemas operativos Windows a partir de la versión XP, liberada en el año 2003.

Si bien la compresión transparente por archivo se muestra mucho más atractiva que la de volumen completo, no es una panacea y es frecuente encontrar en foros en línea la recomendación de deshabilitarla. En primer término, comprimir un archivo implica que un cambio pequeño puede tener un efecto mucho mayor: modificar un bloque puede implicar que el tamaño final de los datos cambie, lo cual se traduciría a la reescritura del archivo desde ese punto en adelante; esto podría mitigarse insertando espacios para preservar el espacio ya ocupado, pero agrega complejidad al proceso (y abona en contra de la compresión). Los archivos comprimidos son además mucho más sensibles a la corrupción de datos, particularmente en casos de fallo de sistema o de energía: dado que un cambio menor puede resultar en la necesidad de reescribir al archivo completo, la ventana de tiempo para que se produzca un fallo se incrementa.

En archivos estructurados para permitir el acceso aleatorio, como podrían ser las tablas de bases de datos, la compresión implicaría que los registros no estarán ya alineados al tamaño que el programa gestor espera, lo cual acarreará necesariamente una penalización en el rendimiento y en la confiabilidad.

Por otro lado, los formatos nativos en que se expresan los datos que típicamente ocupan más espacio en el almacenamiento de los usuarios finales implican ya un alto grado de compresión: los archivos de fotografías, audio o video se codifican bajo diversos esquemas de compresión aptos para sus particularidades. Y comprimir un archivo que ya está comprimido no sólo no reporta ningún beneficio, sino que resulta en desperdicio de trabajo por el esfuerzo invertido en descomprimirlo cada vez que es empleado.

La compresión transparente, archivo por archivo, tiene innegables ventajas, sin embargo, por las desventajas que implica, no puede tomarse como el modo de operación por omisión.

43.5.3. Desduplicación

Hay una estrategia fundamentalmente distinta para optimizar el uso del espacio de almacenamiento, logrando muy altos niveles de *sobreuso*: guardar *sólo una copia* de cada cosa.

Desde fines de los ochenta se han planteado sistemas implementando distintos tipos de desduplicación, aunque su efectividad era muy limitada y, por tanto, su nivel de adopción se mantuvo muy reducido hasta recientemente.

El que se retomara la desduplicación se debe en buena medida a la *consolidación* de servidores ante la adopción a gran escala de mecanismos de virtualización (véanse apéndice IX y en particular la sección I). Dado que un mismo servidor puede estar alojando a decenas o centenas de *máquinas virtuales*, muchas de ellas con el mismo sistema operativo y programas base, los mismos archivos se repiten muchas veces; si el sistema de archivos puede determinar que cierto archivo o bloque está ya almacenado, podría guardar sólamente una copia.

La principal diferencia entre la desduplicación y las *ligas duras* mencionadas en la sección 39.1.5 es que, en caso de que cualquiera de estos archivos (o bloques) sea modificado, el sistema de archivos tomará el espacio necesario para representar estos cambios y evitará que esto afecte a los demás archivos. Además, si dos archivos inicialmente distintos se hacen iguales, se liberará el espacio empleado por uno de ellos de forma automática.

Para identificar qué datos están duplicados, el mecanismo más utilizado es calcular el *hash criptográfico* de los datos;[14] este mecanismo permite una búsqueda rápida y confiable de coincidencias, ya sea por archivo o por bloque.

La desduplicación puede realizarse *en línea* o *fuera de línea* — esto es, analizar los datos buscando duplicidades al momento que estos llegan al sistema, o (dado que es una tarea intensiva, tanto en uso de procesador, como de entrada/salida) realizarla como una tarea posterior de mantenimiento, en un momento de menor ocupación del sistema.

Desduplicar por archivo es mucho más ligero para el sistema que hacerlo por bloque, aunque dadas las estructuras comunes que comparten muchos archivos, desduplicar por bloque lleva típicamente a una mucho mayor optimización del uso de espacio.

Al día de hoy, el principal sistema de archivos que implementa desduplicación es ZFS,[15] desarrollado por Sun Microsystems (hoy Oracle). En Linux, esta característica forma parte de BTRFS, aunque no ha alcanzado los niveles de estabilidad como para recomendarse para su uso en entornos de producción.

En Windows, esta funcionalidad se conoce como *single instance storage* (*almacenamiento de instancia única*). Esta característica apareció, implementada en espacio de usuario y, por tanto, operando únicamente por archivo, como una de las características del servidor de correo *Exchange Server* entre los años 2000 y 2010. A partir de Windows Server 2003, la funcionalidad de desduplicación existe directamente para NTFS, aunque su uso es poco frecuente.

El uso de desduplicación, particularmente cuando se efectúa por bloques, tiene un alto costo en memoria: para mantener buenos niveles de rendimiento, la tabla que relaciona el hash de datos con el sector en el cual está almacenado debe mantenerse en memoria. En el caso de la implementación de ZFS en FreeBSD, la documentación sugiere dedicar 5 GB de memoria por cada TB de almacenamiento (0.5% de la capacidad total) (Rhodes y col. 1995-2014).

[14] Por ejemplo, empleando el algoritmo SHA-256, el cual brinda una probabilidad de *colisión* de 1 en 2^{256}, suficientemente confiable para que la pérdida de datos sea mucho menos probable que la falla del disco.

[15] Las características básicas de ZFS serán presentadas en la sección M.2.

43.6. Sistemas de archivos virtuales

Los sistemas de archivos nacieron para facilitar el uso de dispositivos de almacenamiento para la información que debe persistir a largo plazo. Sin embargo, dado que la abstracción para su uso resulta tan natural para representar todo tipo de información, desde la década de los ochenta aparecieron diversas implementaciones de discos RAM: programas diseñados para tomar un espacio de memoria y darle la semántica de un disco, permitiendo manipular su contenido como si fuera un disco cualquiera. Esto es, presenta almacenamiento volátil, pero de muy alta velocidad.

Estos esquemas gozaron de relativa popularidad a lo largo de los ochenta y noventa; fue, sin embargo, con la popularización de los sistemas que implementan memoria virtual que este esquema se volvió verdaderamente popular. En los sistemas Linux modernos, la norma es el uso de sistemas de archivos virtuales para el espacio de almacenamiento temporal `/tmp`, e incluso otros espacios tradicionalmente persistentes (como el directorio con los archivos de dispositivos, `/dev`) se han convertido en virtuales.

43.6.1. Una ventana al núcleo

El artículo titulado *Processes as files* (Killian 1984), presentado en el congreso de desarrolladores de sistemas Unix del grupo USENIX, presenta la implementación un sistema de archivos virtual para ser montado en el directorio `/proc` del sistema. Dicho sistema de archivos, en vez de representar un mapa de bloques en un dispositivo físico, presentó una ventana a la lista de procesos en ejecución en el sistema. El uso de `/proc` fue incorporado a prácticamente todos los sistemas tipo Unix.

Esta idea ha avanzado y madurado, y al día de hoy es una herramienta fundamental para la administración de sistemas. Si bien la propuesta original de Killian estaba encaminada principalmente a facilitar la implementación de depuradores presentando el espacio de memoria de los procesos, `/proc` es hoy en día una suerte de ventana a estructuras internas del sistema: un árbol de directorios con una gran cantidad de archivos por medio de los cuales se puede monitorear el estado del sistema (memoria libre, número de procesos, consumo de procesador, etc.), e incluso modificar la configuración del sistema en ejecución.

Por ejemplo, en Linux, leer de `/proc/sys/vm/swappiness` dará, por omisión, un valor de 60. Escribir el valor 100 a este archivo tendrá por resultado inmediato que el sistema sea más *agresivo* con la paginación de memoria virtual, buscando dejar más espacio de memoria física disponible (véase la sección 34) de lo que normalmente haría. Por otro lado, escribir el valor uno tendrá por efecto que el sistema operativo realice la paginación sólo si no tiene otra alternativa. Un valor de cero tendrá como efecto el deshabilitar la paginación por completo. Este archivo, al igual que todos los que conforman `/proc`, se presenta ante los procesos en ejecución como un archivo de tipo normal —la particularidad radica en el sistema de archivos en que se ubica.

Claro está, por más que parezca una simple lectura de un archivo, el leer o modificar un valor dentro de `/proc` cruza también por una serie de llamadas al sistema; el que la configuración de la memoria virtual esté disponible mediante algo que *parece ser* un archivo (sin serlo) es precisamente un caso de abstracción hacia una interfaz consistente y conocida.

Capítulo 44

Esquemas de asignación de espacio

Hasta este punto, la presentación de la *entrada de directorio* se ha limitado a indicar que ésta tiene un apuntador al lugar donde inicia el espacio empleado por el archivo. No se ha detallado en cómo se implementa la asignación y administración de dicho espacio. En esta sección se hará un breve repaso de los tres principales mecanismos, para después explicar cómo es la implementación de FAT, abordando sus principales virtudes y debilidades.

44.1. Asignación contigua

Los primeros sistemas de archivos en disco empleaban un esquema de *asignación contigua*. Para implementar un sistema de archivos de este tipo, no haría falta contar con una *tabla de asignación de archivos*: bastaría con la información que forma parte del directorio de FAT —la extensión del archivo y la dirección de su primer *cluster*.

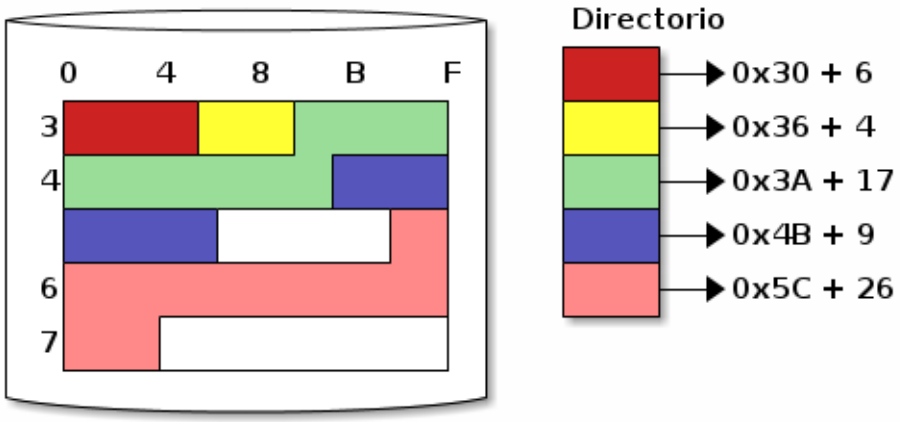


Figura 44.1: Asignación contigua de archivos: directorio con inicio y longitud.

La principal ventaja de este mecanismo de asignación, claro está, es la simplicidad de su implementación. Brinda además la mejor velocidad de transferencia del archivo, dado que al estar cada uno de los archivos en espacio contiguo en el disco, el movimiento de cabezas se mantiene al mínimo. Sin embargo, este mecanismo se vuelve sumamente inconveniente en medios que soporten lectura y escritura: es muy sensible a la *fragmentación externa*; si un archivo requiere crecer, debe ser movido íntegramente a un bloque más grande (lo cual toma demasiado tiempo), y el espacio que libera un archivo en caso de reducirse su necesidad de espacio queda *atrapado* entre bloques asignados. Podemos tener mucho más espacio disponible que el que podamos asignar a un nuevo archivo.

Los esquemas de asignación contigua se emplean hoy en día principalmente en sistemas de archivo de sólo lectura, por ejemplo, lo emplea el sistema principal que utilizan los CD-ROM, ISO-9660, pensado para

aprovechar al máximo un espacio que, una vez grabado, sólo podrá abrirse en modo de sólo lectura. Esto explica porqué, a diferencia de lo que ocurre en cualquier otro medio de almacenamiento, al *quemar* un CD-ROM es necesario preparar primero una *imagen* en la que los archivos ocupen sus posiciones definitivas, y esta imagen debe grabarse en el disco en una sola operación.

44.2. Asignación ligada

Un enfoque completamente distinto sería el de *asignación ligada*. Este esquema brinda mucho mayor flexibilidad al usuario, sacrificando la simplicidad y la velocidad: cada entrada en el directorio apunta a un primer *grupo* de sectores (o *cluster*), y éste contiene un apuntador que indica cuál es el siguiente.

Para hacer esto, hay dos mecanismos: el primero, reservar un espacio al final de cada *cluster* para guardar el apuntador y, el segundo, crear una tabla independiente, que guarde únicamente los apuntadores.

En el primer caso, si se manejan *clusters* de 2 048 bytes, y se reservan los cuatro bytes (32 bits) finales de cada uno, el resultado sería de gran incomodidad a los programadores, ya que, frecuentemente, buscan alinear sus operaciones con las fronteras de las estructuras subyacentes, para optimizar los accesos (por ejemplo, evitar que un sólo registro de base de datos requiera ser leído de dos distintos bloques en disco). El programador tendría que diseñar sus estructuras para ajustarse a la poco ortodoxa cantidad de 2 044 bytes.

Y más allá de esta inconveniencia, guardar los apuntadores al final de cada *cluster* hace mucho más lento el manejo de todos los archivos: al no tener en una sola ubicación la relación de *clusters* que conforman un archivo, todas las transferencias se convierten en *secuenciales*: para llegar directamente a determinado bloque del archivo, habrá que pasar por todos los bloques previos para encontrar su ubicación.

Particularmente por este segundo punto es mucho más común el empleo de una *tabla de asignación de archivos* —y precisamente así es como opera FAT (de hecho, esta tabla es la que le da su nombre). La tabla de asignación es un mapa de los *clusters*, representando a cada uno por el espacio necesario para guardar un apuntador.

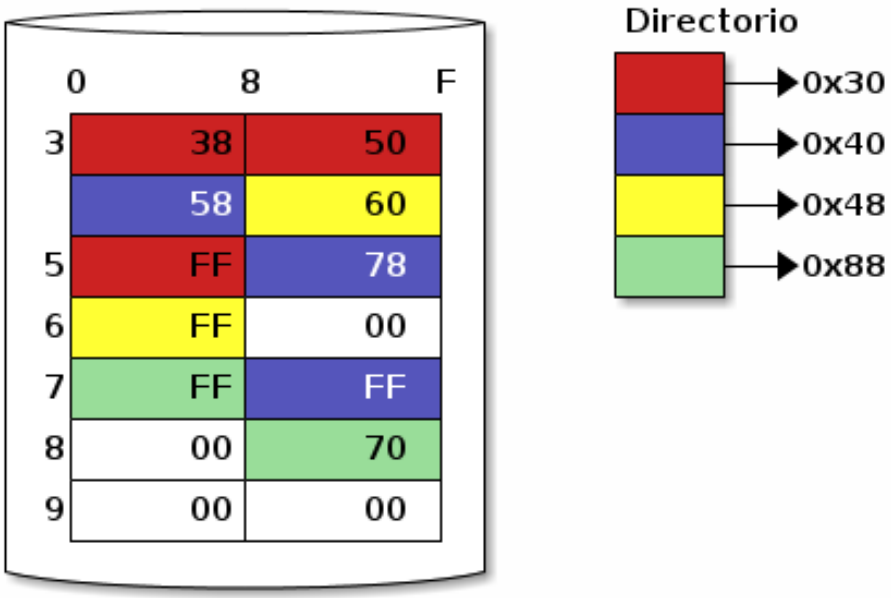


Figura 44.2: Asignación ligada de archivos: directorio con apuntador sólo al primer *cluster*.

La principal ventaja del empleo de asignación ligada es que desaparece la *fragmentación interna*.[1] Al ya no requerir la *pre-asignación* de un espacio contiguo, cada uno de los archivos puede crecer o reducirse según sea necesario.

Ahora, la *asignación ligada* no sólo resulta más lenta que la contigua, sino que presenta una mayor *sobrecarga administrativa*: el espacio desperdiciado para almacenar los apuntadores típicamente es cercano a 1% del disponible en el medio.

Este esquema también presenta *fragilidad de metadatos*: si alguno de los apuntadores se pierde o corrompe, lleva a que se pierda el archivo *completo* desde ese punto y hasta su final (y abre la puerta a la corrupción de otro archivo, si el apuntador queda apuntando hacia un bloque empleado por éste; el tema de fallos y recuperación bajo estos esquemas se cubrirá en la sección 45).

Hay dos mecanismos de mitigación para este problema: el empleado por los sistemas FAT es guardar una (o, bajo FAT12, dos) copias adicionales de la tabla de asignación, entre las cuales el sistema puede verificar que se mantengan consistentes y buscar corregirlas en caso contrario. Por otro lado, puede manejarse una estructura de *lista doblemente ligada* (en vez de una *lista ligada* sencilla) en que cada elemento apunte tanto al siguiente como al anterior, con lo cual, en caso de detectarse una inconsistencia en la información, ésta pueda ser recorrida *de atrás hacia adelante* para confirmar los datos correctos. En ambos casos, sin embargo, la sobrecarga administrativa se duplica.

44.3. Asignación indexada

El tercer modelo es la *asignación indexada*, el mecanismo empleado por casi todos los sistemas de archivos modernos. En este esquema, se crea una estructura intermedia entre el directorio y los datos, única para cada archivo: el *i-nodo* (o *nodo de información*). Cada uno guarda los metadatos y la lista de bloques del archivo, reduciendo la probabilidad de que se presente la *corrupción de apuntadores* mencionada en la sección anterior.

La sobrecarga administrativa bajo este esquema potencialmente es mucho mayor: al asignar el i-nodo, éste se crea ocupando como mínimo un *cluster* completo. En el caso de un archivo pequeño, que quepa en un sólo *cluster*, esto representa un desperdicio de 100% de espacio (un *cluster* para el i-nodo y otro para los datos);[2] para archivos más grandes, la sobrecarga relativa disminuye, pero se mantiene siempre superior a la de la asignación ligada.

Un esquema de asignación indexada brinda una mayor eficiencia de caché que la asignación ligada: si bien en dicho caso es común guardar copia de la tabla de asignación en memoria para mayor agilidad, con la asignación indexada bastará hacer caché *sólo de la información importante*, esto es, únicamente de los archivos que se emplean en un momento dado. El *mapa de asignación* para los archivos y directorios que no hayan sido empleados recientemente no requerirán estar en memoria.

Claro está, mientras que en los esquemas anteriores la tabla central de asignación de archivos puede emplearse directamente como el *bitmap* del volumen, en los sistemas de archivos de asignación indexada se vuelve necesario contar con un *bitmap* independiente — pero al sólo requerir representar si cada *cluster* está vacío u ocupado (y ya no apuntar al siguiente), resulta de mucho menor tamaño.

Ahora, ¿qué pasa cuando la lista de *clusters* de un archivo no cabe en un i-nodo? Este ejemplo se ilustra en el tercer archivo de la figura 44.4: en este caso, cada i-nodo puede guardar únicamente tres apuntadores.[3] Al

[1] Con *fragmentación interna* se hace aquí referencia al concepto presentado en la sección 31.1.1. El fenómeno generalmente conocido como *fragmentación* se refiere a la necesidad de *compactación*; es muy distinto, y sí se presenta bajo este esquema: cada archivo se separa en pequeños *fragmentos* que pueden terminar esparcidos por todo el disco, afectando fuertemente en el rendimiento del sistema.

[2] Algunos sistemas de archivos, como Reiser, BTRFS o UFS, presentan esquemas de asignación *sub-cluster*. Estos denominan *colas* (*tails*) a los archivos muy pequeños, y pueden ubicarlos ya sea dentro de su mismo i-nodo o compartiendo un mismo *cluster* con un *desplazamiento* dentro de éste. Esta práctica no ha sido adoptada por sistemas de archivos de uso mayoritario por su complejidad relativa.

[3] Esto resulta un límite demasiado bajo, y fue elegido meramente para ilustrar el presente punto.

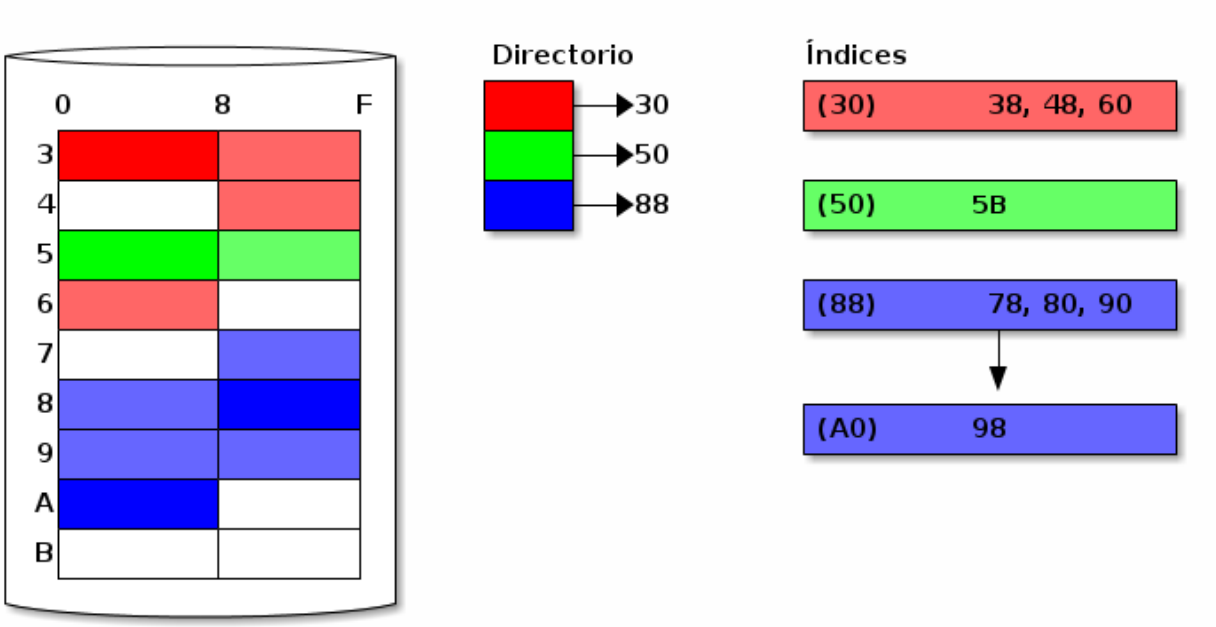


Figura 44.3: Asignación indexada de archivos: directorio con apuntador al i-nodo (ejemplificado con un i-nodo de tamaño extremadamente ineficiente).

tener un archivo con cuatro *clusters*, se hace necesario extender al i-nodo con una lista adicional. La implementación más común de este mecanismo es que, dependiendo del tamaño del archivo, se empleen apuntadores con los niveles de indirección que *vayan haciendo falta*.

¿Qué tan grande sería el archivo máximo direccionable bajo este esquema y únicamente tres indirecciones? Suponiendo magnitudes típicas hoy en día (*clusters* de 4 KB y direcciones de 32 bits), en un *cluster* vacío caben 128 apuntadores ($\frac{4\,096}{32}$). Si los metadatos ocupan 224 bytes en el i-nodo, dejando espacio para 100 apuntadores:

- Un archivo de hasta $(100-3) \times 4$ KB = 388 KB puede ser referido por completo directamente en el i-nodo, y es necesario un sólo acceso a disco para obtener su lista de *clusters*.
- Un archivo de hasta $(100-3+128) \times 4$ KB = 900 KB puede representarse con el bloque de indirección sencilla, y obtener su lista de *clusters* significa dos accesos a disco adicionales.
- Con el bloque de doble indirección, puede hacerse referencia a archivos mucho más grandes:

  $(100-3+128+(128\times128)) \times 4 \text{ KB} = 66\,436 \text{ KB} \approx 65 \text{ MB}$

  Sin embargo, a estas alturas comienza a llamar la atención otro importante punto: para acceder a estos 65 MB, es necesario realizar hasta 131 accesos a disco. A partir de este punto, resulta importante que el sistema operativo asigne *clusters* cercanos para los metadatos (y, de ser posible, para los datos), pues la diferencia en tiempo de acceso puede ser muy grande.
- Empleando triple indirección, se puede llegar hasta:

  $(100-3+128+(128\times128)+(128\times128\times128)) \times 2 \text{ KB} = 8\,455\,044 * kb* \approx 8 \text{ GB}$

  Esto es ya más de lo que puede representarse en un sistema de 32 bits. La cantidad de bloques a leerse, sin embargo, para encontrar todos los *clusters* significarían hasta 16 516 accesos a disco (en el peor de los casos).

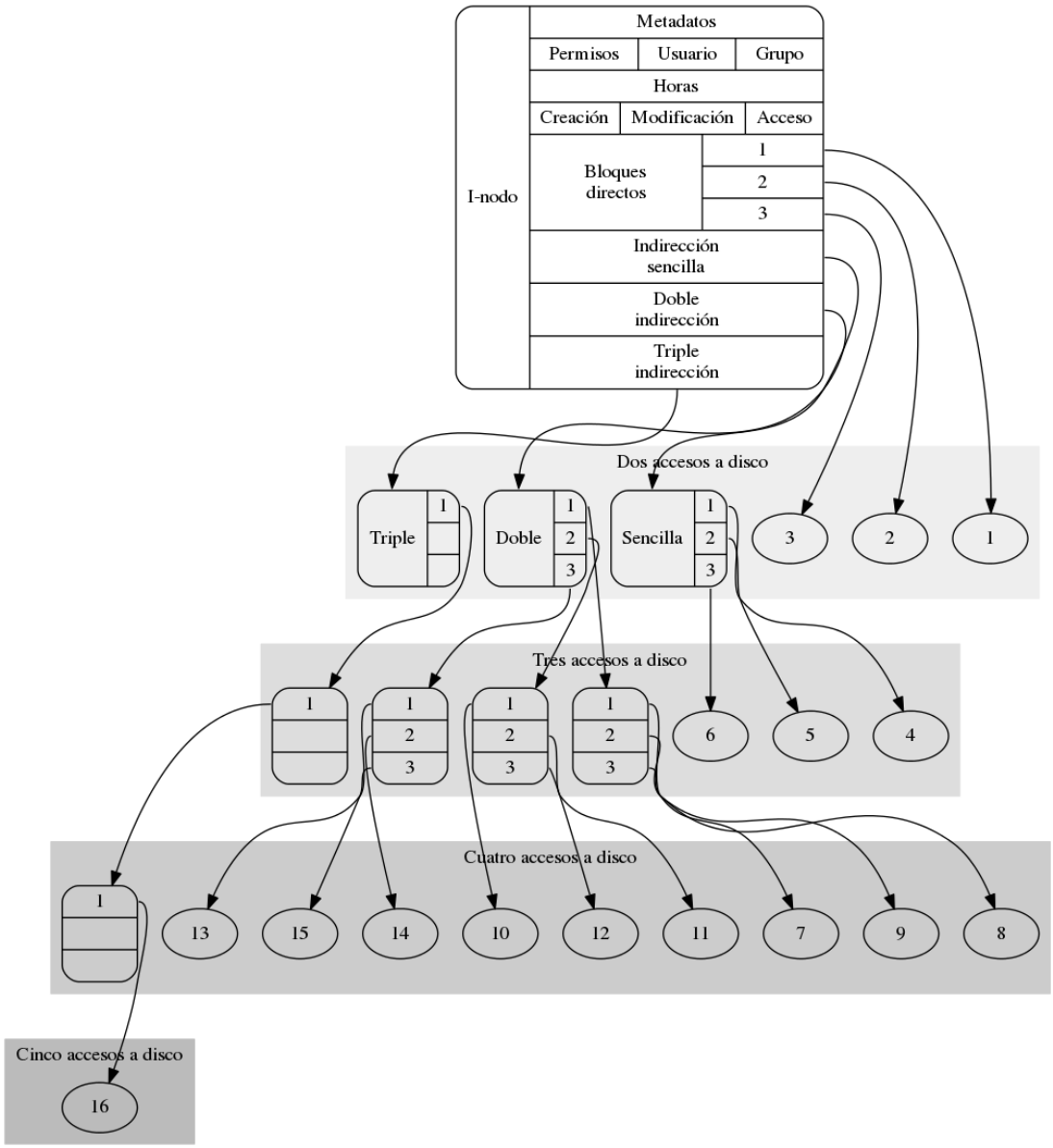


Figura 44.4: Estructura típica de un i-nodo en Unix, mostrando además el número de accesos a disco necesarios para llegar a cada *cluster* (con sólo tres *cluster* por lista).

44.4. Las tablas en FAT

Volviendo al caso que se presenta como ejemplo, para el sistema de archivos FAT, cada entrada del directorio apunta al primer *cluster* que ocupa cada uno de los archivos, y se emplea un esquema de asignación ligada. El directorio tiene también un campo indicando la *longitud total* del archivo, pero esto no es empleado para leer la información, sino para poderla presentar más ágilmente al usuario (y para que el sistema operativo sepa dónde indicar *fin de archivo* al leer el último *cluster* que compone a determinado archivo).

La estructura fundamental de este sistema de archivos es la tabla de asignación de archivos (*file allocation table*) — tanto que de ella toma el nombre por el cual se le conoce, FAT.

Cada entrada de la FAT mide lo que la longitud correspondiente a su versión (12, 16 o 32 bits), y almacena ya sea el siguiente *cluster* del archivo, o alguno de los valores especiales descritos en el cuadro 44.1.

Cuadro 44.1: Valores especiales que puede almacenar FAT; cualquier otro valor indica la dirección del siguiente *cluster* que forma parte del archivo al cual pertenece el registro en cuestión.

FAT12	FAT16	FAT32	Significado
0x000	0x0000	0x00000000	Disponible, puede ser asignado.
0xFF7	0xFFF7	0xFFFFFFF7	*Cluster* dañado, no debe utilizarse.
0xFFF	0xFFFF	0xFFFFFFFF	Último *cluster* de un archivo.

Llama la atención que haya un valor especial para indicar que un *cluster* tiene sectores dañados. Esto remite de vuelta al momento histórico de la creación de la familia FAT: siendo el medio predominante de almacenamiento el disco flexible, los errores en la superficie eran mucho más frecuentes de lo que lo son hoy en día.

Una característica que puede llamar la atención de FAT es que parecería permitir la fragmentación de archivos *por diseño*: dado que el descriptor de cada *cluster debe apuntar* al siguiente, puede asumirse que el *caso común* es que los *clusters* no ocuparán contiguos en el disco. Claro está, la tabla puede apuntar a varios *clusters* adyacentes, pero el sistema de archivos mismo no hace nada para que así sea.

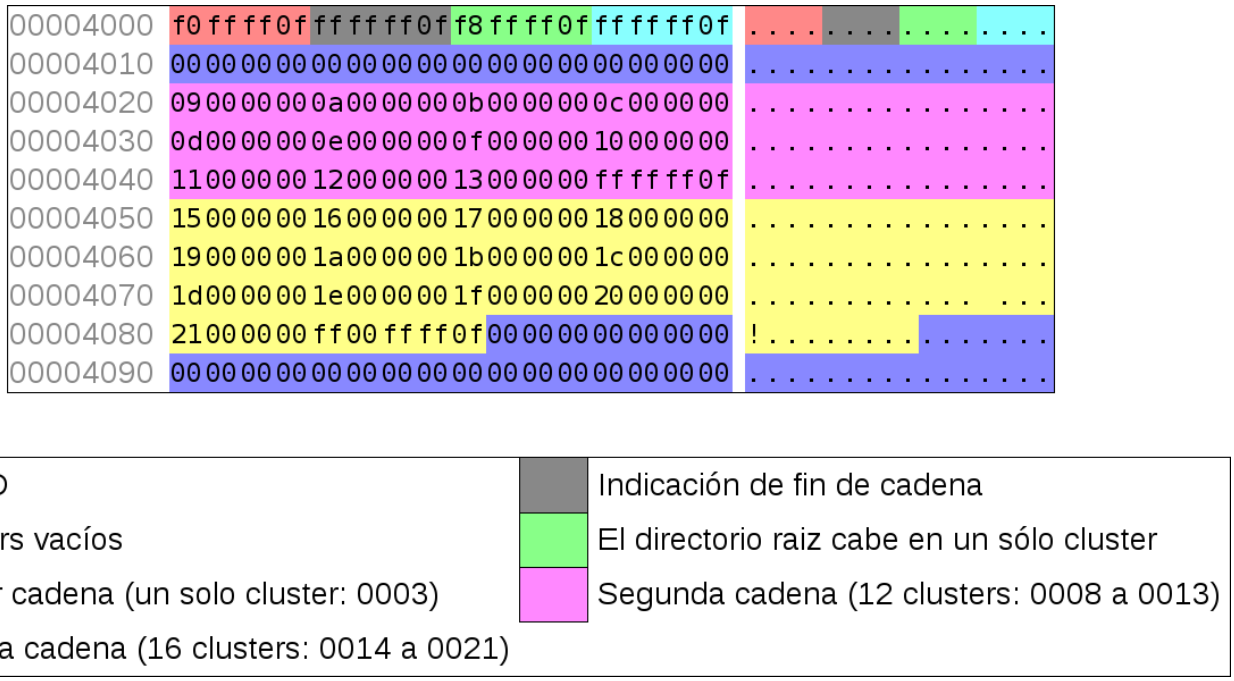


Figura 44.5: Ejemplo de entradas en la tabla de asignación de archivos en FAT32.

En la sección 43.4, al presentar el formato del directorio de FAT, se mencionó que los subdirectorios son en realidad archivos de un tipo especial: una suerte de archivos estructurados (véase la sección 38.5), gestionados por el sistema operativo. Lo único que distingue a un directorio de un archivo normal es que, en la entrada que lo describe en su directorio padre, el doceavo byte de la entrada (que indica los *atributos del archivo*, véanse la figura 43.1 y el cuadro 7.1) tiene activado el bit cuatro.

Un directorio es almacenado en disco *exactamente* como cualquier otro archivo. Si se le asigna únicamente un *cluster*, y el tamaño del *cluster* es pequeño (2 KB), podrá almacenar sólo 64 entradas ($\frac{2\,048}{32}$) y cada *cluster* adicional le dará 64 entradas más. Y dado que es tratado tal cual si fuera un archivo normal, estará sujeto también a la fragmentación: conforme se agreguen entradas al directorio, éste crecerá. Llegado el momento,

requerirá *clusters* adicionales. Y si un directorio termina disperso por todo el disco, resultará –como cualquier otro archivo– más lento leerlo y trabajar con él. Siempre que se abra un archivo dentro de un directorio grande, o que se le recorra para abrir algún archivo en un subdirectorio suyo, el sistema tendrá que buscar todos sus fragmentos a lo largo del disco.

Ante estos dos aspectos, no puede perderse de vista la edad que tiene el sistema FAT. Otros sistemas de archivos más modernos han resuelto este problema mediante los *grupos de asignación*: los directorios del sistema son *esparcidos* a lo largo del volumen, y *se intenta* ubicar los archivos cerca de los directorios desde donde son referidos.[4] Esto tiene como consecuencia que los archivos que presentan *cercanía temática* (que pertenecen al mismo usuario o proyecto, o que por alguna razón están en la misma parte de la jerarquía del sistema) quedan ubicados en disco cerca unos de otros (y cerca de sus directorios). Y dado que es probable que sean empleados aproximadamente al mismo tiempo, esto reduce las distancias que recorrerán las cabezas. Además, al esparcir los archivos, se distribuye también mejor el espacio libre, por lo cual el efecto de los cambios de tamaño de un archivo en lo relativo a la fragmentación se limita a los que forman parte del mismo bloque de asignación.

Los sistemas de archivos que están estructurados siguiendo esta lógica de grupos de asignación no evitan la fragmentación, pero sí la mayor parte de sus consecuencias negativas. Para mantener este esquema operando confiablemente, eso sí, requieren de mantener disponibilidad de espacio — al presentarse saturación, esta estrategia pierde efectividad. Para evitar que esto ocurra, es muy frecuente en los sistemas Unix que haya un cierto porcentaje (típicamente cercano a 5%) del disco disponible únicamente para el administrador — en caso de que el sistema de archivos pase de 95%, los usuarios no podrán escribir en él, pero el administrador puede efectuar tareas de mantenimiento para volver a un rango operacional.

Capítulo 45

Fallos y recuperación

El sistema de archivos FAT es *relativamente frágil*: no es difícil que se presente una situación de *corrupción de metadatos*, y muy particularmente, que ésta afecte la estructura de las tablas de asignación. Los usuarios de sistemas basados en FAT en Windows sin duda conocen a `CHKDSK` y `SCANDISK`, dos programas que implementan la misma funcionalidad base, y difieren principalmente en su interfaz al usuario: `CHKDSK` existe desde los primeros años de MS-DOS, y está pensado para su uso interactivo en línea de comando; `SCANDISK` se ejecuta desde el entorno gráfico, y presenta la particularidad de que no requiere (aunque sí recomienda fuertemente) *acceso exclusivo* al sistema de archivos mientras se ejecuta.

¿Cómo es el funcionamiento de estos programas?

A lo largo de la vida de un sistema de archivos, conforme los archivos se van asignando y liberando, cambian su tamaño, y conforme el sistema de archivos se monta y desmonta, pueden ir apareciendo *inconsistencias* en su estructura. En los sistemas tipo FAT, las principales inconsistencias[1] son:

[4] Claro está, en el caso de los archivos que están como *ligas duras* desde varios directorios, pueden ubicarse sólo cerca de uno de ellos.

[1] Que no las únicas. Éstas y otras más están brevemente descritas en la página de manual de `dosfsck` (véase la sección 46.3).

Archivos cruzados (*cross-linked file*) Recuérdese que la entrada en el directorio de un archivo incluye un apuntador al primer *cluster* de una *cadena*. Cada cadena debe ser única, esto es, ningún *cluster* debe pertenecer a más de un archivo. Si dos archivos incluyen al mismo *cluster*, esto indica una inconsistencia, y la única forma de resolverla es *truncar* uno de los archivos en el punto inmediato anterior a este cruce.

Cadenas perdidas o *huérfanas* (*lost clusters*) Cuando hay espacio marcado como ocupado en la tabla de asignación, pero no hay ninguna entrada de directorio haciendo referencia a ella, el espacio está efectivamente bloqueado y, desde la perspectiva del usuario, inutilizado; además, estas cadenas pueden ser un archivo que el usuario aún requiera.

Este problema resultó tan frecuente en versiones históricas de Unix que incluso hoy es muy común tener un directorio llamado `lost+found` en la raíz de todos los sistemas de archivos, donde `fsck` (el equivalente en Unix de `CHKDSK`) creaba ligas a los archivos perdidos por corrupción de metadatos.

Cada sistema de archivos podrá presentar un distinto conjunto de inconsistencias, dependiendo de sus estructuras básicas y de la manera en que cada sistema operativo las maneja.

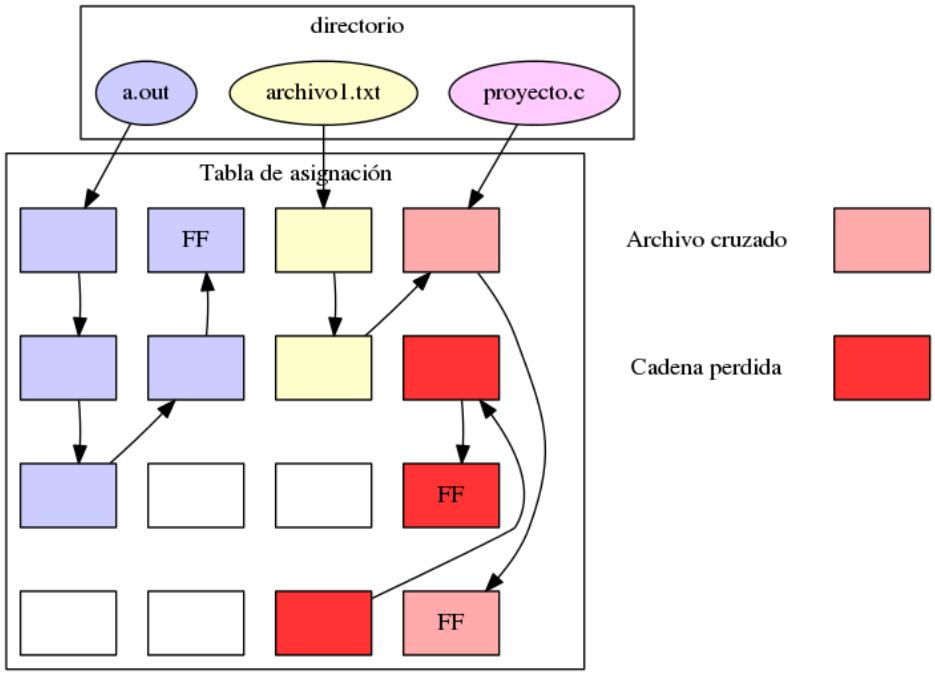


Figura 45.1: Inconsistencias en un sistema de archivos tipo FAT.

En la década de los 1980 comenzaron a venderse los *controladores de disco inteligentes*, y en menos de 10 años dominaban ya el mercado. Estos controladores, con interfaces físicas tan disímiles como SCSI, IDE, o los más modernos, SAS y SATA, introdujeron muchos cambios que fueron disociando cada vez más al sistema operativo de la gestión física directa de los dispositivos; en el apéndice X se presenta con mayor profundidad lo que esto ha significado para el desarrollo de sistemas de archivos y algoritmos relacionados. Sin embargo, para el tema en discusión, los *controladores inteligentes* resultan relevantes porque, si bien antes el sistema operativo podía determinar con toda certeza si una operación se había realizado o no, hoy en día los controladores dan un *acuse de recibo* a la información en el momento en que la colocan en el caché incorporado del dispositivo —en caso de un fallo de corriente, esta información puede no haber sido escrita por completo al disco.

Es importante recordar que las operaciones con los metadatos que conforman el sistema de archivos no son atómicas. Por poner un ejemplo, crear un archivo en un volumen FAT requiere:

1. Encontrar una lista de *clusters* disponibles suficiente para almacenar la información que conformará al archivo.
2. Encontrar el siguiente espacio disponible en el directorio.
3. Marcar en la tabla de asignación la secuencia de *clusters* que ocupará el archivo.
4. Crear en el espacio encontrado una entrada con el nombre del archivo, apuntando al primero de sus *clusters*.
5. Almacenar los datos del archivo en cuestión en los *clusters* determinados en el paso 1.

Cualquier fallo que se presente después del tercer paso (tras efectuarse la primera modificación) tendrá como consecuencia que el archivo resulte corrupto, y muy probablemente todo que el sistema de archivos *presente inconsistencias* o *esté en un estado inconsistente*.

45.1. Datos y metadatos

En el ejemplo recién presentado, el sistema de archivos estará en un estado consistente siempre que se terminen los pasos 3 y 4 —la consistencia del sistema de archivos es independiente de la validez de los datos del archivo. Lo que busca el sistema de archivos, más que garantizar la integridad de los *datos* de uno de los archivos, es asegurar la de los *metadatos*: los datos que describen la estructura.

En caso de que un usuario desconecte una unidad a media operación, es muy probable que se presente pérdida de información, pero el sistema de archivos debe buscar no presentar ningún problema que ponga en riesgo *operaciones posteriores* o *archivos no relacionados*. La corrupción y recuperación de datos en archivos corruptos y truncados, si bien es también de gran importancia, cae más bien en el terreno de las aplicaciones del usuario.

45.2. Verificación de la integridad

Cada sistema operativo incluye programas para realizar verificación (y, en su caso, corrección) de la integridad de sus sistemas de archivo. En el caso de MS-DOS y Windows, como ya se vio, estos programas son `CHKDSK` y `SCANDISK`; en los sistemas Unix, el programa general se llama `fsck`, y típicamente emplea a asistentes según el tipo de sistema que haya que revisar — `fsck.vfat`, `fsck.ext2`, etcétera.

Estos programas hacen un *barrido* del sistema de archivos, buscando evidencias de inconsistencia. Esto lo hacen, en líneas generales:

- Siguiendo todas las cadenas de *clusters* de archivos o tablas de i-nodos, y verificando que no haya archivos cruzados (compartiendo erróneamente bloques).
- Verificando que todas las cadenas de *clusters*, así como todos los directorios, sean alcanzables y sigan una estructura válida.
- Recalculando la correspondencia entre las estructuras encontradas y los diferentes bitmaps y totales de espacio vacío.

Estas operaciones son siempre procesos intensivos y complejos. Como requieren una revisión profunda del volumen entero, es frecuente que duren entre decenas de minutos y horas. Además, para poder llevar a cabo su tarea deben ejecutarse teniendo acceso exclusivo al volumen a revisar, lo cual típicamente significa colocar al sistema completo en modo de mantenimiento.

Dado el elevado costo que tiene verificar el volumen entero, en la década de los noventa surgieron varios esquemas orientados a evitar la necesidad de invocar a estos programas de verificación: las *actualizaciones suaves*, los *sistemas de archivos con bitácora*, y los *sistemas de archivos estructurados en bitácora*.

45.3. Actualizaciones suaves

El esquema de *actualizaciones suaves* (*soft updates*) aparentemente es el más simple de los que se presentan, pero su implementación resultó mucho más compleja de lo inicialmente estimado y, en buena medida, por esta causa no ha sido empleado más ampliamente. La idea básica detrás de este esquema es estructurar el sistema de archivos de una forma más simple y organizar las escrituras del mismo de modo que el estado resultante *no pueda* ser inconsistente, ni siquiera en caso de fallo, y de exigir que todas las operaciones de actualización de metadatos se realicen de forma *síncrona*.[2]

Ante la imposibilidad de tener un sistema *siempre consistente*, esta exigencia se relajó para permitir inconsistencias *no destructivas*: pueden presentarse *cadenas perdidas*, dado que esto no pone en riesgo a ningún archivo, sólo disminuye el espacio total disponible.

Esto, aunado a una reestructuración del programa de verificación (`fsck`) como una tarea *ejecutable en el fondo*[3] y en una tarea de *recolector de basura*, que no requiere intervención humana (dado que no pueden presentarse inconsistencias destructivas), permite que un sistema de archivos que no fue *limpiamente desmontado* pueda ser montado y utilizado de inmediato, sin peligro de pérdida de información o de corrupción.

Al requerir que todas las operaciones sean síncronas, parecería que el rendimiento global del sistema de archivos tendría que verse afectado, pero por ciertos patrones de acceso muy frecuentes, resulta incluso beneficioso. Al mantenerse un ordenamiento lógico entre las dependencias de todas las operaciones pendientes, el sistema operativo puede *combinar* muchas de estas y reducir de forma global las escrituras a disco.

A modo de ejemplos: si varios archivos son creados en el mismo directorio de forma consecutiva, cada uno de ellos mediante una llamada `open()` independiente, el sistema operativo combinará todos estos accesos en uno solo, reduciendo el número de llamadas. Por otro lado, un patrón frecuente en sistemas Unix es que, para crear un archivo de uso temporal reforzando la confidencialidad de su contenido, el proceso solicite al sistema la creación de un archivo, abra el archivo recién creado, y ya teniendo al descriptor de archivo, lo elimine —en este caso, con estas tres operaciones seguidas, *soft updates* podría ahorrarse por completo la escritura a disco.

Esta estrategia se vio afectada por los controladores inteligentes: si un disco está sometido a carga intensa, no hay garantía para el sistema operativo del orden que seguirán *en verdad* sus solicitudes, que se *forman* en el caché propio del disco. Dado que las actualizaciones suaves dependen tan profundamente de confiar en el ordenamiento, esto rompe por completo la confiabilidad del proceso.

Las actualizaciones suaves fueron implementadas hacia 2002 en el sistema operativo FreeBSD, y fueron adoptadas por los principales sistemas de la familia BSD, aunque NetBSD lo retiró en 2012, prefiriendo el empleo de sistemas con bitácora, tema que será tratado a continuación. Muy probablemente, la lógica detrás de esta decisión sea la cantidad de sistemas que emplean esta segunda estrategia, y lo complejo que resulta dar mantenimiento dentro del núcleo a dos estrategias tan distintas.

[2] Esto es, no se le reporta éxito en alguna operación de archivos al usuario sino hasta que ésta es completada y grabada a disco.

[3] Una tarea que no requiere de intervención manual por parte del operador, y se efectúa de forma automática como parte de las tareas de mantenimiento del sistema.

45.4. Sistemas de archivo con bitácora

Este esquema tiene su origen en el ámbito de las bases de datos distribuidas. Consiste en separar un área del volumen y dedicarla a llevar una bitácora (*journal*) con todas las *transacciones* de metadatos.[4] Una *transacción* es un conjunto de operaciones que deben aparecer como atómicas.

La bitácora se implementa generalmente como una *lista ligada circular*, con un apuntador que indica cuál fue la última operación realizada exitosamente. Periódicamente, o cuando la carga de transferencia de datos disminuye, el sistema verifica qué operaciones quedaron pendientes, y *avanza* sobre la bitácora, marcando cada una de las transacciones conforme las realiza.

En caso de tener que recuperarse de una condición de fallo, el sistema operativo sólo tiene que leer la bitácora, encontrar cuál fue la última operación efectuada, y aplicar las restantes.

Una restricción de este esquema es que las transacciones guardadas en la bitácora deben ser *idempotentes*—esto es, si una de ellas es efectuada dos veces, el efecto debe ser exactamente el mismo que si hubiera sido efectuada una sola vez. Por poner un ejemplo, no sería válido que una transacción indicara "agregar al directorio x un archivo llamado y", dado que si la falla se produce después de procesar esta transacción pero antes de avanzar al apuntador de la bitácora, el directorio x quedaría con dos archivos y —una situación inconsistente. En todo caso, tendríamos que indicar "registrar al archivo y en la posición z del directorio x"; de esta manera, incluso si el archivo ya había sido registrado, puede volverse a hacerlo sin peligro.

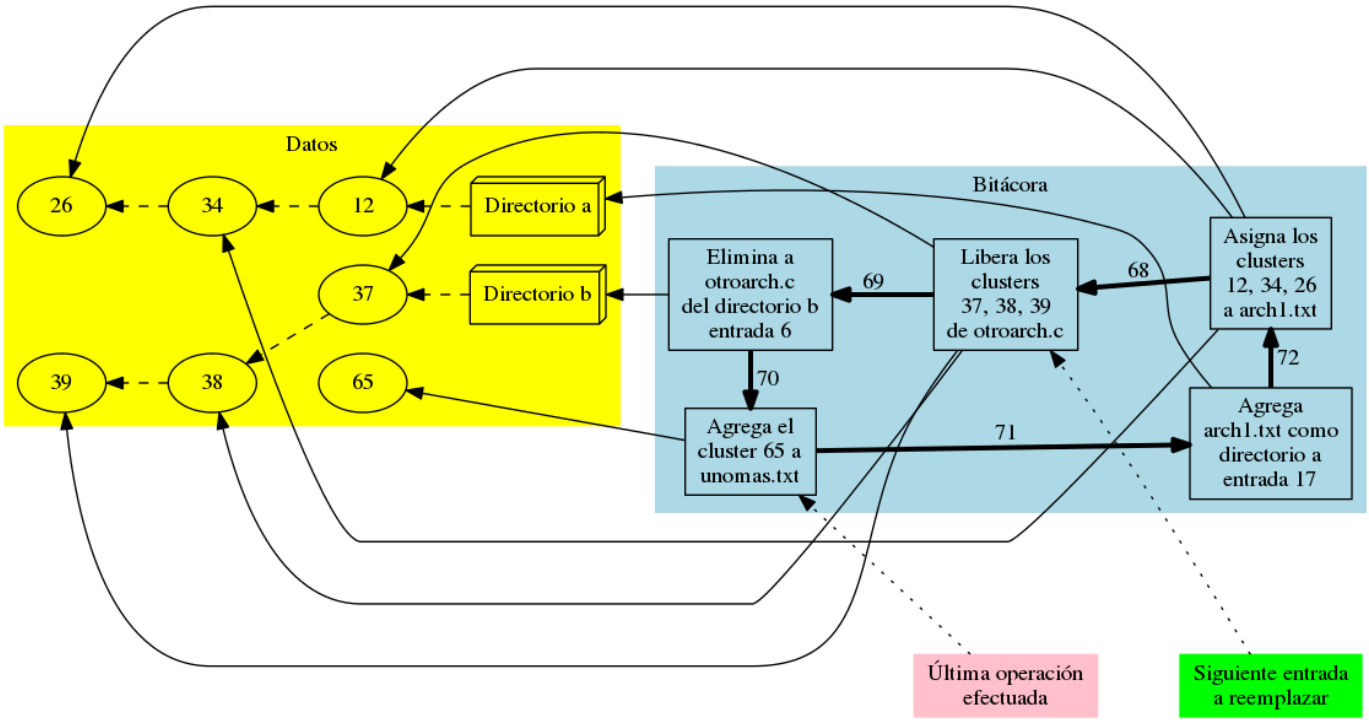


Figura 45.2: Sistema de archivos con bitácora.

Este esquema es el más utilizado hoy en día, y está presente en casi todos los sistemas de archivos modernos. Si bien con un sistema con bitácora no hace falta verificar el sistema de archivos completo tras una detención abrupta, esta no exime de que, de tiempo en tiempo, el sistema de archivos sea verificado: es alta-

[4]Hay implementaciones que registran también los datos en la bitácora, pero tanto por el tamaño que ésta requiere como por el efecto en velocidad que significa, son poco utilizadas. La sección 45.5 presenta una idea que elabora sobre una bitácora que almacena tanto datos como metadatos.

mente recomendado hacer una verificación periódica en caso de presentarse alguna corrupción, sea por algún bug en la implementación, fallos en el medio físico, o factores similarmente poco frecuentes.

La mayor parte de los sistemas de archivos incluyen contadores de *cantidad de montajes* y de *fecha del último montaje*, que permiten que el sistema operativo determine, automáticamente, si corresponde hacer una verificación preventiva.

45.5. Sistemas de archivos estructurados en bitácora

Si se lleva el concepto del sistema de archivos con bitácora a su límite, y se designa a *la totalidad* del espacio en el volumen como la bitácora, el resultado es un sistema de archivos *estructurado en bitácora* (*log-structured file systems*). Obviamente, este tipo de sistemas de archivos presenta una organización completa radicalmente diferente de los sistemas de archivo tradicionales.

Las ideas básicas detrás de la primer implementación de un sistema de archivos de esta naturaleza (Ousterhut y Rosenblum, 1992) apuntan al empleo agresivo de caché de gran capacidad, y con un fuerte mecanismo de *recolección de basura*, reacomodando la información que esté más cerca de la *cola* de la bitácora (y liberando toda aquella que resulte redundante).

Este tipo de sistemas de archivos facilita las escrituras, haciéndolas siempre secuenciales, y buscan –por medio del empleo del caché ya mencionado– evitar que las cabezas tengan que desplazarse con demasiada frecuencia para recuperar fragmentos de archivos.

Una consecuencia directa de esto es que los sistemas de archivos estructurados en bitácora fueron los primeros en ofrecer *fotografías* (*snapshots*) del sistema de archivos: es posible apuntar a un momento en el tiempo, y –con el sistema de archivos aún en operación– montar una copia de sólo lectura con la información del sistema de archivos *completa* (incluyendo los datos de los archivos).

Los sistemas de archivo estructurados en bitácora, sin embargo, no están optimizados para cualquier carga de trabajo. Por ejemplo, una base de datos relacional, en que cada uno de los registros es típicamente actualizado de forma independiente de los demás, y ocupan apenas fracciones de un bloque, resultaría tremendamente ineficiente. La implementación referencia de Ousterhut y Rosenblum fue parte de los sistemas BSD, pero dada su tendencia a la *extrema fragmentación*, fue eliminado de ellos.

Este tipo de sistemas de archivo ofrece características muy interesantes, aunque es un campo que aún requiere de mucha investigación e implementaciones ejemplo. Muchas de las implementaciones en sistemas libres han llegado a niveles de funcionalidad aceptables, pero por diversas causas han ido perdiendo el interés o el empuje de sus desarrolladores, y su ritmo de desarrollo ha decrecido. Sin embargo, varios conceptos muy importantes han nacido bajo este tipo de sistemas de archivos, algunos de los cuales (como el de las *fotografías*) se han ido aplicando a sistemas de archivo estándar.

Por otro lado, dado el fuerte crecimiento que están registrando los medios de almacenamiento de estado sólido (en la sección K.2 se abordarán algunas de sus particularidades), y dado que estos sistemas logran aprovechan mejor varias de sus características, es probable que el interés en estos sistemas de archivos resurja.

Capítulo 46

Ejercicios

46.1. Preguntas de autoevaluación

1. Asuma el siguiente sistema de archivos basado en *asignación indexada*. Cada *cluster* mide 4 096 bytes, y el apuntador a un bloque requiere 32 bits (4 bytes). Dados los metadatos que van a almacenarse en el i-nodo del archivo, dentro del i-nodo principal puede guardar 24 apuntadores directos, y está considerando permitir indirección sencilla y doble.

   ¿Cuál es el tamaño máximo de archivo que podrá manejar este sistema de archivos?

2. El sistema de archivos *Reiser* está basado en la *asignación indexada*, pero introduce un nuevo concepto: las *colitas* (*tails*). Éstas permiten que distintos archivos pequeños, de hasta 4 KB, puedan ser almacenados *en un mismo cluster*. Esto permite a Reiser ahorrar espacio en disco (se estima que logra una eficiencia hasta 5% mayor que sistemas que no emplean esta técnica), y además reduce el tiempo requerido para recuperar estos archivos, dado que los datos pueden almacenarse en el mismo i-nodo que alguna de sus bibliotecas, sin requerir leer ningún bloque adicional.

   Reiser se popularizó alrededor del 2001, pero su desarrollo se ha detenido, y esta característica en particular no ha sido adoptada más que por unos pocos sistemas de archivos (UFS2 y BTRFS). ¿Qué problemas encuentra con el planteamiento de las *colitas*?

3. Describa el funcionamiento de un *sistema de archivos con bitácora* (*journaling file system*). ¿Cómo nos asegura que el sistema se mantendrá consistente después de una interrupción abrupta del suministro eléctrico?

46.2. Temas de investigación sugeridos

Desduplicación Una de las características que ofrecen varios sistemas operativos de última generación es la *desduplicación*, presentada en la sección 43.5.3: la detección de sectores idénticos pertenecientes a más de un archivo, para evitar repetirlos varias veces en el disco (es un fenómeno que ocurre mucho más de lo que esperaríamos). Esta detección se realiza típicamente por medio de *hashes criptográficos*.

¿Cómo opera un poco más a detalle este mecanismo, qué tan confiable es? (esto es, ¿se recomienda utilizar ya en sistemas en producción?) ¿Qué tan eficiente resulta con los datos que se encuentran típicamente, qué pasa con el espacio libre reportado al sistema, no se cae en riesgos de *sobrecomprometimiento* (*overcommit*)? Respecto a su forma de operación, ¿qué diferencias tienen la *desduplicación en línea* y la *desduplicación fuera de línea* (*online deduplication*, *offline deduplication*), Cómo opera el *hash criptográfico*? Y de forma general, ¿hay veces que este mecanismo resulte insuficiente, qué alternativas hay?

Como referencia informal al respecto, léase el siguiente hilo de discusión al respecto en la lista de DebConf (el Congreso de Debian):

`http://lists.debconf.org/lurker/message/20130813.100610.f38cd67f.en.html`

46.3. Lecturas relacionadas

- Dominic Giampaolo (1999). *Practical File System Design with the Be File System*. Morgan Kaufmann Publishers Inc. ISBN: 1-55860-497-9. URL: http://www.nobius.org/~dbg/practical-file-system-design.pdf. El autor fue parte del equipo que implementó el sistema operativo BeOS, un sistema de alto rendimiento pensado para ejecutar en estaciones de alto rendimiento, particularmente enfocado al video. El proyecto fracasó a la larga, y BeOS (así como BeFS, el sistema que describe) ya no se utilizan. Este libro, descargable desde el sitio Web de su autor, tiene una muy buena descripción de varios sistemas de archivos, y aborda a profundidad técnicas que hace 15 años eran verdaderamente novedosas y hoy forman parte de casi todos los sistemas de archivos con uso amplio, e incluso algunas que no se han logrado implementar y que BeFS sí ofrecía.

- T. J. Killian (1984). «Processes as Files». En: USENIX Association. Salt Lake City. URL: http://lucasvr.gobolinux.org/etc/Killian84-Procfs-USENIX.pdf

- David A. Huffman (1952). «A method for the construction of Minimum-Redundancy Codes». En: vol. 40. I. R. E. URL: http://compression.graphicon.ru/download/articles/huff/huffman_1952_minimum-redundancy-codes.pdf

- Mufti Mohammed (2007). *FAT Root Directory Structure on Floppy Disk and File Information*. Codeguru. URL: http://www.codeguru.com/cpp/cpp/cpp_mfc/files/article.php/c13831

- Peter Clark (2001). *Beginning to see the light — FAT16, File Allocation Table: 16bit*. URL: http://www.beginningtoseethelight.org/fat16)

- Marshall K. McKusick y col. (1984). «A Fast File System for UNIX». En: *ACM Trans. Comput. Syst.* 2.3, págs. 181-197. ISSN: 0734-2071. DOI: 10.1145/989.990. URL: http://doi.acm.org/10.1145/989.990

- Marshall Kirk McKusick (2012). «Disks from the Perspective of a File System». En: *ACM Queue* 10.9. URL: https://queue.acm.org/detail.cfm?id=2367378

    - Traducción al español: César Yáñez (2013). *Los discos desde la perspectiva de un Sistema de Archivos*. URL: http://cyanezfdz.me/2013/07/04/los-discos-desde-la-perspectiva-de-un-sistema-de-archivos-es.html

- Mendel Rosenblum (1992). «The Design and Implementation of a Log-structured File System». Tesis doct. EECS Department, University of California, Berkeley. URL: http://www.eecs.berkeley.edu/Pubs/TechRpts/1992/6267.html

- Dave Poirier (2001-2011). *The Second Extended File System: Internal Layout*. URL: http://www.nongnu.org/ext2-doc/

- César Yáñez (2012). *NILFS2 en Linux*. URL: http://cyanezfdz.me/2012/08/30/nilfs2-en-linux-es.html

- Jonathan Corbet (2012a). *A hash-based DoS attack on Btrfs*. URL: http://lwn.net/Articles/529077/,

- Christoph Berg. *Default /etc/apache2/mods-available/disk_cache.conf is incompatible with ext3*. Debian Project. URL: bugs.debian.org/682840. Reporte de fallo de Debian ilustrando los límites en números de archivos para Ext3.

- John S. Heidemann y Gerald J. Popek (1994). «File-system Development with Stackable Layers». En: *ACM Trans. Comput. Syst.* 12.1, págs. 58-89. ISSN: 0734-2071. DOI: 10.1145/174613.174616. URL: http://doi.acm.org/10.1145/174613.174616
- Thomas E. Anderson y col. (1996). «Serverless Network File Systems». En: *ACM Trans. Comput. Syst.* 14.1, págs. 41-79. ISSN: 0734-2071. DOI: 10.1145/225535.225537. URL: http://doi.acm.org/10.1145/225535.225537
- Valerie Aurora (2009). *Log-structured file systems: There's one in every SSD*. URL: https://lwn.net/Articles/353411/